МИНОБРНАУКИ РОССИИ
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«МИРЭА – Российский технологический университет»
РТУ МИРЭА

Институт искусственного интеллекта
Кафедра промышленной информатики

РАБОТА ДОПУЩЕНА К ЗАЩИТЕ

Заведующий
кафедрой_____В.А. Холопов

«01»____июня____ 2026 г.

ВЫПУСКНАЯ КВАЛИФИКАЦИОННАЯ РАБОТА

по направлению подготовки бакалавров 09.03.01 Информатика и вычислительная техника

На тему: «Информационно-управляющая система выращивания растений с использованием технологий гидропоники и аэропоники»

Обучающийся _____ *Бурукина Анна Александровна*
подпись *Фамилия, имя, отчество*

шифр 22К0458
группа КВБО-02-22

Руководитель работы _____ *доцент* *Курнасов Е.В.*
подпись

Консультант _____ *старший преподаватель* *Володина А.М.*
подпись

Нормоконтроль _____ *старший преподаватель* *Володина А.М.*
подпись

Москва 2026 г.

МИНОБРНАУКИ РОССИИ
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«МИРЭА – Российский технологический университет»
РТУ МИРЭА

Институт искусственного интеллекта

Кафедра промышленной информатики

СОГЛАСОВАНО	УТВЕРЖДАЮ
Заведующий кафедрой _____ *подпись*	Директор института _____ *подпись*
Холопов Владимир Анатольевич	Магомедов Шамиль Гасангусейнович
«20» апреля 2026 г.	«20» апреля 2026 г.

ЗАДАНИЕ
на выполнение выпускной квалификационной работы бакалавра

Обучающийся Бурукина Анна Александровна
Фамилия Имя Отчество

Шифр 22К0458

Направление подготовки 09.03.01 Информатика и вычислительная техника
индекс направления *наименование направления*

Группа КВБО-02-22

1. Тема выпускной квалификационной работы: «Информационно-управляющая система выращивания растений с использованием технологий гидропоники и аэропоники».

2.Цель и задачи выпускной квалификационной работы

Цель работы: повышение эффективности процесса выращивания растений в замкнутой установке с использованием гидропонной и аэропонной технологий.

Задачи работы: проанализировать особенности гидропонной и аэропонной технологий выращивания растений; определить параметры, влияющие на обеспечение благоприятных условий выращивания; сформировать требования к системе мониторинга и управления условиями выращивания; выполнить моделирование элементов информационно-управляющей системы (ИУС); разработать архитектуру и алгоритм работы ИУС; разработать базу данных, компоненты программного обеспечения и интерфейс оператора.

3. Этапы выпускной квалификационной работы

№ этапа	Содержание этапа выпускной квалификационной работы	Результат выполнения этапа ВКР	Срок выполнения
1	Технологический раздел		27.04.2026
1.1	Разработка технологического процесса выращивания растений в совмещенной гидропонно-аэропонной системе		
1.2	Описание оборудования, применяемого в разрабатываемой системе		
1.3	Определение параметров контроля и регулирования		
1.4	Определение требований к разрабатываемой информационно-управляющей системы		
1.5	Постановка цели и задач ВКР		
2	Проектный раздел		04.05.2026
2.1	Моделирование элементов информационно-управляющей системы		
2.2	Разработка архитектуры программного обеспечения		
2.3	Разработка алгоритмов работы программных компонентов информационно-управляющей системы		
3	Программный раздел		06.05.2026
3.1	Разработка базы данных		
3.2	Разработка кода программных компонентов системы		
3.3	Разработка пользовательского интерфейса		
4	Аннотация, введение, заключение, список источников		14.05.2026
5	Презентация		15.05.2026
6	Нормоконтроль		16.05.2026

4. Перечень разрабатываемых документов и графических материалов

Пояснительная записка, презентация в PowerPoint.

5. Руководитель и консультанты выпускной квалификационной работы

Функциональные обязанности	Должность в Университете	Фамилия, имя, отчество	Подпись
Руководитель ВКР	Доцент	Курнасов Евгений Вячеславович	
Консультант специальной части ВКР	Старший преподаватель	Володина Анна Михайловна	

Задание выдал
Руководитель
ВКР:_____
подпись
«20» апреля 2026 г.

Задание принял к исполнению
Обучающийся:_____
подпись
«20» апреля 2026 г.

АННОТАЦИЯ

Информационно-управляющая система выращивания растений в совмещенной гидропонно-аэропонной установке предназначена для автоматизации контроля технологических параметров, поддержания благоприятных условий роста и повышения эффективности беспочвенного выращивания листового салата. Актуальность разработки обусловлена необходимостью развития технологий растениеводства в контролируемой среде, позволяющих рационально использовать водные и земельные ресурсы, обеспечивать стабильные условия выращивания растений и повышать качество управления технологическим процессом.

Работа состоит из введения, трех глав, заключения, списка использованных источников и приложений.

В первой главе рассмотрена разработка технологического процесса выращивания листового салата в совмещенной гидропонно-аэропонной системе, описано оборудование, применяемое в системе, определены параметры контроля и регулирования, а также сформулированы требования к разрабатываемой информационно-управляющей системе.

Во второй главе выполнено моделирование элементов информационно-управляющей системы в нотациях IDEF0, DFD и BPMN, рассмотрена архитектура программного обеспечения с использованием нотации Arcimate, а также описаны алгоритмы работы системы.

В третьей главе разработана база данных, описана реализация программных компонентов системы и пользовательский интерфейс веб-приложения, предназначенного для мониторинга параметров, изменения уставок, запуска принудительного полива, просмотра измерений, оборудования и журнала ошибок.

В заключении обобщены результаты выполненной работы и сформулированы выводы по разработке информационно-управляющей системы.

Источниками проведения исследования являются научные статьи, учебные пособия, нормативные документы и электронные ресурсы, посвященные технологиям гидропоники и аэропоники, выращиванию салата в контролируемой среде и автоматизации процессов растениеводства. Также использованы материалы по контролю параметров микроклимата, кислотности и электропроводности питательного раствора, применению индекса NDVI для оценки состояния растений, проектированию информационно-управляющих систем, моделированию бизнес-процессов, разработке баз данных, архитектуры программного обеспечения и клиент-серверных веб-приложений.

Ключевые слова: информационно-управляющая система, гидропоника, аэропоника, совмещенная гидропонно-аэропонная установка, контролируемая среда, микроклимат, питательный раствор, NDVI.

СОДЕРЖАНИЕ

ВВЕДЕНИЕ 7
1 ТЕХНОЛОГИЧЕСКИЙ РАЗДЕЛ 9
1.1 Разработка технологического процесса выращивания растений в совмещенной гидропонно-аэропонной системе 9
1.2 Описание оборудования, применяемого в разрабатываемой системе 12
1.3 Определение параметров контроля и регулирования 14
1.4 Определение требований к разрабатываемой информационно-управляющей системе…… 15
1.5 Постановка цели и задач ВКР 22
2 ПРОЕКТНЫЙ РАЗДЕЛ 23
2.1 Моделирование элементов информационно-управляющей системы 23
2.2 Разработка архитектуры программного обеспечения 36
2.3 Разработка алгоритма работы информационно-управляющей системы 39
2.3.1 Алгоритм запуска принудительного полива 39
2.3.2 Алгоритм обновления уставок выбранного параметра 40
3 ПРОГРАММНЫЙ РАЗДЕЛ 42
3.1 Разработка базы данных 42
3.2 Разработка кода программных компонентов системы 45
3.3 Разработка пользовательского интерфейса 54
ЗАКЛЮЧЕНИЕ 61
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ 63
Приложение А 67
Приложение Б 83
Приложение В 86
Приложение Г 94
Приложение Д 101

ВВЕДЕНИЕ

По оценкам, к 2050 году численность населения Земли составит примерно 10 миллиардов человек. Удовлетворение потребностей растущего населения в калориях и питательных веществах является серьезной задачей для человечества. Растущее население, основанное на городской агломерации, в сочетании с ограниченными природными ресурсами и глобальным потеплением увеличивает остроту этой проблемы, создавая угрозу безопасности пищевых продуктов во всем мире. Традиционное сельское хозяйство плохо подходит для решения этой задачи, поскольку оно оказывает негативное воздействие на окружающую среду, неэффективно использует большое количество воды, высокие концентрации питательных веществ и пестицидов в водном стоке, высокие выбросы парниковых газов и усиливает деградацию почв и эрозию. По данным ФАО размеры пахотной земли на душу населения сократятся на треть в 2050 г. по сравнению с 1970 годом. Другими словами, земель для сельскохозяйственного назначения будет меньше. Еще одной проблемой является то, что 1/3 населения планеты страдает от голода и недоедания. Для решения этих проблем необходимо разработать устойчивые меры по производству и поставке продовольствия.

Высокие темпы производства продовольствия, которые существуют сегодня, к сожалению, могут не сохраниться к 2050 году. Следовательно, это требует изучения альтернативных подходов, таких как выращивание продуктов питания в закрытых помещениях. Именно такие подходы позволяют увеличить объемы производства продуктов питания и их качество. Это связано с тем, что производители могут контролировать и управлять факторами окружающей среды в процессе производства — температурой, влажностью, почвенными факторами и др. Такие технологии квалифицируют как сельское хозяйство в контролируемой среде, растениеводческие фабрики, системы замкнутого цикла, беспочвенное выращивание и вертикальные фермы.

Одним из перспективных направлений развития растениеводства в контролируемой среде является использование беспочвенных технологий выращивания, к которым относятся гидропоника и аэропоника.

Гидропоника повышает образование биомассы и снижает использование удобрений и воды. Аэропоника еще больше повышает эффективность выращивания за счет распыления питательного раствора непосредственно на подвешенные в воздухе корни. Это позволяет оптимизировать баланс кислорода и воды в ризосфере.

Листовой салат (*Lactuca sativa L.*) является одной из наиболее распространенных овощных культур, выращиваемых в контролируемой среде с искусственным освещением.

Продуктивность салата в беспочвенных системах зависит от баланса питательных веществ и контроля условий среды.

1 ТЕХНОЛОГИЧЕСКИЙ РАЗДЕЛ

1.1 Разработка технологического процесса выращивания растений в совмещенной гидропонно-аэропонной системе

Технологический процесс выращивания листового салата в совмещенной гидропонно-аэропонной системе разрабатывается с учетом необходимости поддержания благоприятных условий роста и возможности сопоставления двух технологий выращивания в рамках одной установки. Для этого система организуется в виде единого производственного помещения с общим микроклиматом, внутри которого выделяются гидропонная и аэропонная зоны. Такое решение позволяет минимизировать влияние внешних факторов и обеспечить одинаковые условия воздушной среды для обеих технологий.

Растения размещаются в сетчатых горшках на многоярусных стеллажах, обеспечивающих компактное расположение зон выращивания в пределах одного помещения (Рисунок 1.1).



Рисунок 1.1 — Размещение растений на стеллаже

Для каждого яруса предусмотривается собственная линия освещения. Освещение осуществляется светильниками полного спектра, работающими по фотопериоду 18/6. Предусматривается независимое включение и выключение освещения по ярусам, что позволяет отключать неиспользуемые зоны и снижать энергопотребление.

Важной особенностью организации освещения является каскадное включение светильников в утренний период: ярусы запускаются последовательно с временным интервалом, что ограничивает пусковые токи и предотвращает перегрузку системы электроснабжения при одновременном старте большого числа источников света.

Поддержание микроклимата является одной из основных частей технологического процесса. Замкнутый характер системы упрощает поддержание стабильных параметров среды и позволяет создавать условия, благоприятные для роста растений.

В помещении контролируются температура, относительная влажность воздуха и концентрация углекислого газа, которые существенно влияют на рост листового салата.

Температура является одним из основных факторов, влияющих на рост и развитие растений. Она напрямую влияет на температуру листа и воздуха и косвенно влияет на поглощение и транспорт воды и питательных веществ через корни. Оптимальная температура необходима для быстрого роста и высокого накопления органических веществ. При слишком высокой температуре ферменты фотосинтеза могут разрушаться и терять активность.

Относительная влажность воздуха является важным фактором, влияющим на работу устьиц и рост растений. При низкой относительной влажности испарение с листьев усиливается. Это приводит к потере воды, закрытию устьиц и прекращению поступления углекислого газа, в результате чего снижается интенсивность фотосинтеза. А при повышенной влажности нарушается работа устьиц и процессы водообмена растений.

Обогащение воздушной среды углекислым газом используется для стимуляции роста салата, увеличения урожайности и повышения интенсивности фотосинтеза. Углекислый газ положительно влияет на фотосинтез только до определенного уровня концентрации. При дальнейшем повышении концентрации интенсивность фотосинтеза перестает увеличиваться и затем снижается.

Регулирование концентрации углекислого газа осуществляется дозированным подмесом CO_2 из баллонов по заданному временному регламенту.

Для замкнутой системы выращивания растений характерна повышенная влажность, возникающая вследствие испарения влаги и последующей конденсации. При работе кондиционеров, удаление влаги осуществляется за счет конденсации на теплообменниках, а при отсутствии потребности в охлаждении применяется осушение воздуха.

При низких температурах используется тепловентиляция, а в теплый период требуется активное охлаждение из-за высокой внутренней тепловой нагрузки, создаваемой осветительными установками, электрооборудованием и осушителями.

Перед началом полива осуществляется подготовка питательного раствора. На данном этапе контролируются кислотность (pH) и электропроводность (EC) раствора. Контроль pH необходим для поддержания доступности питательных веществ для растений, а показатель EC характеризует концентрацию растворенных минеральных веществ.

Полив реализуется двумя способами.

В гидропонной зоне используется метод периодического затопления. При данном способе питательный раствор через заданные промежутки времени подается в лоток с растениями до установленного уровня, после чего сливается обратно в резервуар. Такой режим обеспечивает поступление к корням влаги и питательных веществ, а в промежутках между поливами сохраняет доступ кислорода к корневой системе.

В разрабатываемой системе также используется ультразвуковая аэропоника, при которой питательный раствор преобразуется в мелкодисперсный туман и подается в закрытую корневую камеру.

Такой способ подачи обеспечивает поступление влаги к корневой системе и сохраняет хорошую аэрацию. Для предотвращения пересыхания корней в аэропонной зоне контролируется влажность в корневой камере.

Для аэропоники критически важно обеспечить бесперебойную подачу питательного раствора, так как даже кратковременные перерывы полива могут привести к ухудшению состояния растений.

Для сопоставления эффективности технологий и оценки физиологического состояния растений используется вегетационный индекс NDVI. Нормализованный разностный вегетационный индекс NDVI является важным показателем, позволяющим оценивать здоровье растений, жизнеспособность экосистемы и урожайность. Сравнивая отражение красного и ближнего инфракрасного света, NDVI предоставляет ценную информацию о динамике растительности (Рисунок 1.2). NDVI позволяет фиксировать ухудшение физиологического состояния растений до появления внешних признаков. На основе динамики изменения индекса NDVI выполняется корректировка регламента полива при выявлении устойчивого снижения индекса или отклонения от эталонной траектории роста.

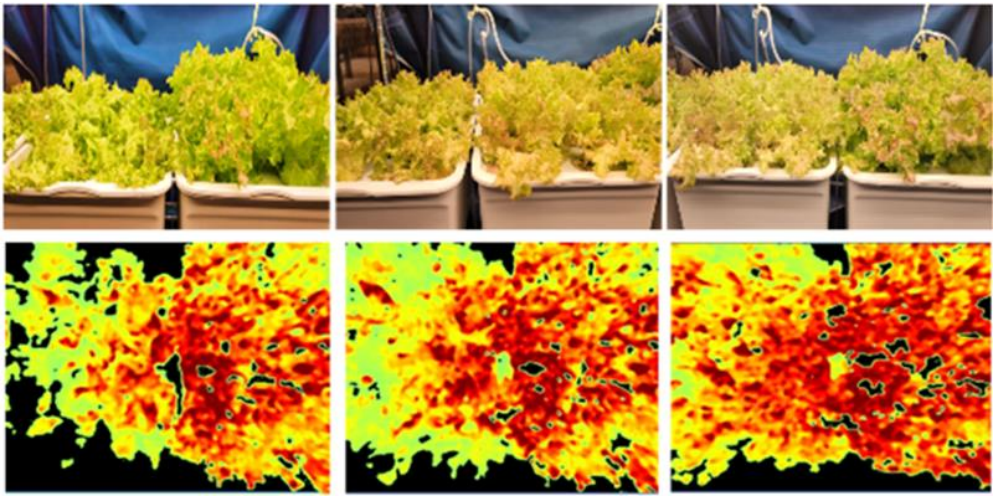


Рисунок 1.2 — Сопоставление внешнего состояния салата и карт распределения NDVI

1.2 Описание оборудования, применяемого в разрабатываемой системе

Для контроля технологического процесса в системе используется комплекс датчиков, предназначенных для мониторинга параметров среды и состояния растений.

Основной состав оборудования, применяемого в системе, представлен в Таблице 1.1.

Таблица 1.1 — Оборудование

№	Параметр	Прибор	Тип прибора	Границы измерения	Технические характеристики
1	**Влажность**	Комбинированный цифровой датчик	Настенный комбинированный датчик WB-MSW v.4	0-95 % (±3 %)	ИК-передатчик с дальностью до 10 м Интерфейс: RS-485, Modbus RTU
2	**Температура**			-40-+80 °C (±0.5 °C)	
3	**Концентрация углекислого газа**			400-10000 ppm	
4	**Уровень кислотности раствора**	Промышленный контроллер pH	AQUA-LAB AQ-150	0.00 ~ 14.00	Выходы:4-20 мА токовая петла Погрешность 0.01
5	**Электропроводность (EC) раствора**	Промышленный кондуктометр	AQUA-LAB AQ-EC150-RS485	0,05-200 мСм/см	Выходы: RS-485 Modbus/RTU и 4-20мА
6	**Влажность в зоне корней**	Высококачественный датчик влажности и температуры	EE310	0-100%	Выход RS-485, ModBUS RTU, Ethernet, Modbus TCP, 4-20 мА, 0-10 В, 0-5 В
7	**Индекс NDVI**	Двухканальный датчик для определения индекса NDVI методом измерения интенсивности сигнала при длинах волн 650 нм и 810 нм.	NDVI S2-112-SS	2х полный солнечный свет	Выход (чувствительность): 12,5 мВ на Вт $м^{-2}$ $нм^{-1}$ $ср^{-1}$ (красный); 25 мВ на Вт $м^{-2}$ $нм^{-1}$ $ср^{-1}$ (БИК)

Для обеспечения регулирования технологического процесса в системе используются исполнительные устройства, применение которых необходимо для поддержания заданных условий выращивания растений и своевременной корректировки параметров при отклонении от установленных диапазонов значений.

Перечень исполнительных устройств, применяемых в системе представлен в Таблица 1.2.

Таблица 1.2 — Исполнительные устройства

№	Исполнительное устройство	Тип устройства
1	Ballu BFH-S10	Тепловентилятор
2	Daikin FTXM35R	Кондиционер
3	Trotec TTK 350 S	Осушитель воздуха
4	Danfoss EV220B	Электромагнитный клапан подачи CO2
5	Reef Octopus VarioS-4 DC Pump	Насос подачи раствора
6	ASCO 238610 Series Solenoid Valve	Электромагнитный клапан подачи раствора
7	HaoPai 24V Ultrasonic Mist Maker	Ультразвуковой распылитель аэропонной системы
8	Purity PZ 50-125/40	Дренажный насос

1.3 Определение параметров контроля и регулирования

Эффективная работа гидропонно-аэропонной системы возможна при постоянном контроле и регулировании основных технологических параметров.

Список параметров представлен в Таблице 1.3.

Таблица 1.3 — Контролируемые параметры

Группа параметров	Наименование параметра	Диапазон	Единица измерения
Параметры среды	Температура воздуха	22-25	%
	Влажность воздуха	70-80	°C
	Концентрация CO_2	1000-1500	Ppm (миллионнная доля)
Параметры раствора	Кислотность раствора (pH)	5,8-6,2	pH
	Электропроводность раствора (EC)	1,6-2,2	мС/см
Аэропоника	Влажность в зоне корней	95-100	%
Параметр состояния растений	NDVI (вегетационный индекс)	0.5–0.75	—

Для контроля температуры, относительной влажности воздуха и уровня концентрации углекислого газа используется настенный комбинированный цифровой датчик WB-MSW v.4.

Для контроля параметров питательного раствора применяются датчики кислотности и электропроводности, обеспечивающие мониторинг химического состава раствора и концентрации растворенных питательных веществ.

В аэропонной подсистеме контролируется влажность в зоне корней, поддерживаемая на высоком уровне для предотвращения пересыхания корневой системы при ультразвуковом распылении питательного раствора.

Для оценки состояния растений применяется двухканальный датчик NDVI S2-112-SS, который определяет вегетационный индекс NDVI методом измерения интенсивности отраженного света при длинах волн 650 нм (красный спектр) и 810 нм (ближний инфракрасный спектр).

1.4 Определение требований к разрабатываемой информационно-управляющей системе

1.4.1 Нефункциональные требования

1. Требования к производительности:

- система должна обеспечивать работу в реальном времени с периодом опроса датчиков не более 1 секунды;
- время реакции на изменение параметра не должно превышать 3 секунды;
- среднее время отображения обновленных данных на панели оператора — не более 1 секунды;
- система должна обеспечивать хранение не менее 3 месяцев архивных данных без потери целостности и производительности.

2. Требования к надежности и отказоустойчивости:

- система должна обеспечивать автоматический перезапуск сервисов после отключения питания;
- предусматривается аварийный режим работы — поддержание текущих параметров без операторского вмешательства при временном отключении системы.

3. Требования к безопасности оборудования.

Требования должны соответствовать регламенту предприятия

4. Требования к удобству использования (эргономике):

- все основные параметры должны отображаться на едином экране мониторинга;
- в интерфейсе предусмотрена возможность масштабирования для различных разрешений экранов операторских станций.

5. Требования к информационной совместимости:

- система должна поддерживать единый протокол обмена данными;
- система должна содержать модуль конвертации данных для преобразования данных в единый формат.

6. Требования к масштабируемости и модифицируемости:

- архитектура системы должна позволять добавление новых датчиков и исполнительных устройств без остановки работы;
- возможность подключения дополнительных тепличных секций и зон выращивания без изменения базового программного ядра;
- расширение функционала должно выполняться через обновления модулей без полной переустановки системы;
- база данных должна поддерживать добавление таблиц и атрибутов без нарушения структуры существующих данных.

7. Требования к диагностике и обслуживанию:

- система должна обеспечивать автоматическую диагностику состояния оборудования и каналов связи;
- все неисправности должны фиксироваться в электронном журнале инцидентов с указанием времени, узла и причины;
- предусмотрена система уведомлений о неисправностях.

8. Требования к эксплуатации и условиям функционирования:

- система должна работать круглосуточно в автоматическом режиме;

- допустимая температура эксплуатации оборудования: от +10 до +30 °C;
- относительная влажность: до 90 % без конденсации;
- электропитание: 220 В, 50 Гц, с защитой от перепадов напряжения;
- средства ввода-вывода и операторы защищены от поражения электрическим током;
- при отключении питания система должна сохранять все данные и восстанавливаться без вмешательства пользователя.

9. Требования к хранению и архивированию данных:

- все технологические данные и логи должны храниться не менее 12 месяцев;
- архивы параметров должны автоматически создаваться и храниться на выделенном сервере хранения;
- должна быть обеспечена возможность поиска по временным меткам и параметрам;
- архивы должны быть доступны для экспорта и восстановления;
- формат хранения данных — SQL + резервные CSV-копии.

1.4.2 Требования к обеспечениям

1. Требования к информационному обеспечению.

Информационное обеспечение должно обеспечивать сбор, хранение, передачу, обработку и представление данных о параметрах технологического процесса в реальном времени.

Требования:

- информация должна храниться в реляционной СУБД (MySQL);
- должна быть реализована функция резервного копирования и восстановления базы данных;
- система должна предотвращать случайное или преднамеренное изменение сохраненных данных датчиков;

- система должна обеспечивать ведение централизованной базы данных, содержащей технологические параметры; состояние оборудования и исполнительных механизмов; результаты контроля и анализа; журналы событий и действий пользователей;
- все изменения данных должны фиксироваться в журнале аудита с указанием пользователя и времени;
- должно обеспечиваться автоматическое архивирование данных за период не менее 3 месяцев;
- форматы хранения и экспорта — CSV, XML, JSON;
- должен быть реализован механизм защиты от дублирования и потери данных при передаче;
- должна обеспечиваться верификация данных датчиков перед их сохранением в БД;
- для визуализации необходимо обеспечивать представление данных в виде графиков, таблиц и отчетов;
- должно обеспечиваться реальное время обработки данных (период опроса ≤ 1 сек);
- все расчетные операции выполняются в реальном времени с обновлением каждые 3–5 сек;
- результаты анализа сохраняются в базе данных и используются подсистемой визуализации для построения отчетов.

Информационное взаимодействие.

Система должна обеспечивать обмен данными по сетям Ethernet и RS-485 с протоколами Modbus RTU/TCP**.**

2. Требования к программному обеспечению.

Программное обеспечение должно обеспечивать функционирование подсистем визуализации, контроля, сбора информации и управления.

Требования:

- программное обеспечение должно быть разделено на модули визуализации, контроля, сбора информации и управления;

- ПО должно поддерживать обновление модулей без остановки основной системы;
- при сбое любого компонента должна выполняться его автоматическая перезагрузка;
- ПО должно обеспечивать работу под ОС Linux и поддержку сетевых протоколов WebSocket/HTTP.

3. Требования к техническому обеспечению.

Техническое обеспечение включает совокупность устройств, обеспечивающих сбор, обработку и передачу информации, а также выполнение управляющих воздействий.

Требования:

1. Минимальные требования к серверу:
   - процессор: 4 ядра / 8 потоков (Intel Xeon / AMD Ryzen);
   - оперативная память: не менее 16 ГБ;
   - накопители: SSD 256 ГБ (система), HDD/SSD 500 ГБ (архив);
   - сеть: 1×Gigabit Ethernet;
   - ОС: Windows Server / Linux;
   - источник бесперебойного питания: автономность ≥ 30 мин.
2. Минимальные требования к ПК оператор:
   - процессор: Intel Core i3 / AMD Ryzen 3;
   - оперативная память: не менее 8 ГБ;
   - накопитель: SSD 120–240 ГБ;
   - монитор: 22–24″, Full HD;
   - сеть: Ethernet 100/1000 Мбит/с;
   - ОС: Windows 10/11 или Linux.

4. Все датчики должны соответствовать паспортным заводским характеристикам и обеспечивать измерения в пределах, необходимых для соблюдения агротехнологических требований.

5. Требования к организационному обеспечению.

Организационное обеспечение должно регламентировать порядок эксплуатации и распределение ролей персонала.

Требования:

- должно проводиться обучение персонала работе с АС и оборудованием;
- должен быть утвержден план профилактического обслуживания и план аварийного реагирования;
- все действия пользователей должны фиксироваться в электронном журнале;
- система должна быть защищена от несанкционированного изменения конфигурации.

6. Требования к метрологическому обеспечению.

Метрологическое обеспечение предназначено для обеспечения точности измерений и достоверности данных.

Требования:

- все средства измерений должны быть сертифицированы и внесены в Государственный реестр СИ РФ;
- поверка и калибровка датчиков должна проводиться не реже 1 раза в год;
- погрешность измерений не должна превышать:
    1. Температуры — ±0,5 °C.
    2. Влажности — ±2 %.
    3. CO_2 — ±50 ppm.
    4. NDVI — ±0,02.
- должно обеспечиваться автоматическое сравнение текущих данных с эталонными диапазонами;
- все результаты поверки должны регистрироваться в журнале калибровок.

7. Требования к правовому обеспечению.

Правовое обеспечение должно гарантировать соответствие системы законодательству РФ и нормативным актам в области автоматизированных систем.

Требования:

- должны соблюдаться санитарные, экологические и энергетические нормы;
- должны выполняться требования по охране труда и технике безопасности при эксплуатации оборудования;
- использование только лицензионного программного обеспечения;
- наличие юридически подтвержденных прав на систему;
- наличие лицензий, ключей активации и разрешений на используемое программное обеспечение;
- соблюдение требований в области защиты информации и корректного хранения данных.

8. Требования к лингвистическому обеспечению.

Лингвистическое обеспечение обеспечивает единообразие терминологии и удобство взаимодействия оператора с системой.

Требования:

- интерфейс пользователя и эксплуатационная документация должны быть выполнены на русском языке;
- должна быть возможность переключения интерфейса на английский язык;
- используемая терминология должна соответствовать отраслевым стандартам: NDVI — индекс состояния растений;
- все предупреждения и сообщения должны быть стандартизированы по форме и цветовой индикации: зеленый — норма; желтый — предупреждение; красный — тревога;
- документация должна использовать унифицированные обозначения и наименования параметров.

9. Требования к математическому обеспечению.

Математическое обеспечение должно обеспечивать выполнение расчетных и аналитических функций АС.

Требования:

- в системе должны быть реализованы алгоритмы: фильтрации и усреднения сигналов с датчиков; анализа трендов и отклонений от эталонных значений; расчета суточных и недельных графиков полива и орошения; формирования управляющих команд для насосов, клапанов и вентиляторов (ПИД-регуляторы);
- все расчетные операции выполняются в реальном времени с обновлением каждые 3–5 сек.;
- должны быть реализованы алгоритмы обнаружения отказов датчиков и расхождений показаний.

1.5 Постановка цели и задач ВКР

Цель работы: повышение эффективности процесса выращивания растений в замкнутой установке с использованием гидропонной и аэропонной технологий.

Задачи работы: проанализировать особенности гидропонной и аэропонной технологий выращивания растений; определить параметры, влияющие на обеспечение благоприятных условий выращивания; сформировать требования к системе мониторинга и управления условиями выращивания; выполнить моделирование элементов информационно-управляющей системы (ИУС); разработать архитектуру и алгоритм работы ИУС; разработать базу данных, компоненты программного обеспечения и интерфейс оператора.

2 ПРОЕКТНЫЙ РАЗДЕЛ

2.1 Моделирование элементов информационно-управляющей системы

2.1.1 Моделирование функциональной структуры системы в нотации IDEF0

Логика функционирования системы отражена с помощью нотации IDEF0. Контекстная диаграмма представлена на Рисунке 2.1.

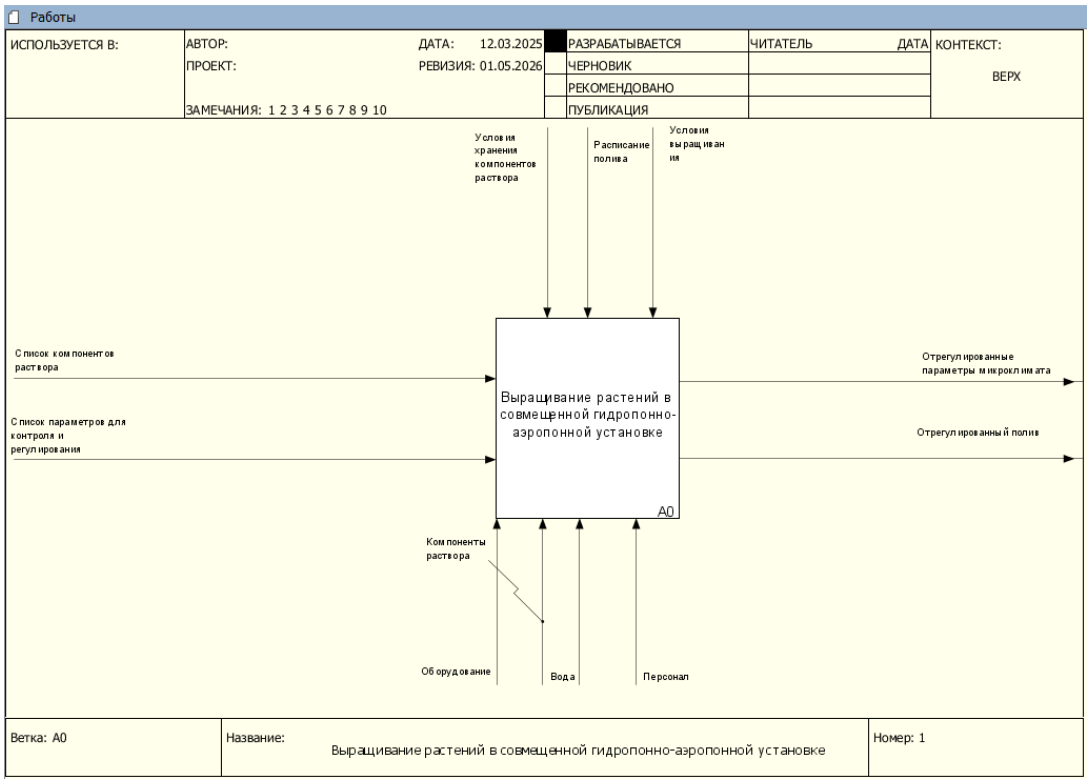


Рисунок 2.1 — Контекстная диаграмма

Процесс выращивания растений с использованием технологий гидропоники и аэропоники разделен на три этапа: подготовка питательного раствора, полив и формирование микроклимата. Персонал с помощью оборудования и необходимых составляющих, руководствуясь условиями хранения компонентов раствора и условиями выращивания растений, осуществляет подготовку раствора для полива.

На этапе полива персонал руководствуется условиями выращивания и расписанием. Излишки раствора используются повторно в следующем цикле полива.

Для создания оптимальных условий выращивания растений в замкнутой экосистеме необходимо обеспечить полный контроль над климатом системы. Персонал с помощью оборудования контролирует параметры среды в соответствии с условиями выращивания растений. В результате осуществления климат-контроля формируются отрегулированные параметры уровня относительной влажности воздуха, температуры и концентрации углекислого газа в системе (Рисунок 2.2).

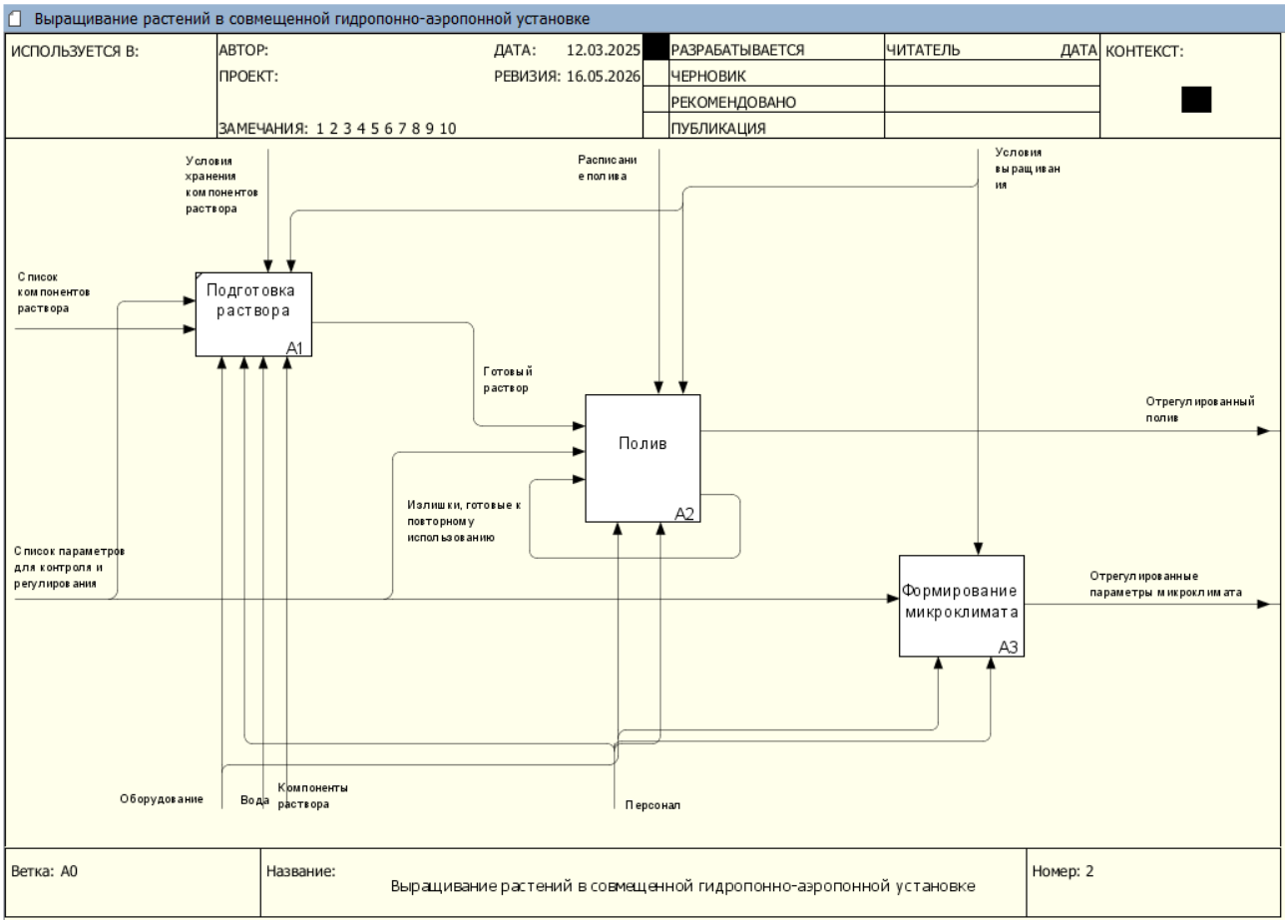


Рисунок 2.2 — Декомпозиция первого уровня

Система полива представлена двумя типами технологий — гидропоника и аэропоника. Готовый раствор подается к растениям в соответствии с условиями выращивания и расписанием полива. Аналогично персонал осуществляет полив растений с помощью аэропонной технологии.

В результате использования данных технологий формируются излишки раствора, готовые к повторному использованию.

Для сравнения эффективности двух технологий и отслеживания состояния растений необходимо контролировать вегетационный индекс NDVI. NDVI выступает как единый индикатор состояния растений независимо от технологии, с помощью которой осуществляется полив. В результате контроля формируются рекомендации по корректировке полива в соответствии с состоянием растений, а также осуществляется регулировка полива (Рисунок 2.3).

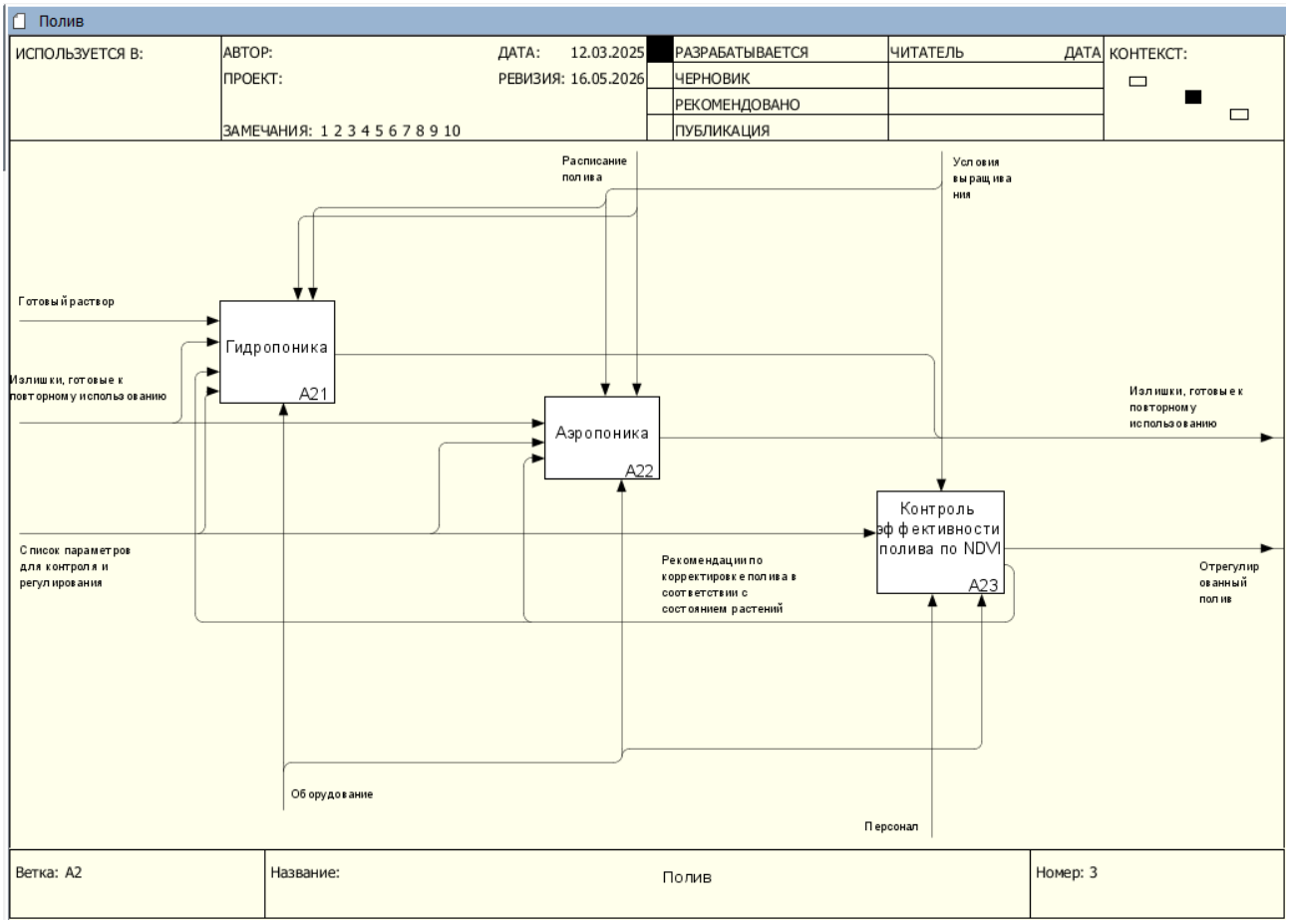


Рисунок 2.3 — Декомпозиция блока полив

Для формирования микроклимата необходимо с помощью оборудования измерить текущие значения контролируемых параметров. Измерение выполняется в соответствии с условиями выращивания растений и на основе списка параметров, подлежащих регулированию.

В результате этого процесса формируются измеренные параметры микроклимата, которые далее передаются на этап контроля.

Затем выполняется контроль параметров микроклимата. На данном этапе измеренные значения сравниваются с заданными уставками параметров.

Если значения параметров не находятся в допустимых пределах, то формируются рекомендации по корректировке микроклимата, на основе которых изменяются уставки параметров. В результате этого процесса формируются отрегулированные параметры микроклимата (Рисунок 2.4).

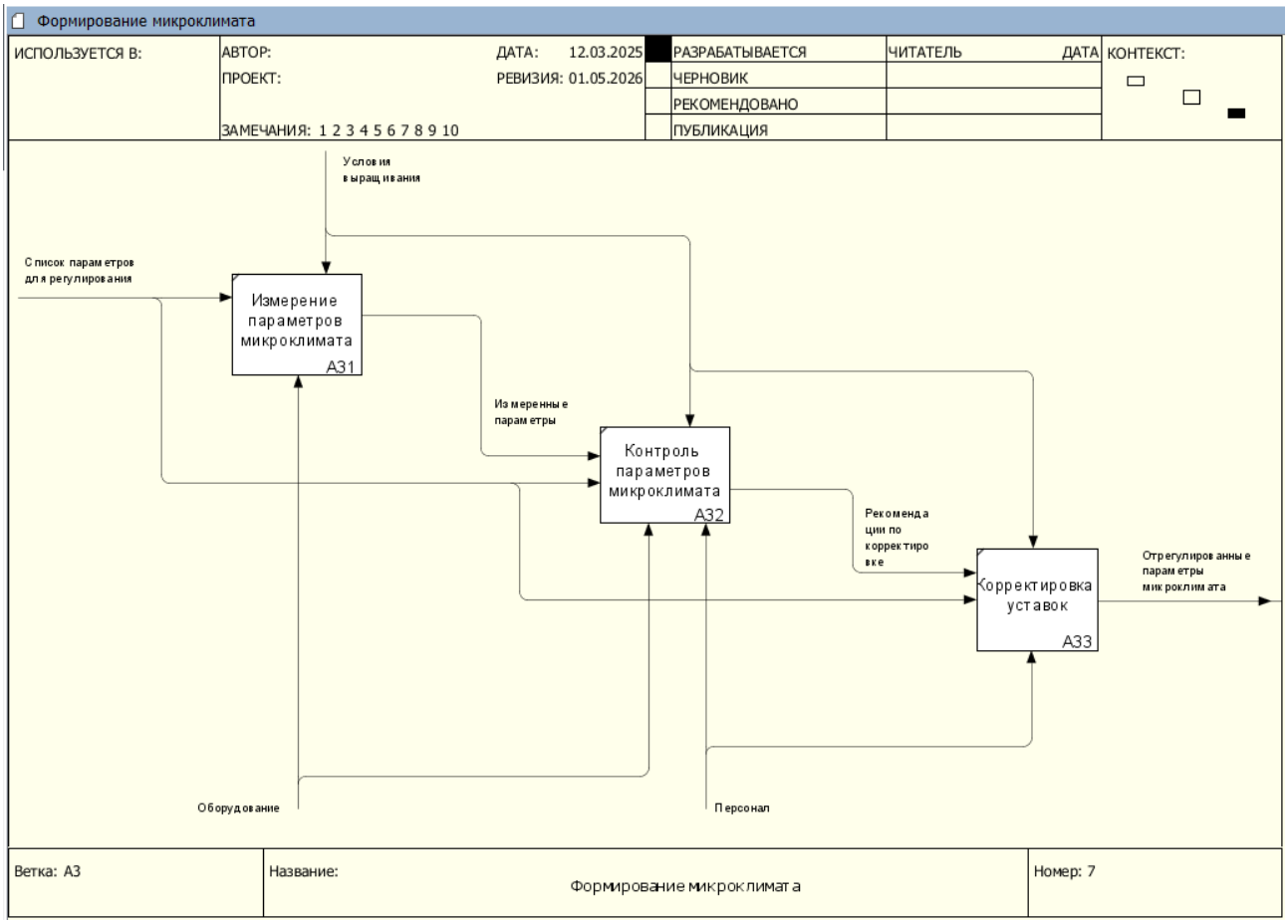


Рисунок 2.4 — Декомпозиция процесса контроля микроклимата

2.1.2 Моделирование информационных потоков системы в нотации DFD

Для отображения информационных потоков автоматизированной системы выращивания растений использовалась нотация DFD.

Контекстная диаграмма представлена на Рисунке 2.5.

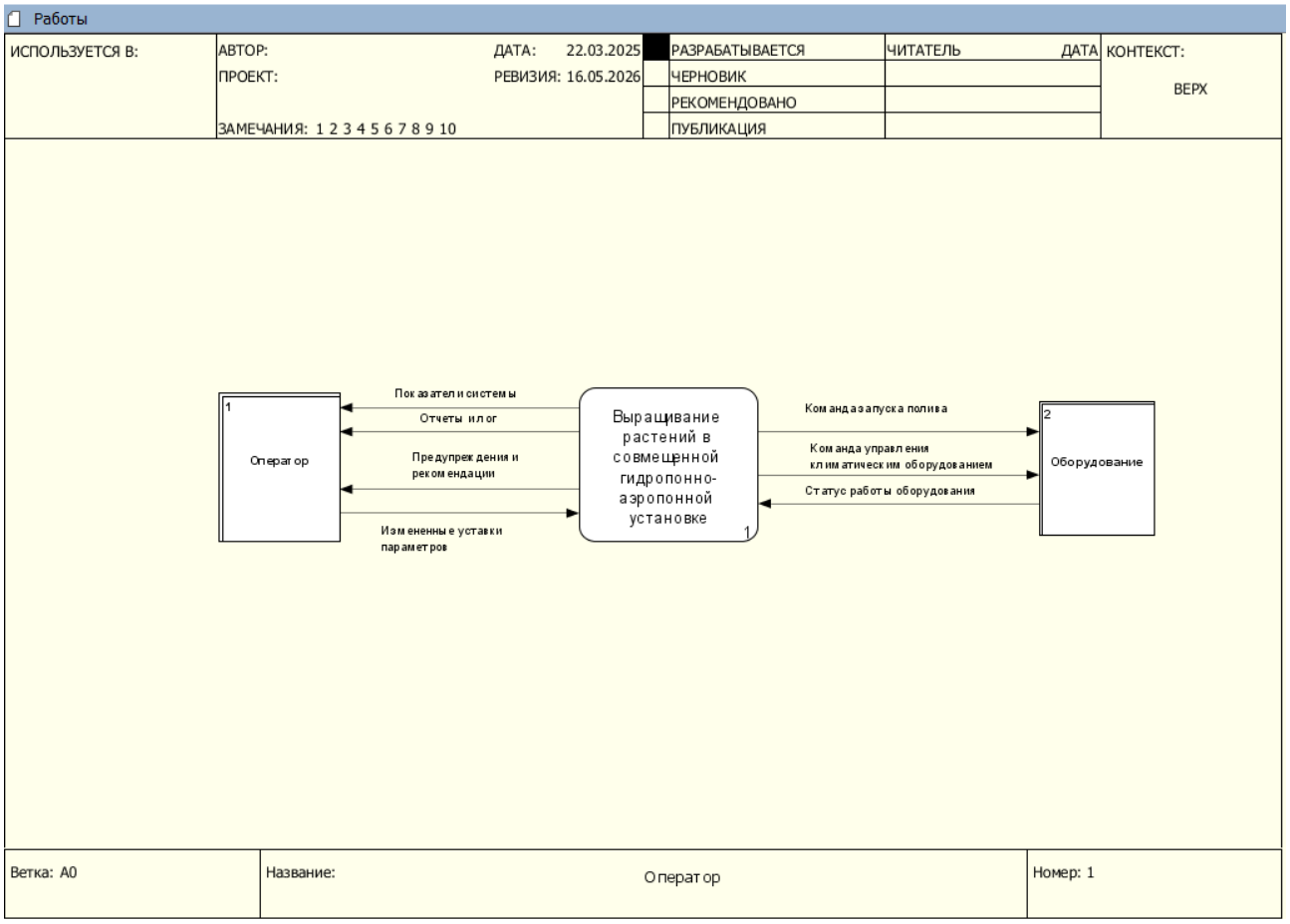


Рисунок 2.5 — Контекстная диаграмма

Система выращивания растений в совмещенной гидропонно-аэропонной установке руководствуется условиями выращивания растений, уставками параметров и обновленными в случае корректировки отклонений уставками параметров. Процесс выращивания растений получает от оператора измененные уставки. Из процесса оператору передаются показатели системы, предупреждения и рекомендации, которые формируются при выходе параметров за допустимые пределы, и отчеты, и лог, содержащие сведения о выполненных действиях, ошибках и изменениях параметров.

От оборудования в процесс поступает статус работы оборудования, а из процесса передаются управляющие воздействия для запуска полива и управления климатом.

Из хранилища в процесс поступает информация о доступном оборудовании, то есть о датчиках и исполнительных устройствах, которые могут использоваться в установке.

В обратном направлении процесс передает данные о необходимом оборудовании, которое требуется для выполнения технологического процесса. Так же происходит процесс обмена информацией о сотрудниках, участвующих в работе системы (Рисунок 2.6).

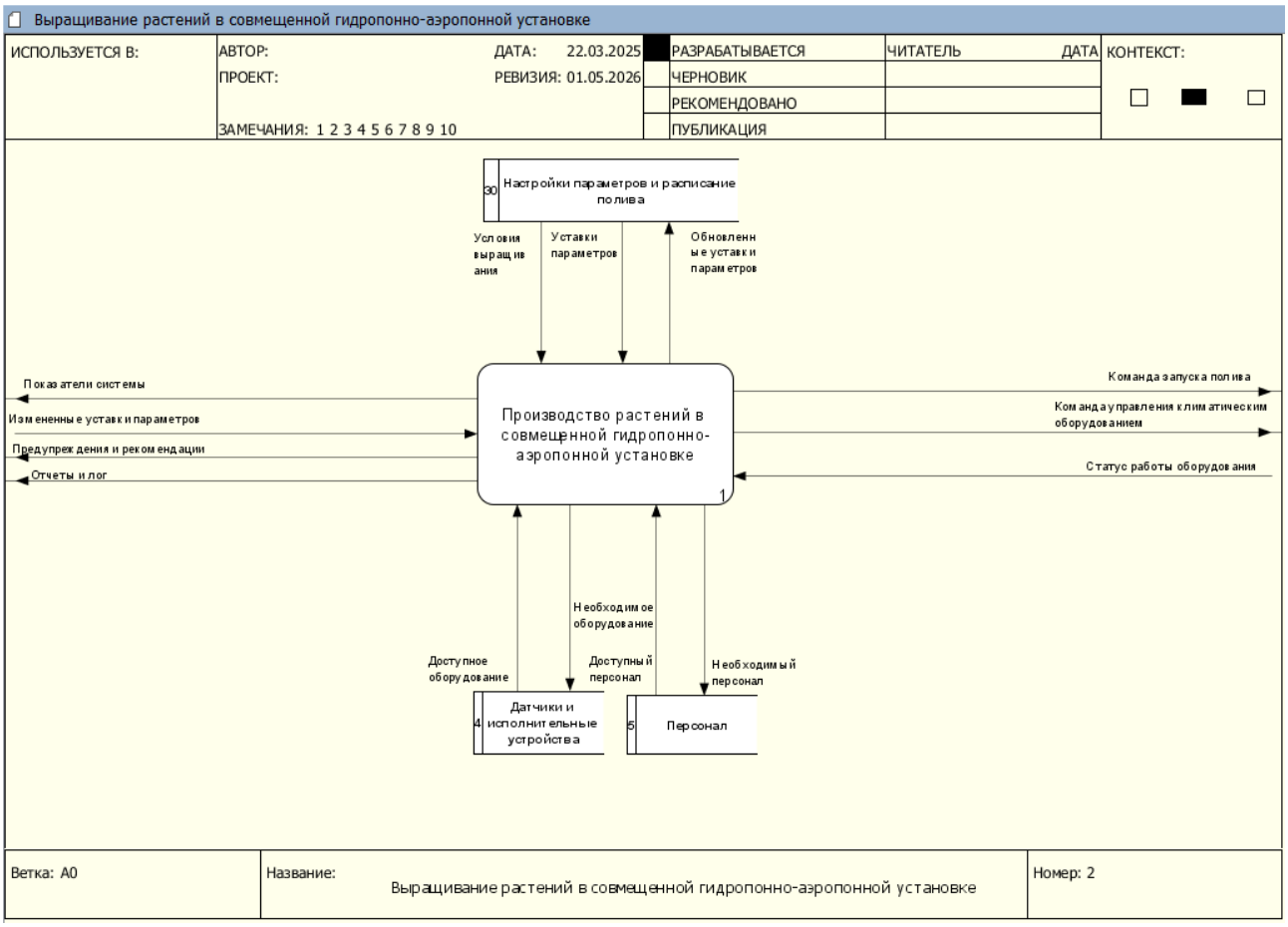


Рисунок 2.6 — Контекстная диаграмма уровня A-0

Производства начинается с процесса подготовки питательного раствора, при котором учитываются условия выращивания растений. После выполнения операции формируется сигнал о готовности раствора, который передается в процесс полива.

Полив осуществляется в соответствии с условиями выращивания и уставками параметров. Процесс полива использует данные о доступном оборудовании и персонале. На основе этих данных система формирует команду запуска полива для оборудования.

В процессе климат-контроля формируются предупреждения и рекомендации для оператора, а также текущие значения контролируемых параметров.

Процесс формирует команду управления климатическим оборудованием. Такая команда включает в себя управление отоплением, кондиционерами, осушителями и подачей углекислого газа.

Если в процессе полива и формирования микроклимата выявляются отклонения параметров от заданных диапазонов, то формируется запись об отклонении параметра, которая передается в журнал ошибок.

При корректировке отклонений формируется запись об изменении уставок, которая хранится в журнале действий оператора (Рисунок 2.7).

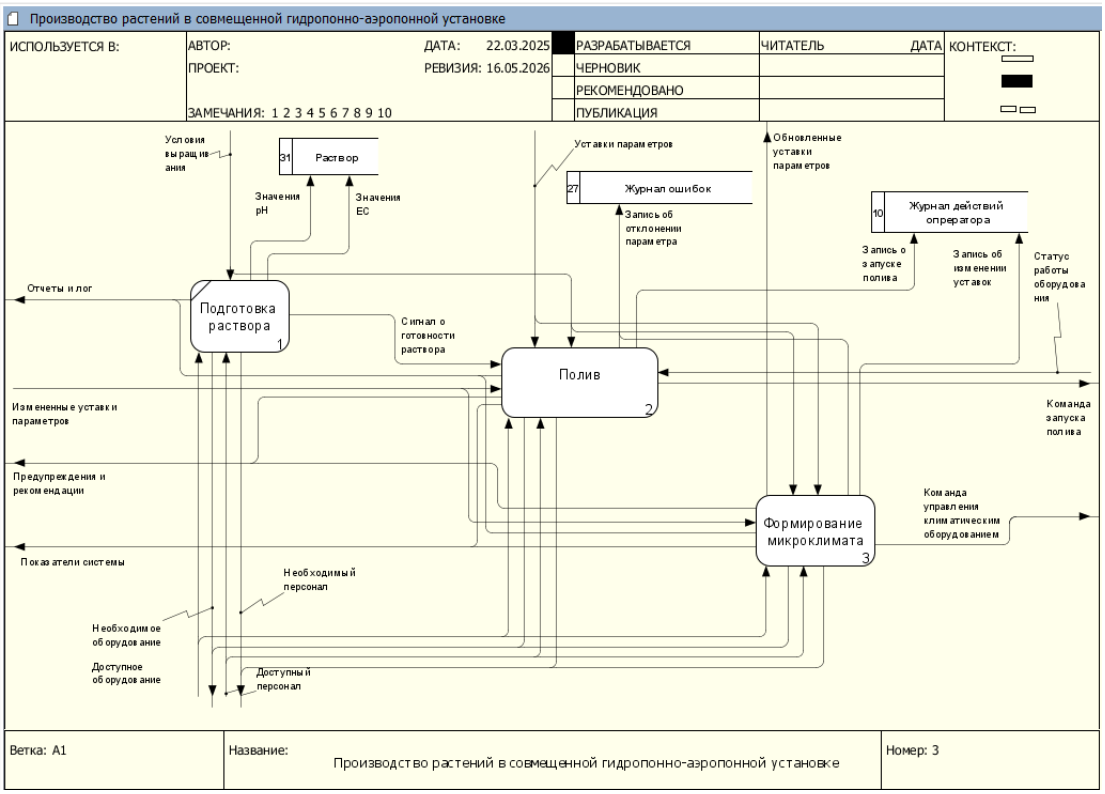


Рисунок 2.7 — Декомпозиция контекстной диаграммы

Декомпозиция процесса полива представлена двумя технологиями и контролем эффективности полива с помощью индекса NDVI.

Для осуществления процесса полива необходимо знать расписание для каждой технологии, а также список зон, требующих полива. В ходе процесса в базе данных появляются записи о сеансе полива с учетом зон и технологий.

После выполнения полива в гидропонной и аэропонной зонах осуществляется контроль эффективности полива в соответствии с индексом NDVI. В ходе процесса сохраняются время измерения и значения индекса NDVI, а также статус состояния растений.

На основе этих данных система оценивает, насколько эффективно выполняется полив и соответствует ли состояние растений заданным условиям выращивания. В случае отклонения значений индекса от нормы формируется команда запуска принудительного полива для требуемых зон (Рисунок 2.8).

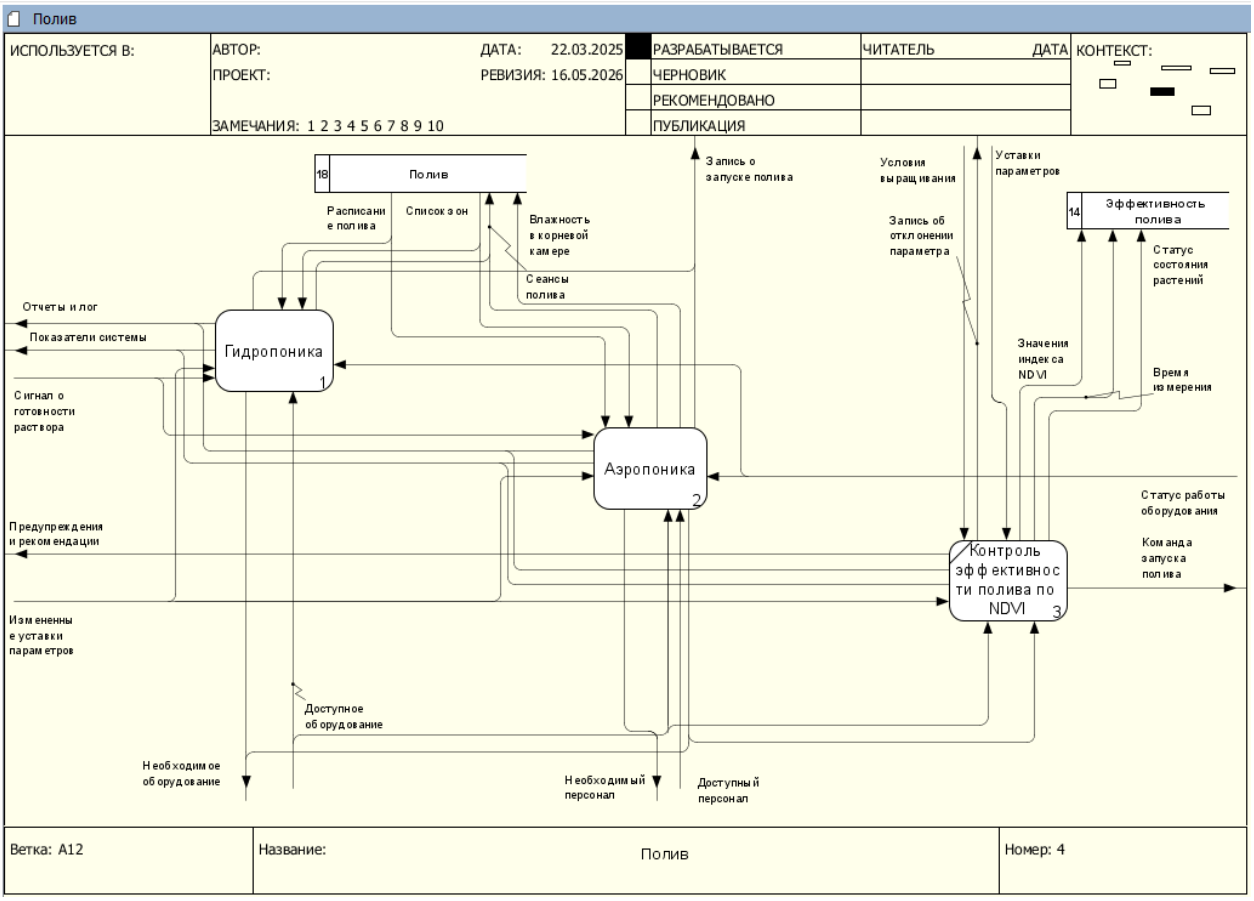


Рисунок 2.8 — Декомпозиция процесса полива

При формировании микроклимата системы сначала выполняется измерение параметров микроклимата. На этом этапе фиксируются текущие значения контролируемых параметров: температуры, относительной влажности воздуха и концентрации углекислого газа. Для выполнения измерения используется список контролируемых параметров. В результате процесса формируются текущие значения параметров, которые сохраняются в таблице измерений.

В процессе контроля учитываются уставки параметров и их текущие значения и формируются рекомендации по корректировке микроклимата. В результате корректировок уставок параметров климата формируется команда для управления климатическим оборудованием.

В базе данных сохраняются время измерения и текущие значения параметров (Рисунок 2.9).

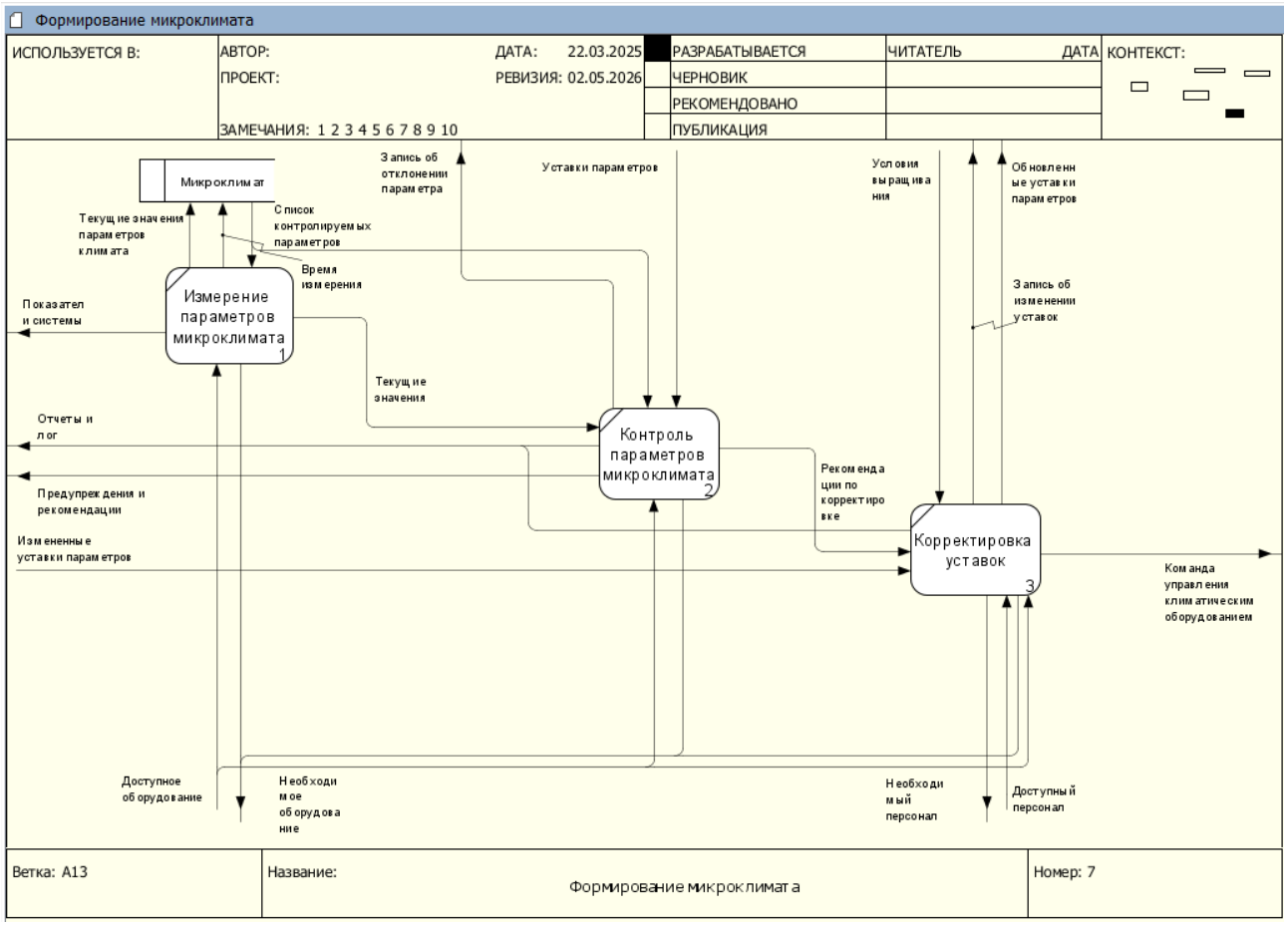


Рисунок 2.9 — Декомпозиция процесса контроля микроклимата

2.1.3 Моделирование сценариев функционирования системы в нотации BPMN

Разработанная BPMN-диаграмма отражает процесс взаимодействия оператора с веб-приложением, предназначенным для поддержки управления процессом выращивания растений в совмещенной гидропонно-аэропонной установке. Модель показывает общий сценарий работы пользователя в системе и распределение функций между оператором, подсистемами информационной системы и АСУ ТП.

Процесс начинается с авторизации оператора, после которой пользователю предоставляется доступ к главному экрану веб-приложения. Пользователь может либо продолжить работу на главной странице, либо перейти в другие разделы веб-приложения.

Главная страница выступает основным рабочим окном системы, на котором оператор получает сводную информацию о состоянии контролируемых параметров микроклимата, формирует отчетность по выбранным параметрам и изменяет уставки выбранных параметров (Рисунок 2.10-2.11).

Переход в другие разделы веб-приложения представлен в диаграмме в виде подпроцессов.

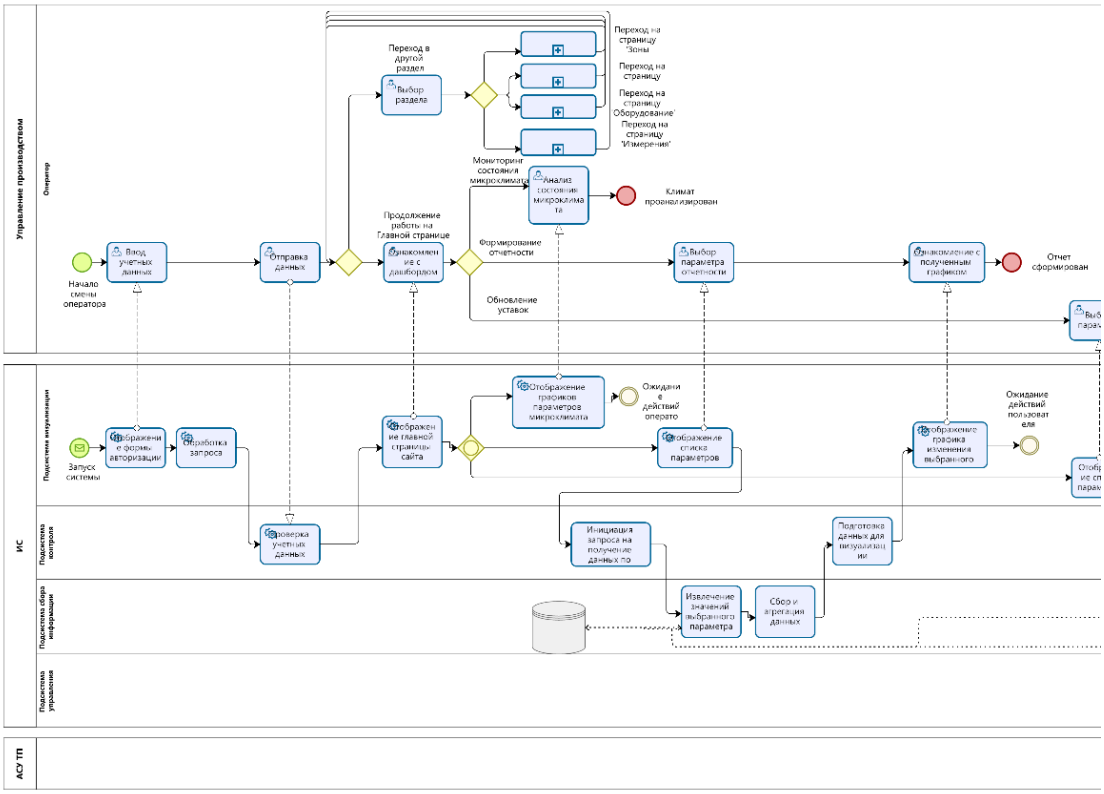


Рисунок 2.10 — Процесс взаимодействия оператора с главным экраном

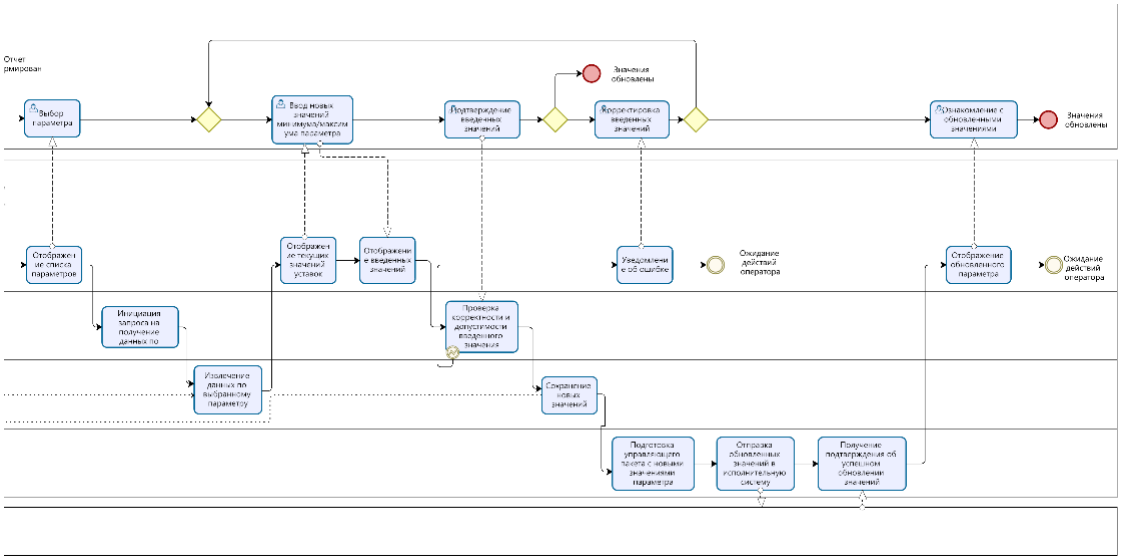


Рисунок 2.11 — Процесс изменения уставок выбранных параметров

При переходе в раздел «Зоны полива» оператор получает возможность управления поливом растений в выбранных зонах. На странице отображается список доступных технологий выращивания и соответствующих им зон полива.

Оператор может выбрать конкретную технологию или работать со всеми доступными зонами, после чего осуществляется отображение соответствующего набора зон. Далее пользователь выбирает одну или несколько зон, для которых требуется запустить принудительный полив.

После запуска полива в системе формируются управляющие команды, которые передаются в автоматизированную систему управления технологическим процессом. Параллельно в базе данных создается запись о сеансе полива, фиксирующая факт его запуска и параметры выполнения. После получения подтверждения о выполнении команды пользователю отображается информация об успешном запуске полива (Рисунок 2.12).

Одновременно с этим для выбранных зон начинается сбор измеряемых параметров, которые сохраняются в базе данных и отображаются в интерфейсе системы, что позволяет оператору в режиме реального времени отслеживать состояние зон после запуска полива.

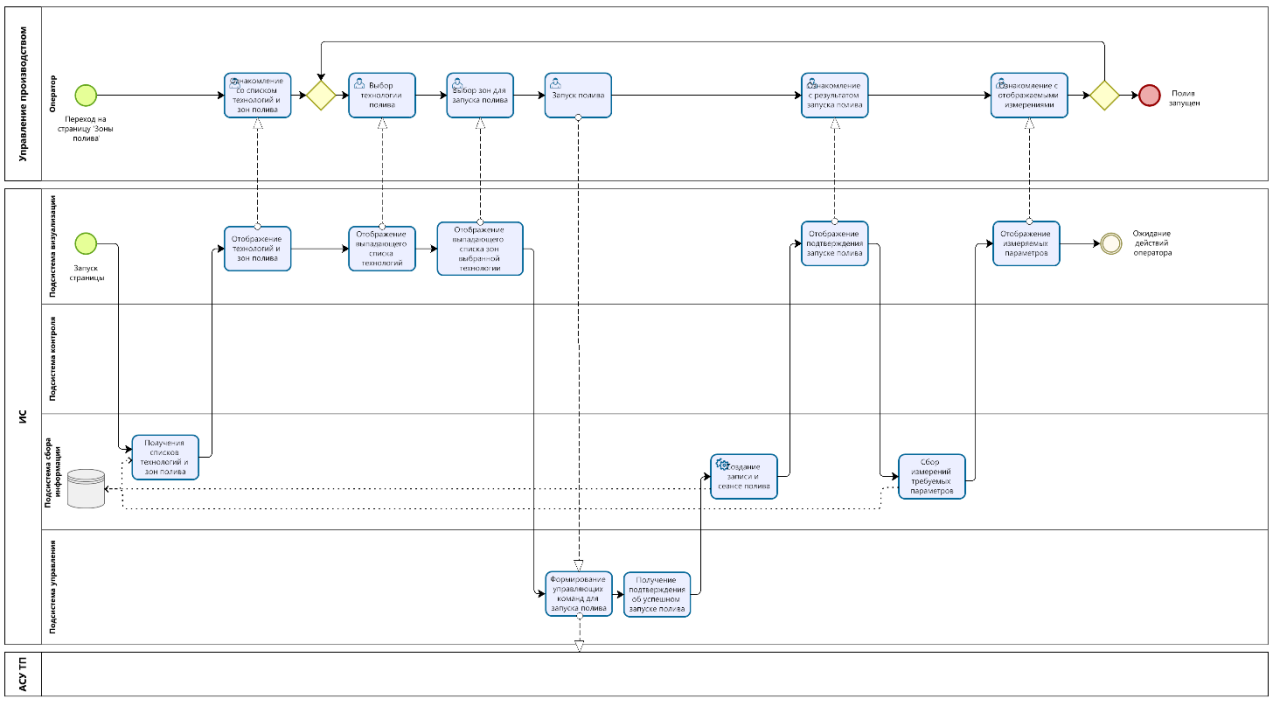


Рисунок 2.12 — Процесс запуска оператором сеанса принудительного полива

При переходе в раздел «Измерения» оператор получает возможность просмотра и анализа текущих значений параметров микроклимата. После открытия страницы подсистема визуализации отображает список доступных измерений, предварительно полученных из базы данных подсистемой сбора информации.

Оператор осуществляет просмотр измерений с возможностью их фильтрации по выбранному параметру. Для этого пользователь вводит название параметра в строку поиска, после чего подсистема визуализации осуществляет фильтрацию отображенного списка измерений.

После выполнения фильтрации оператор получает возможность ознакомиться с отобранными значениями параметров и оценить их текущее состояние. Процесс носит циклический характер: пользователь может неоднократно изменять параметр поиска и просматривать результаты, что отражено в диаграмме через возврат к этапу ввода параметра (Рисунок 2.13).

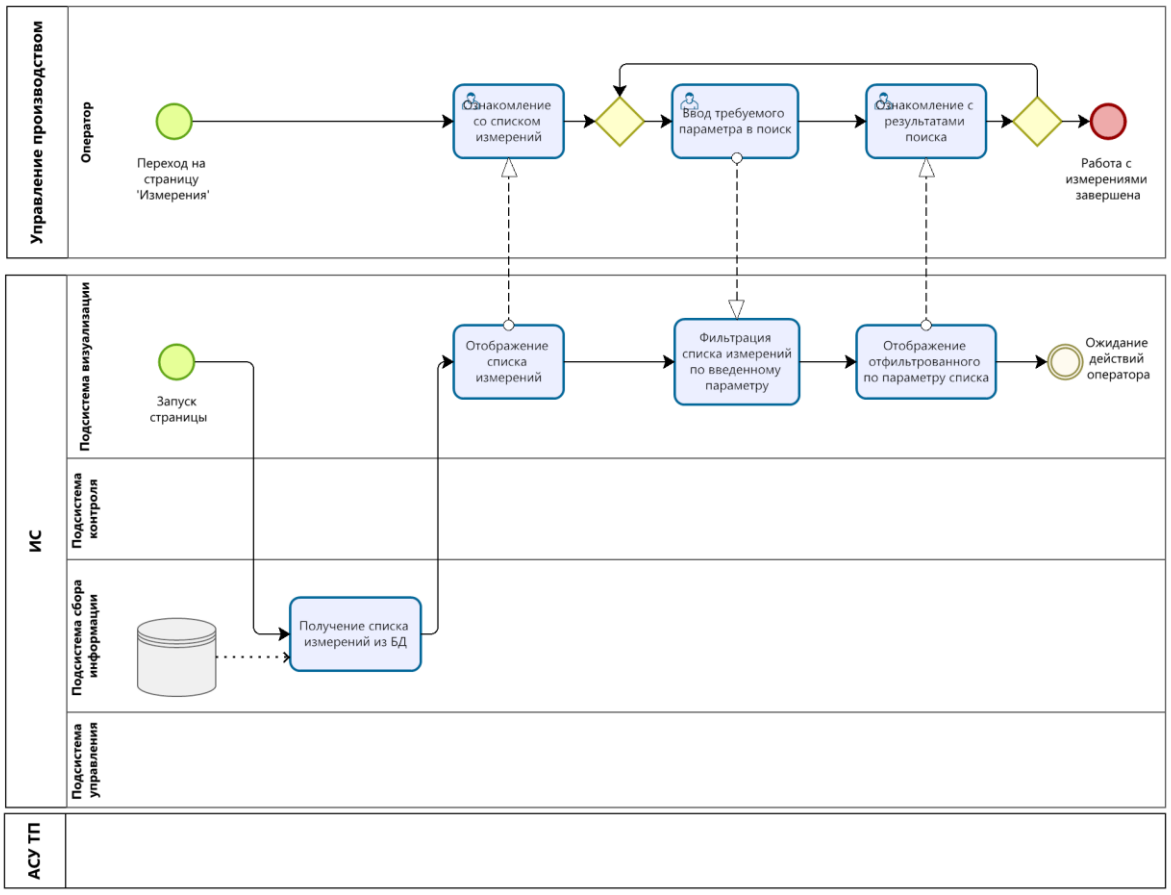


Рисунок 2.13 — Процесс просмотра измерений

При переходе в раздел «Оборудование» оператор получает возможность просматривать списки доступного оборудования, данные для которых предварительно извлекаются из базы данных подсистемой сбора информации.

Оператор может выполнить поиск конкретного устройства, осуществив ввод его название или идентификатора. В результате пользователю отображаются только те устройства, которые соответствуют заданным критериям (Рисунок 2.14).

После выполнения фильтрации оператор получает возможность ознакомиться с найденными устройствами и их характеристиками. Процесс может повторяться многократно, что позволяет последовательно уточнять параметры поиска и анализировать различные устройства.

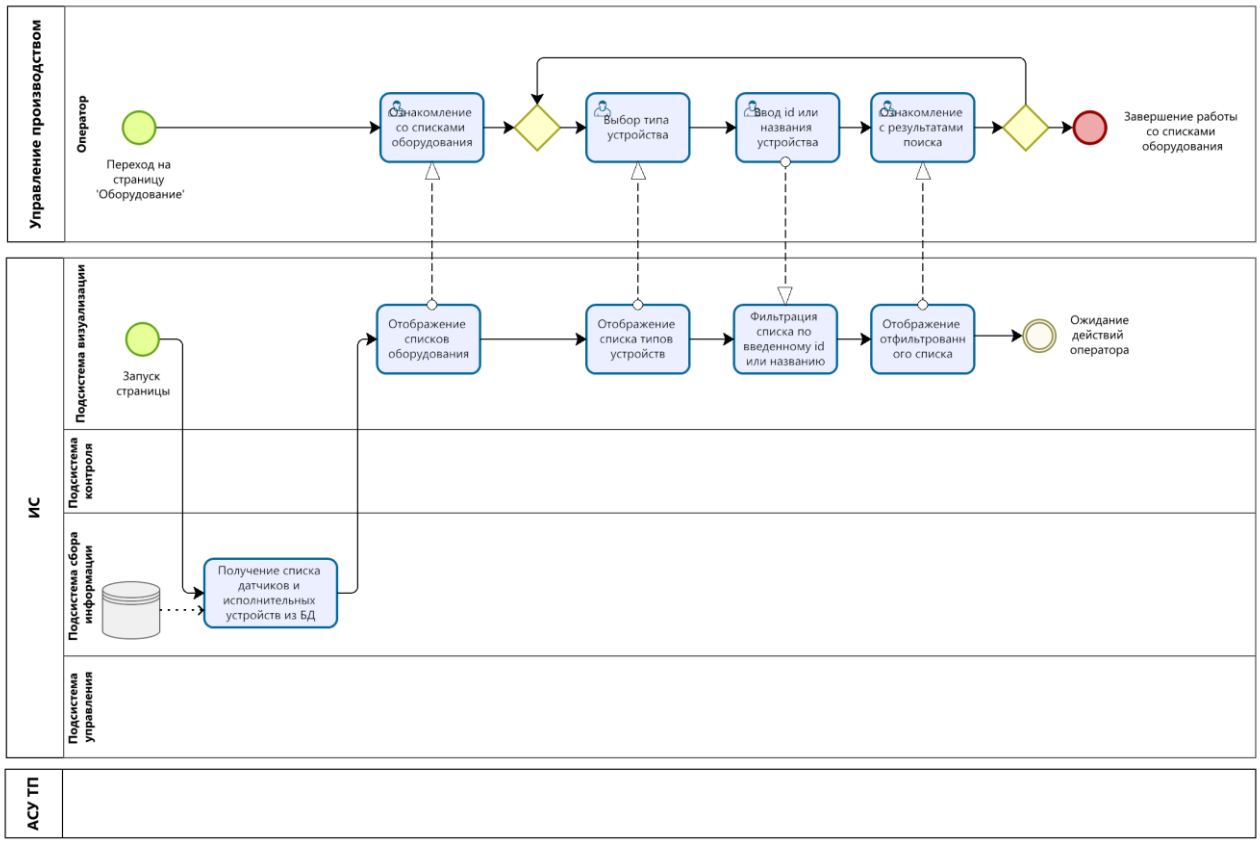


Рисунок 2.14 — Процесс работы оператора с разделом «Оборудование»

При переходе в раздел «Ошибки» оператор получает доступ к журналу ошибок и событий системы. После открытия страницы подсистема визуализации отображает список записей, предварительно извлеченных из базы данных подсистемой сбора информации.

Оператор имеет возможность ознакомиться с представленными данными и при необходимости выполнить поиск записей по коду или описанию ошибки.

Дополнительно оператору доступна возможность выгрузки журнала ошибок в файл формата CSV.

При запуске экспорта подсистема визуализации формирует файл на основе текущих отображаемых данных и инициирует его скачивание. Оператор получает сформированный файл для дальнейшего использования (Рисунок 2.15).

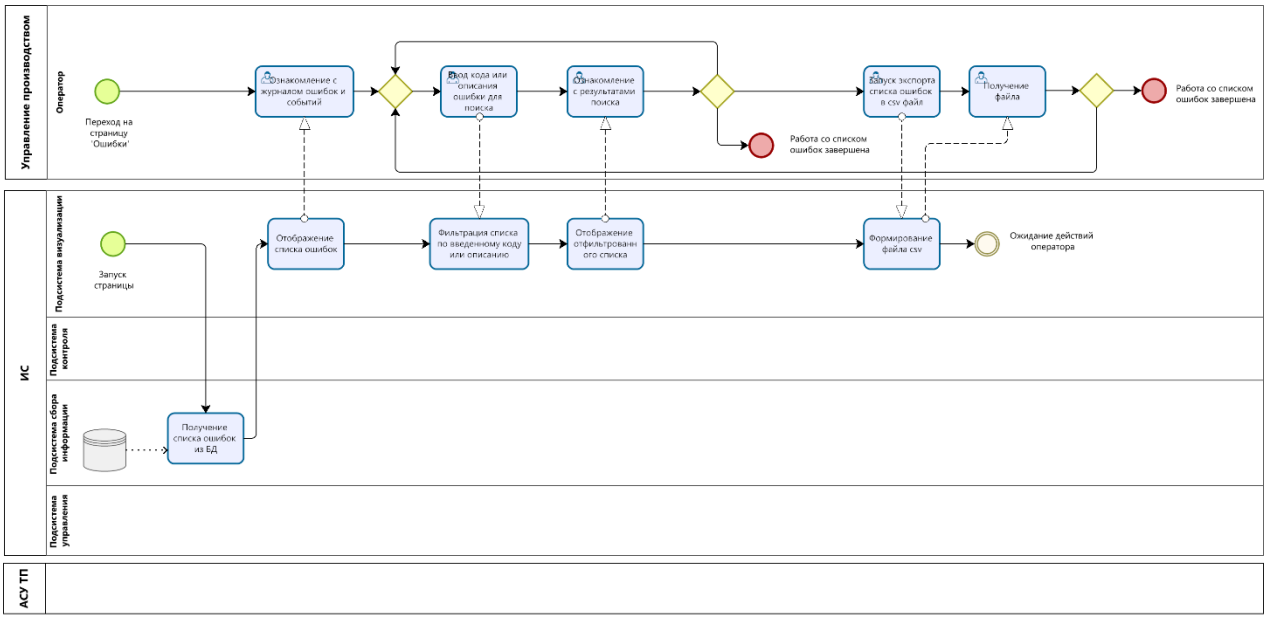


Рисунок 2.15 — Процесс работы оператора с разделом «Ошибки»

2.2 Разработка архитектуры программного обеспечения

2.2.1 Описание бизнес-слоя архитектуры программного обеспечения

Разработанная архитектурная диаграмма в нотации ArchiMate отражает структуру информационно-управляющей системы выращивания растений и демонстрирует распределение функций и компонентов по бизнес-, прикладному и технологическому слоям, обеспечивая целостное представление о работе системы.

На бизнес-слое представлена логика функционирования системы на уровне предметной области и взаимодействия оператора с приложением. Процесс начинается с формирования запроса данных оператором, после чего осуществляется сбор и получение информации.

Далее выполняется контроль параметров технологического процесса, на основании которого формируются уведомления об отклонениях. В зависимости от результатов анализа происходит обновление параметров управления, а также актуализация данных в хранилище и последующее оформление отчетности.

Бизнес-слой задает последовательность основных этапов обработки данных и определяет требования к реализации системы.

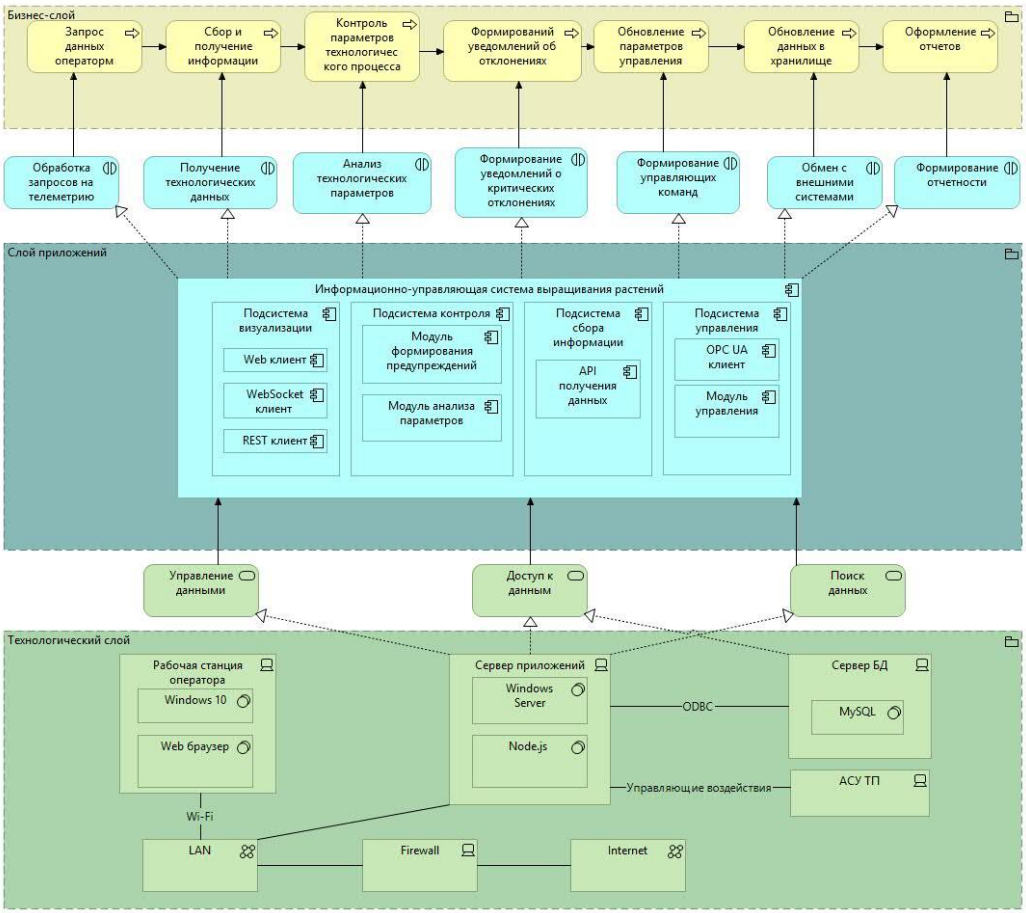


Рисунок 2.16 — Диаграмма Archimate

2.2.2 Описание прикладного слоя архитектуры программного обеспечения

Прикладной слой реализует бизнес-логику и включает как функциональные элементы, так и структурные компоненты системы. На уровне функций отражены процессы обработки запросов на получение телеметрии, получения технологической информации, анализа параметров, формирования уведомлений о критических отклонениях, формирования управляющих команд, обмена данными с внешними системами и формирования отчетности.

Эти функции реализуются в рамках информационно-управляющей системы, представленной набором взаимосвязанных подсистем. Подсистема визуализации обеспечивает взаимодействие с пользователем через web-клиент, REST-клиент и WebSocket-клиент.

Подсистема контроля выполняет анализ параметров и формирование предупреждений. Подсистема сбора информации отвечает за получение данных через API, а подсистема управления формирует управляющие воздействия и обеспечивает взаимодействие с исполнительными устройствами посредством OPC UA клиента. Прикладной слой выступает связующим звеном между бизнес-логикой и технической реализацией системы.

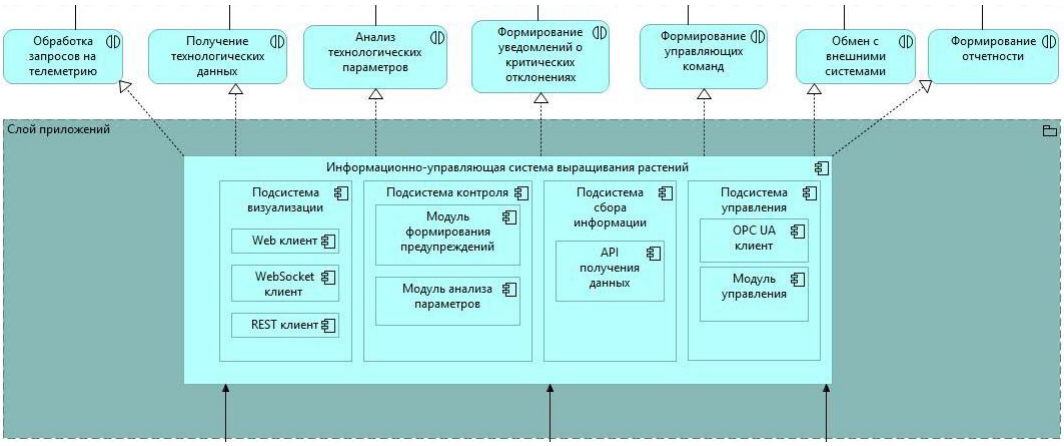


Рисунок 2.17 — Слой приложений архитектуры программного обеспечения

2.2.3 Описание технологического слоя архитектуры программного обеспечения

Технологический слой включает инфраструктуру, необходимую для функционирования системы. В его состав входят рабочая станция оператора, сервер приложений и сервер базы данных. Рабочая станция включает операционную систему и веб-браузер, через который осуществляется доступ к системе. Сервер приложений функционирует под управлением операционной системы Windows Server и использует среду выполнения Node.js для реализации серверной логики. Сервер базы данных основан на СУБД MySQL и предназначен для хранения технологических данных.

Взаимодействие компонентов осуществляется через сетевую инфраструктуру, включающую локальную сеть, межсетевой экран и интернет.

Обмен данными между сервером приложений и базой данных осуществляется через соответствующий механизм подключения, а взаимодействие с автоматизированной системой управления технологическим процессом реализуется посредством передачи управляющих воздействий.

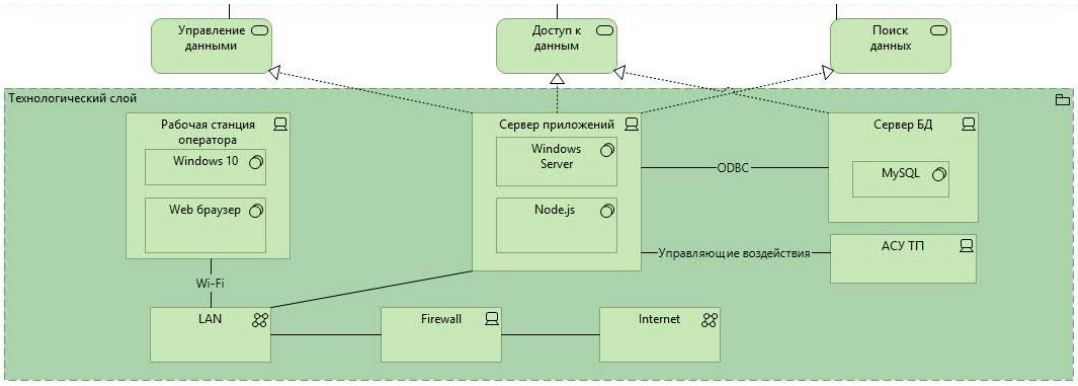


Рисунок 2.18 — Технологический слой архитектуры программного обеспечения

2.3 Разработка алгоритма работы информационно-управляющей системы

2.3.1 Алгоритм запуска принудительного полива

Алгоритм запуска принудительного полива представлен в виде блок-схемы и отражает последовательность действий оператора при ручном запуске полива для выбранных зон установки.

В начале алгоритма оператор выбирает для какой технология будет запускаться полив. Выбор технологии необходим для определения перечня доступных зон.

После назначения технологии оператор выбирает одну или несколько зон, в которых требуется запустить полив. Далее осуществляется запуск полива. На этом этапе система инициирует выполнение полива в выбранных зонах в соответствии с назначенной технологией.

После выполнения команды алгоритм завершается (Рисунок 2.19).

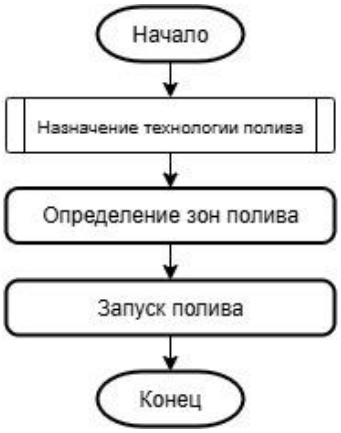


Рисунок 2.19 — Блок-схема алгоритма запуска принудительного полива

2.3.2 Алгоритм обновления уставок выбранного параметра

Алгоритм обновления уставок параметров представлен в виде блок-схемы и отражает последовательность действий оператора и программных компонентов системы при изменении допустимых диапазонов значений технологических параметров.

В начале выполнения алгоритма оператор выбирает требуемый параметр контроля, после чего система отображает текущие значения уставок.

Далее выполняется проверка соответствия уставок условиям. Если уставки не соответствуют заданным требованиям, оператору предлагается выполнить ввод новых уставок. Если уставки соответствуют условиям, система выполняет проверку корректности диапазона: значение нижней границы не должно превышать значение верхней границы, иначе оператору предлагается повторный ввод значений.

Если значения корректны, то они сохраняются в безе данных, после чего формируются соответствующие управляющие команды. Затем сформированные команды передаются в автоматизированную систему управления технологическим процессом, и система получает обновленные значения параметров, подтверждающие применение новых уставок.

На завершающем этапе обновленные значения отображаются пользователю, после чего выполнение алгоритма заканчивается (Рисунок 2.20).

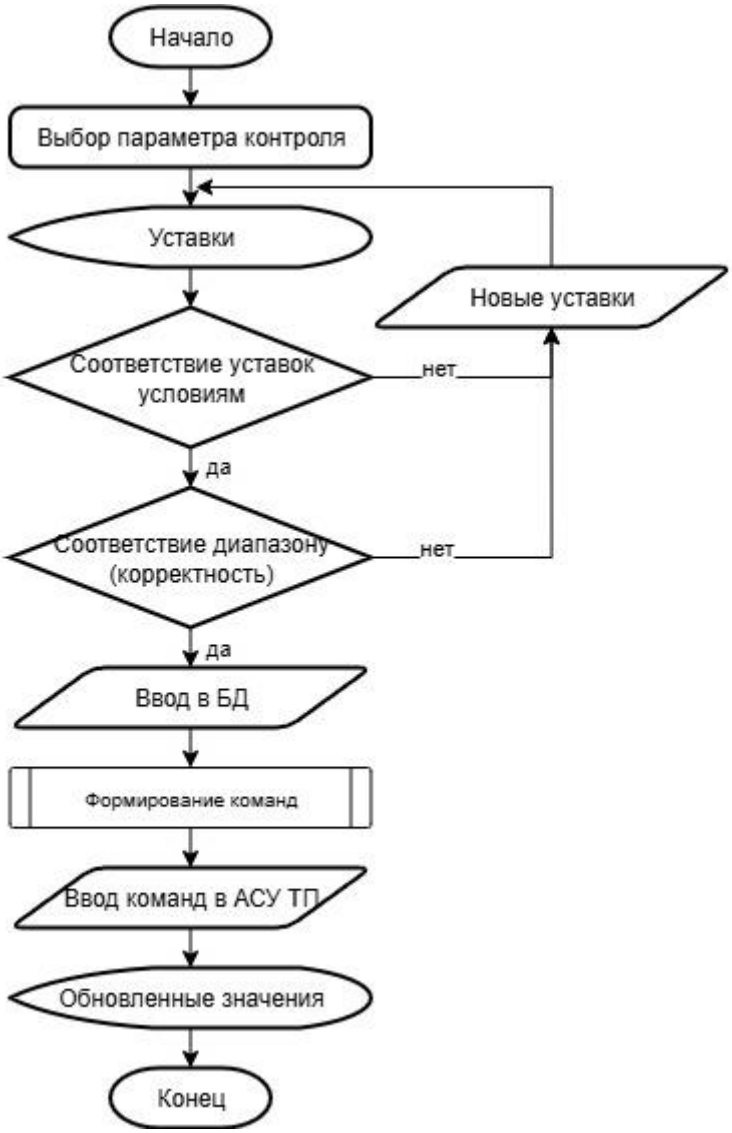


Рисунок 2.20 — Блок-схема алгоритма изменения уставок параметра

3 ПРОГРАММНЫЙ РАЗДЕЛ

3.1 Разработка базы данных

База данных разрабатываемой системы предназначена для хранения и обработки информации, необходимой для мониторинга и управления процессом выращивания растений в совмещенной гидропонно-аэропонной установке. Структура базы данных построена таким образом, чтобы обеспечить хранение информации о зонах выращивания, используемых технологиях, датчиках, исполнительных устройствах, технологических параметрах, измерениях, уставках, ошибках, действиях сотрудников и расписании полива. Физическая модель базы данных представлена на EER-диаграмме, включающей основные таблицы системы и связи между ними (Рисунок 3.1).

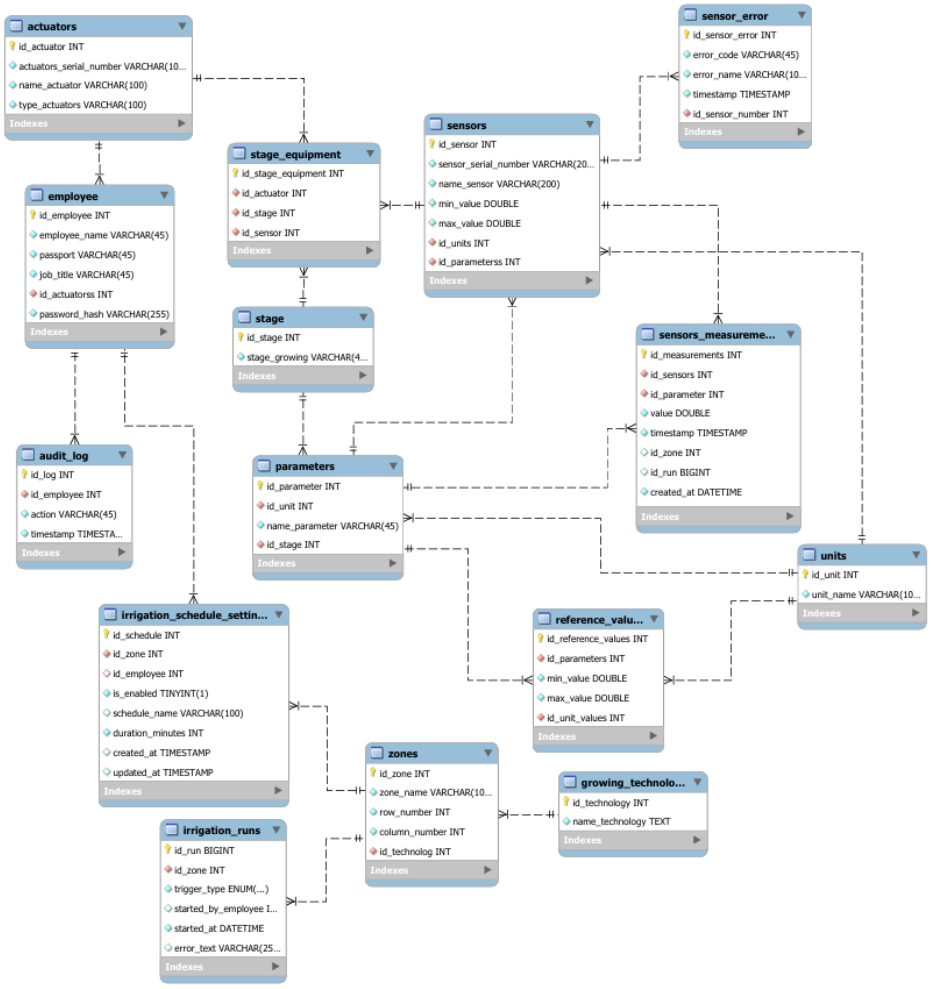


Рисунок 3.1 — EER-диаграмма

Основной таблицей базы данных является таблица sensors_measurements. Она предназначена для хранения результатов измерений, поступающих от датчиков. Таблица показывает, какой параметр был измерен, каким датчиком, в какой зоне выращивания и в какой момент времени. На основе таблицы sensors_measurements в веб-приложении отображаются текущие значения параметров, строятся графики, формируются отчеты и выполняется анализ состояния зон выращивания.

Для описания зон выращивания используется таблица zones. В ней хранится информация об отдельных зонах, в которых осуществляется контроль параметров и управление процессом выращивания. Информация о технологиях выращивания хранится в таблице growing_technology. Связь данной таблицы с таблицей zones указывает, по какой технологии осуществляется выращивание в каждой зоне. Это необходимо для дальнейшего контроля параметров, анализа условий выращивания и формирования отчетных данных.

Для хранения этапов выращивания предусмотрена таблица stage. Этапы используются для привязки технологических параметров и оборудования к определенной стадии выращивания.

Технологические параметры системы хранятся в таблице parameters. Связь параметров с этапами выращивания и единицами измерения позволяет корректно обрабатывать и отображать данные в системе.

Единицы измерения вынесены в отдельную таблицу units. Такая организация данных исключает дублирование одинаковых единиц измерения в разных таблицах и обеспечивает единый формат хранения информации о параметрах и результатах измерений.

Для хранения уставок параметров используется таблица reference_values. Данная таблица задает допустимые диапазоны значений технологических параметров. Эти значения используются для контроля состояния системы и выявления отклонений контролируемых параметров от эталонных значений.

Информация о датчиках хранится в таблице sensors. Эта таблица определяет, какой параметр измеряет конкретный датчик и в каких пределах должны находиться его значения.

Для регистрации отклонений технологических параметров используется таблица sensor_error. Данная таблица хранит случаи выхода параметров за допустимые диапазоны значений. Информация об исполнительных устройствах хранится в таблице actuators.

Связь между этапами выращивания, датчиками и исполнительными устройствами реализована через таблицу stage_equipment. Данная таблица позволяет определить, какие датчики и исполнительные устройства используются на конкретном этапе выращивания.

Для хранения информации о сотрудниках предусмотрена таблица employee. Она используется для авторизации пользователей и хранит пароль в виде хэша, что повышает безопасность авторизации, так как пароль не сохраняется в базе данных в открытом виде.

Действия сотрудников фиксируются в таблице audit_log. Эта таблица используется для ведения журнала действий пользователей. С ее помощью можно определить, какой сотрудник выполнил определенное действие: изменил уставки параметров или запустил полив.

Таблица irrigation_schedule_settings хранит настройки расписания полива для зон выращивания. В ней указывается, к какой зоне относится расписание, какой сотрудник его создал или изменил, активно ли данное расписание, его название, длительность полива в минутах, а также дата создания и последнего обновления записи.

Сеансы запуска полива сохраняются в таблице irrigation_runs. Эта таблица используется для хранения истории запусков полива, включая автоматические и принудительные включения, а также для фиксации ошибок, возникших при выполнении полива.

Связи между таблицами обеспечивают целостность данных и позволяют объединять информацию из разных частей системы.

3.2 Разработка кода программных компонентов системы

Описание программной реализации пользовательского интерфейса включает ключевые фрагменты кода, обеспечивающие работу веб-приложения информационно-управляющей системы. Основная логика интерфейса связана с получением данных из базы данных, отображением текущего состояния установки и выполнением управляющих действий оператором.

Пользовательский интерфейс поддерживает основные сценарии работы с системой: авторизацию сотрудника, просмотр параметров микроклимата, анализ измерений, изменение уставок, запуск принудительного полива, просмотр журнала ошибок и обработку предупреждений. Для выполнения этих действий клиентская часть приложения обращается к серверной части, получает необходимые данные и отображает их в удобном для оператора виде.

Авторизация пользователя.

Функция handleSubmit обрабатывает отправку формы авторизации. Она проверяет, заполнены ли поля входа, передает введенные данные на сервер для проверки и при успешной авторизации сохраняет сведения о сотруднике в веб-приложении. Это позволяет идентифицировать пользователя и использовать его данные при выполнении действий в системе.

Листинг 1 — Функция авторизации пользователя

```
const handleSubmit = async (event) => {
  event.preventDefault();
  const first = firstName.trim();
  const last = lastName.trim();
  if (!first || !last || !password) {
    setError('Заполните имя, фамилию и пароль');
    return;
  }
  setLoading(true);
  setError('');
  try {
    const data = await apiLogin({ firstName: first, lastName: last, password });
    login(data);
    navigate(from, { replace: true });
  } catch {
    setError('Неверное имя, фамилия или пароль');
  } finally {
    setLoading(false);
  }
};
```

Фрагмент, представленный на Листинге 2, связывает форму входа с серверной проверкой учетных данных. Если авторизация проходит успешно, пользователь перенаправляется на главную страницу веб-приложения.

Сервер получает данные пользователя, ищет сотрудника в таблице employee и проверяет пароль. При успешной проверке клиент получает идентификатор и имя сотрудника.

Листинг 2 — Серверная обработка авторизации

```
app.post('/api/auth/login', async (req, res, next) => {
  try {
    const pool = await poolPromise;
    const { firstName, lastName, password } = req.body || {};
    const first = normalizeSpace(firstName);
    const last = normalizeSpace(lastName);
    const pass = String(password || '');
    if (!first || !last || !pass) {
      return res.status(400).json({ error: 'Введите имя, фамилию и пароль' });
    }

    const cols = await getColumns(pool, 'employee');
    const colMap = buildColumnMap(cols);
    const passCol = pickColumn(colMap, ['password', 'pass', 'password_hash',
'hash']);

    const sql = `SELECT * FROM employee WHERE (${conditions.join(' OR ')}) LIMIT
20`;
    const [rows] = await pool.query(sql, params);

    let matched = null;
    for (const row of rows) {
      if (verifyPassword(pass, row?.[passCol])) {
        matched = row;
        break;
      }
    }
    if (!matched) return res.status(401).json({ error: 'Неверные данные' });

    res.json({ id, name: name || `${first} ${last}`.trim() });
  } catch (err) {
    next(err);
  }
});
```

Получение текущих параметров микроклимата.

Функция getDashboard используется в клиентской части веб-приложения для получения данных для главной страницы (Листинг 3). Она отправляет запрос на сервер и получает объект aggregates, содержащий текущие значения температуры воздуха, относительной влажности и концентрации CO_2, а также список зарегистрированных ошибок.

Листинг 3 — Клиентская функция запроса текущих параметров микроклимата

```
async getDashboard() {
  const data = await request('/api/dashboard2');
  const aggregates = data?.aggregates ?? data ?? {};
  const errors = asArray(data?.errors);
  const alertCount = errors.filter((e) => (e.severity || e.status) ===
'critical').length;
  const warnCount = errors.filter((e) => (e.severity || e.status) ===
'warning').length;
  return { updatedAt: Date.now(), aggregates, errors, alertCount, warnCount };
}
```

На серверной стороне эти данные формируются функцией buildDashboardSnapshot (Листинг 4). Она получает последние записи из таблицы измерений и подготавливает их для передачи в интерфейс. Совместное использование клиентской функции запроса и серверной функции формирования данных показывает полный путь передачи информации от базы данных до интерфейса главной страницы.

Листинг 4 — Серверная функция формирования текущих параметров микроклимата

```
async function buildDashboardSnapshot(pool) {
  const aggregates = { temperature: null, humidity: null, co2: null };
  let errors = [];
  let updatedAt = Date.now();

  const [rows] = await pool.query(
    `SELECT id_parameter, value, timestamp, sensor_id
    FROM (
      SELECT m.id_parameter,
             m.value,
             m.timestamp,
             m.id_sensors AS sensor_id,
             ROW_NUMBER() OVER (
               PARTITION BY m.id_parameter
               ORDER BY m.timestamp DESC, m.id_measurements DESC
             ) AS rn
      FROM sensors_measurements m
      WHERE m.id_parameter IN (1, 2, 3)
        AND m.timestamp >= DATE_SUB(NOW(), INTERVAL 5 MINUTE)
    ) latest
    WHERE rn = 1
    ORDER BY timestamp DESC`,
  );

  for (const row of rows) {
    const pid = Number(row.id_parameter);
    if (pid === 1) aggregates.temperature = row.value;
    else if (pid === 2) aggregates.humidity = row.value;
    else if (pid === 3) aggregates.co2 = row.value;
  }
  return { aggregates, errors, updatedAt };
}
```

Изменение уставок параметров.

Для изменения допустимых диапазонов контролируемых параметров используется функция handleReferenceSubmit (Листинг 5). Она обрабатывает отправку формы, проверяет введенные пользователем минимальное и максимальное значения и передает обновленные уставки на сервер для сохранения. После выполнения этой операции система использует новые значения при контроле отклонений параметров.

Функция обеспечивает проверку введенных значений перед их сохранением в системе. Это позволяет исключить запись некорректных уставок: пустых значений или диапазона, в котором минимальное значение превышает максимальное. После успешного сохранения данные на главной странице обновляются, и оператор видит актуальные допустимые границы выбранного параметра.

Листинг 5 — Изменение уставок контролируемых параметров на главной странице

```
const handleReferenceSubmit = useCallback(async () => {
  const minRaw = editMin === '' ? loadedRef.min : editMin;
  const maxRaw = editMax === '' ? loadedRef.max : editMax;
  const min = Number(minRaw);
  const max = Number(maxRaw);

  if (minRaw === '' || maxRaw === '') {
    setEditError('Заполните оба значения');
    return;
  }
  if (Number.isNaN(min) || Number.isNaN(max)) {
    setEditError('Минимум и максимум должны быть числами');
    return;
  }
  if (min > max) {
    setEditError('Минимум не может быть больше максимума');
    return;
  }

  setSaving(true);
  try {
    await updateReference({ parameterId: editParam, min, max, employeeId:
employee?.id });
    setEditSuccess('Диапазон обновлен');
    setLoadedRef({ min, max });
    loadMainRefs();
  } finally {
    setSaving(false);
  }
}, [editMax, editMin, editParam, employee?.id, loadedRef, loadMainRefs]);
```

Серверная часть выполняет дополнительную проверку полученных значений перед сохранением (Листинг 6). После успешной проверки новый диапазон допустимых значений записывается в таблицу reference_values, где хранятся уставки контролируемых параметров. Также система фиксирует выполненное действие в журнале, что позволяет определить, какой сотрудник изменил уставки и в какой момент времени это было сделано.

Листинг 6 — Серверное сохранение уставок

```
app.post('/api/reference', async (req, res, next) => {
  try {
    const pool = await poolPromise;
    const { parameterId, min_value, max_value, employeeId } = req.body || {};
    const paramId = Number(parameterId);
    const min = Number(min_value);
    const max = Number(max_value);

    if (Number.isNaN(paramId) || Number.isNaN(min) || Number.isNaN(max)) {
      return res.status(400).json({ error: 'Минимум/максимум должны быть
числами' });
    }
    if (min > max) {
      return res.status(400).json({ error: 'Минимум не может быть больше
максимума' });
    }

    await pool.query(
      `UPDATE reference_values
       SET min_value = ?, max_value = ?
       WHERE ${idCol} = ?`,
      [min, max, paramId],
    );

    if (employeeId != null) {
      await logAction(pool, { employeeId, parameterId: paramId, actionText });
      await logAudit(pool, { employeeId, action: actionText });
    }

    res.json({ ok: true, parameterId: paramId, min_value: min, max_value:
max });
  } catch (err) {
    next(err);
  }
});
```

Запуск принудительного полива.

Функция handleForce предназначена для запуска полива по команде оператора (Листинг 7). Перед выполнением действия проверяется, выбрана ли хотя бы одна зона полива и определена ли технология выращивания. После успешной проверки функция отправляет запрос на сервер для запуска принудительного полива.

Данный механизм позволяет оператору вручную инициировать полив вне расписания, например, при снижении значения индекса NDVI.

Листинг 7 — Запуск принудительного полива оператором

```
const handleForce = async () => {
  if (!forceZoneIds.length || !selectedZones.length) {
    setForceStatus({ type: 'danger', text: 'Выберите зоны' });
    return;
  }

  const invalidZones = selectedZones.filter((zone) =>
getWateringDurationSec(zone) == null);
  if (invalidZones.length) {
    const names = invalidZones.map(getZoneDisplayName).join(', ');
    setForceStatus({ type: 'danger', text: `Неизвестная технология:
${names}` });
    return;
  }

  setForceLoading(true);
  try {
    const res = await forceWateringBatch(
      selectedZones.map((zone) => zone.id),
      { startedByEmployee: employee?.id ?? null },
    );
    const results = Array.isArray(res?.results) ? res.results : [];
    const failed = results.filter((result) => !result.ok);
    const success = results.filter((result) => result.ok);
    // далее формируется сообщение об успешном или частичном запуске
  } finally {
    setForceLoading(false);
    await load();
  }
};
```

Серверная часть определяет продолжительность принудительного полива с учетом выбранной технологии выращивания. Для гидропонной зоны устанавливается длительность 600 секунд, а для аэропонной зоны — 10 секунд. После этого сервер вызывает хранимую процедуру start_irrigation_for_zone, которая создает сеанс полива для выбранной зоны и фиксирует его выполнение в базе данных.

Листинг 8 — Серверный запуск полива

```
app.post('/api/zones/water/force', async (req, res, next) => {
  try {
    const pool = await poolPromise;
    const { zoneIds, startedByEmployee } = req.body || {};
    if (!Array.isArray(zoneIds) || !zoneIds.length) {
      return res.status(400).json({ error: 'zoneIds must be a non-empty
array' });
    }

    const ids = Array.from(new Set(zoneIds.map((v) =>
Number(v)).filter(Number.isFinite)));
    const results = [];
```

Продолжение Листинга 8

```
    for (const id of ids) {
      const zone = zoneMap.get(Number(id));
      let duration = null;
      if (zone.technology === 1) duration = 600;
      else if (zone.technology === 2) duration = 10;

      try {
        const [resultSets] = await pool.query(
          'CALL start_irrigation_for_zone(?, ?, ?, ?)',
          [id, 'MANUAL', duration, startedByEmployee ?? null],
        );
        results.push({ id, ok: true, runId });
      } catch (err) {
        results.push({ id, ok: false, error: err?.message || 'Failed to start
irrigation' });
      }
    }
    const success = results.filter((r) => r.ok).length;
    res.json({ ok: success === results.length, success, failed: results.length -
success, results });
  } catch (err) {
    next(err);
  }
});
```

Получение журнала ошибок.

Для отображения журнала ошибок используется функция getErrors (Листинг 9). При открытии соответствующей вкладки она отправляет запрос на сервер и получает список зарегистрированных ошибок и событий системы. Полученные данные отображаются в интерфейсе, что позволяет оператору просматривать зафиксированные отклонения, аварийные ситуации и другие события, требующие внимания.

Листинг 9 — Загрузка журнала ошибок

```
useEffect(() => {
  const load = async () => {
    try {
      const data = await getErrors();
      setErrors(data || []);
    } catch (e) {
      console.error('errors load error', e);
      setErrors([]);
    }
  };
  load();
}, []);
```

Серверная часть получает последние записи из таблицы sensor_error и дополняет их информацией об измерениях, уставках и единицах измерения.

Это позволяет отображать в интерфейсе не только код ошибки, но и фактическое значение параметра с допустимым диапазоном, что упрощает анализ причины отклонения (Листинг 10).

Листинг 10 — Серверное получение журнала ошибок

```
app.get('/api/errors', async (_req, res) => {
  const pool = await poolPromise;
  const [rows] = await pool.query(
    `SELECT se.id_sensor_error AS id,
            se.timestamp AS ts,
            se.error_code AS code,
            se.error_name AS text,
            'warning' AS severity,
            se.id_sensor_number AS sensorId
     FROM sensor_error se
     ORDER BY se.timestamp DESC
     LIMIT 100`,
  );

  const measurements = await fetchMeasurementsByPairs(pool, pairs);
  const refMap = new Map();

  const enriched = rows.map((r) => {
    let severity = r.severity || 'warning';
    const measurement = measurementList[0] || null;
    if (measurement) {
      const computed = getDeviationSeverity(measurement.value, ref?.min_value,
ref?.max_value);
      if (computed) severity = computed;
    }
    return { ...r, severity, value, parameterId, refMin, refMax, unit };
  });

  res.json(enriched.map((r) => ({ ...r, zoneId: null })));
});
```

Формирование предупреждений при отклонениях параметров.

Функция getDeviationSeverity определяет уровень отклонения параметра от допустимого диапазона уставок. Если значение находится в пределах от refMin до refMax, предупреждение не формируется. При выходе значения за границу диапазона рассчитывается величина отклонения delta, после чего определяется порог threshold, равный 5% от ширины допустимого диапазона. Если величина отклонения не превышает этот порог, параметру присваивается статус Warning; если отклонение больше порога, формируется статус Alarm.

Листинг 11 — Определение степени отклонения параметра

```
const getDeviationSeverity = (value, refMin, refMax) => {
  const val = toNumber(value);
  const min = toNumber(refMin);
  const max = toNumber(refMax);
  if (val === null || min === null || max === null) return null;
  if (val >= min && val <= max) return null;
```

Продолжение Листинга 11

```
  const delta = val < min ? min - val : val - max;
  const range = max - min;
  const threshold = Math.max(range * 0.05, 2);
  return delta <= threshold ? 'warning' : 'critical';
};
```

Выполняется проверка измеренных значений на соответствие заданным уставкам (Листинг 12). Для каждого параметра определяется, находится ли текущее значение ниже минимальной или выше максимальной допустимой границы. Если отклонение обнаружено, система формирует код и описание ошибки.

Перед созданием новой записи выполняется проверка на дублирование: система ищет, была ли уже зафиксирована такая ошибка для данного датчика и времени измерения. Если совпадающая запись отсутствует, в таблицу sensor_error добавляется новая запись об отклонении. Это позволяет автоматически фиксировать выход параметров за допустимый диапазон и отображать такие события в журнале ошибок интерфейса.

Листинг 12 — Создание записи об ошибке при выходе параметра за установленный диапазон

```
for (const row of rows) {
  const min = row.ref_min;
  const max = row.ref_max;
  const val = Number(row.value);
  const outLow = min != null && !Number.isNaN(val) && val < Number(min);
  const outHigh = max != null && !Number.isNaN(val) && val > Number(max);
  if (!outLow && !outHigh) continue;
  if (!row.sensor_id) continue;

  const code = `OUT-${row.id_parameter}`;
  const name = `Вне диапазона: ${row.parameter ?? row.id_parameter}`;

  const [exists] = await pool.query(
    `SELECT id_sensor_error
     FROM sensor_error
     WHERE id_sensor_number = ? AND timestamp = ? AND error_code IN
(${codePlaceholders})
     LIMIT 1`,
    [row.sensor_id, row.ts, ...duplicateCodes],
  );

  if (!exists.length) {
    await pool.query(
      `INSERT INTO sensor_error (error_code, error_name, timestamp,
id_sensor_number)
       VALUES (?, ?, ?, ?)`,
      [code, name, row.ts, row.sensor_id],
    );
  }
}
```

Функция check выполняет проверку журнала ошибок и выбирает событие, которое необходимо отобразить оператору в виде уведомления (Листинг 13). Она получает список ошибок с сервера, проверяет наличие актуальных записей и отбирает только те события, которые требуют реакции пользователя.

Если такое событие найдено, оно сохраняется в состоянии notice, после чего в интерфейсе отображается модальное окно с рекомендацией для оператора. Это позволяет своевременно уведомлять оператора об отклонениях параметров и не перегружать интерфейс лишними сообщениями.

Листинг 13 — Отображение предупреждения оператору

```
const check = async () => {
  const rows = await getErrors();
  if (cancelled || !Array.isArray(rows) || !rows.length) return;

  const correctionAlert = rows
    .filter((row) => isAlert(row) && isHumidityTemperatureCorrection(row))
    .filter((row) => toNumber(row.id) > lastCorrectionHandledRef.current &&
isFreshEvent(row))
    .sort((a, b) => toNumber(b.id) - toNumber(a.id))[0];

  if (correctionAlert) {
    setNotice(correctionAlert);
    initializedRef.current = true;
    return;
  }

  const latestAlert = rows
    .filter((row) => isAlert(row) && !isNdviA2Alert(row)
&& !isSilentJournalAlert(row))
    .sort((a, b) => toNumber(b.id) - toNumber(a.id))[0];

  setNotice(latestAlert);
};
```

3.3 Разработка пользовательского интерфейса

Интерфейс реализован в виде одностраничного приложения, где пользователь может переходить между разделами без полной перезагрузки страницы.

При входе в приложение пользователю отображается окно авторизации. На данной странице сотрудник вводит свои данные для доступа к системе.

После успешной авторизации открывается главная страница веб-приложения, где отображается навигационное меню и доступные функциональные разделы.

Авторизация необходима для того, чтобы фиксировать действия пользователя: изменение эталонных значений или запуск принудительного полива в журнале действий (Рисунок 3.2).

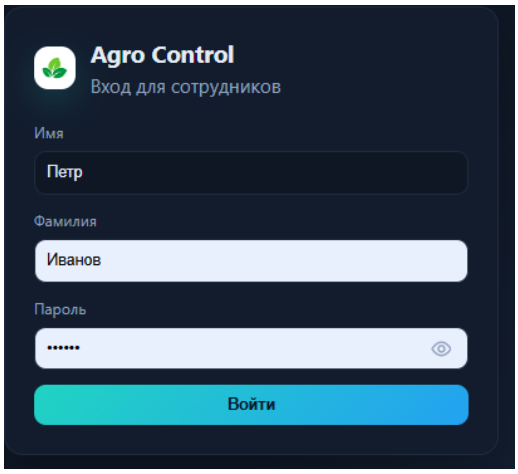


Рисунок 3.2 — Страница авторизации

На главной странице отображаются ключевые параметры системы: температура воздуха, относительная влажность воздуха, уровень концентрации углекислого газа. Значения представлены в виде карточек и графиков, что позволяет быстро оценить текущее состояние микроклимата. Для каждого параметра предусмотрена визуальная индикация состояния. Если показатель находится в пределах нормы, он отображается как нормальный, а при отклонении система выделяет его как предупреждение или критическое состояние. Также на главной странице отображаются графики изменения параметров микроклимата во времени, благодаря чему пользователь может отслеживать динамику, а не только текущее значение (Рисунок 3.3).



Рисунок 3.3 — Главная страница

Блок отчетности на главной странице предназначен для просмотра и анализа изменения параметров системы во времени. В нем пользователь может выбрать необходимый показатель и увидеть его динамику на графике. В отчетности доступны такие параметры, как температура воздуха, влажность воздуха, CO_2 и NDVI.

Особое значение имеет режим сравнения NDVI для разных технологий выращивания. В этом режиме на одном графике отображаются значения NDVI для гидропоники и аэропоники, что позволяет сравнить состояние растений при разных способах выращивания (Рисунок 3.4).

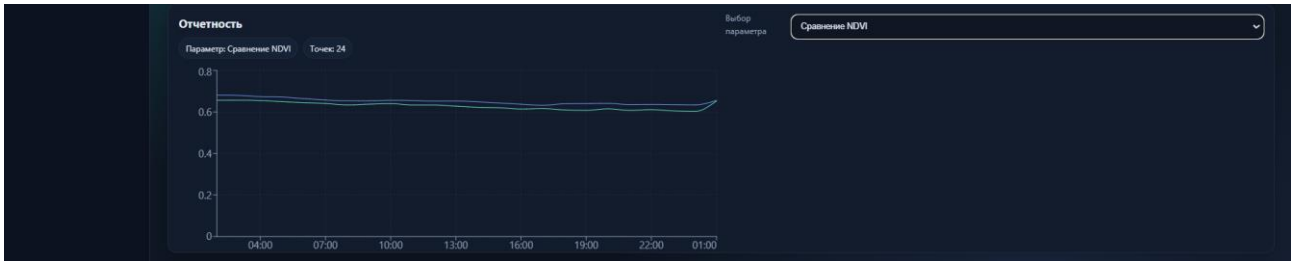


Рисунок 3.4 — Блок отчетности на главной странице

На главной странице приложения также расположен блок изменения уставок, предназначенный для настройки допустимых диапазонов контролируемых параметров. Данный блок позволяет пользователю изменять минимальные и максимальные значения, с которыми система сравнивает поступающие от датчиков данные (Рисунок 3.5).



Рисунок 3.5 — Блок изменения уставок на главной странице

Отдельный раздел интерфейса посвящен зонам полива, где пользователь видит список всех зон выращивания, разделенных по технологиям и зонам. Для каждой зоны отображается текущее значение NDVI и статус полива.

Пользователь может воспользоваться поиском по названию зоны, отфильтровать зоны по технологии, а также выбрать одну или несколько зон для принудительного запуска полива (Рисунок 3.6).

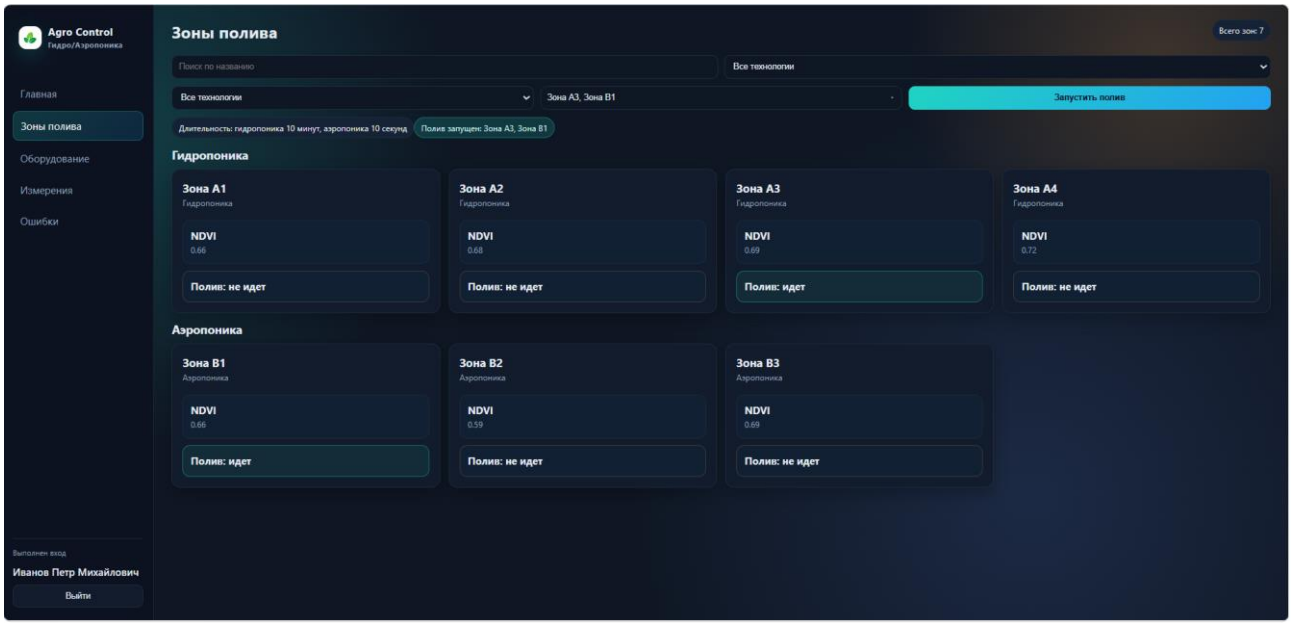


Рисунок 3.6 — Вкладка «Зоны полива»

На вкладке «Оборудование» отображается информация о датчиках и исполнительных устройствах (Рисунок 3.7).

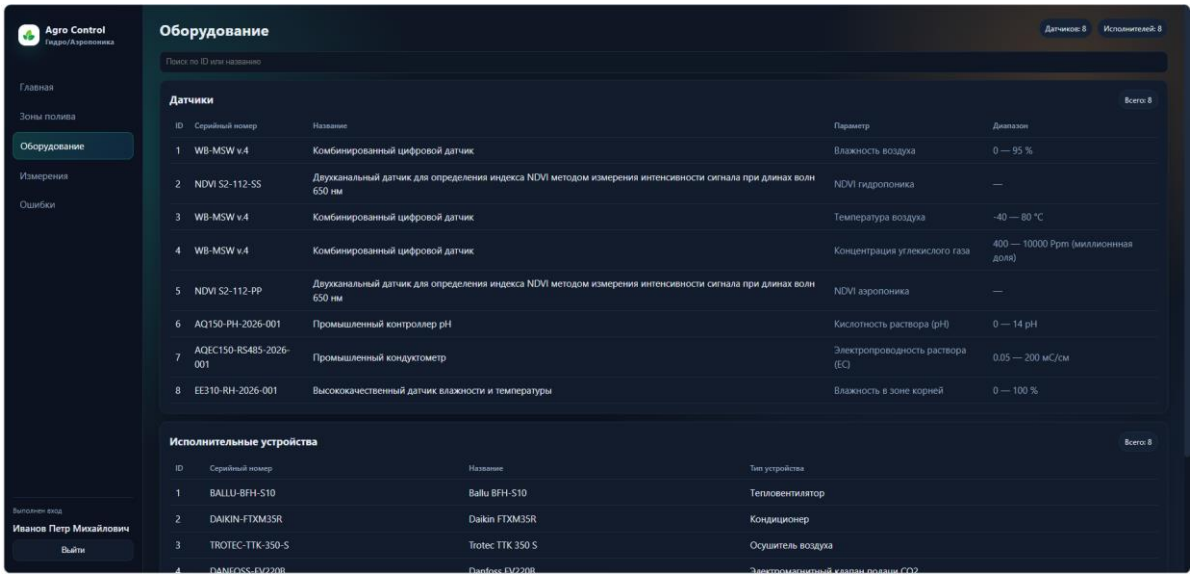


Рисунок 3.7 — Вкладка «Оборудование»

Вкладка «Измерения» предназначена для просмотра зарегистрированных значений параметров. Здесь отображаются данные, поступающие с датчиков, включая время измерения, параметр, значение и единицу измерения. Этот раздел используется для более подробного анализа исходных данных, на основе которых строятся графики, отчеты и уведомления.

Он позволяет проверить, какие именно значения были получены системой в определенный момент времени (Рисунок 3.8).

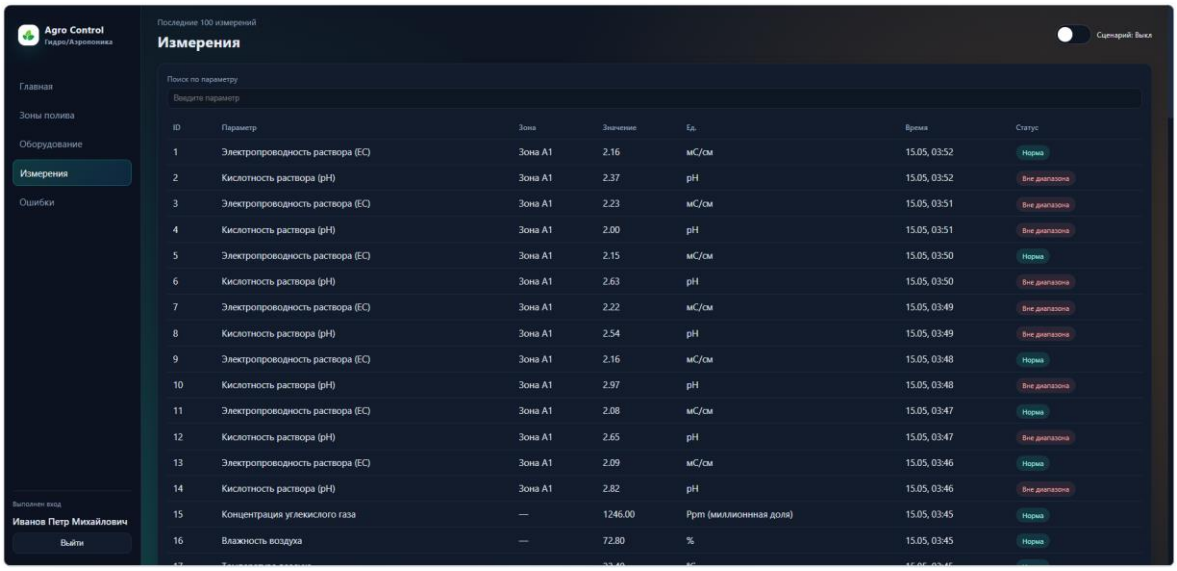


Рисунок 3.8 — Вкладка «Измерения»

Во вкладке «Ошибки» отображаются предупреждения и аварийные события. Каждая ошибка содержит время возникновения, код, описание и статус.

Такой раздел позволяет пользователю быстро определить, какие отклонения были зафиксированы системой (Рисунок 3.9).

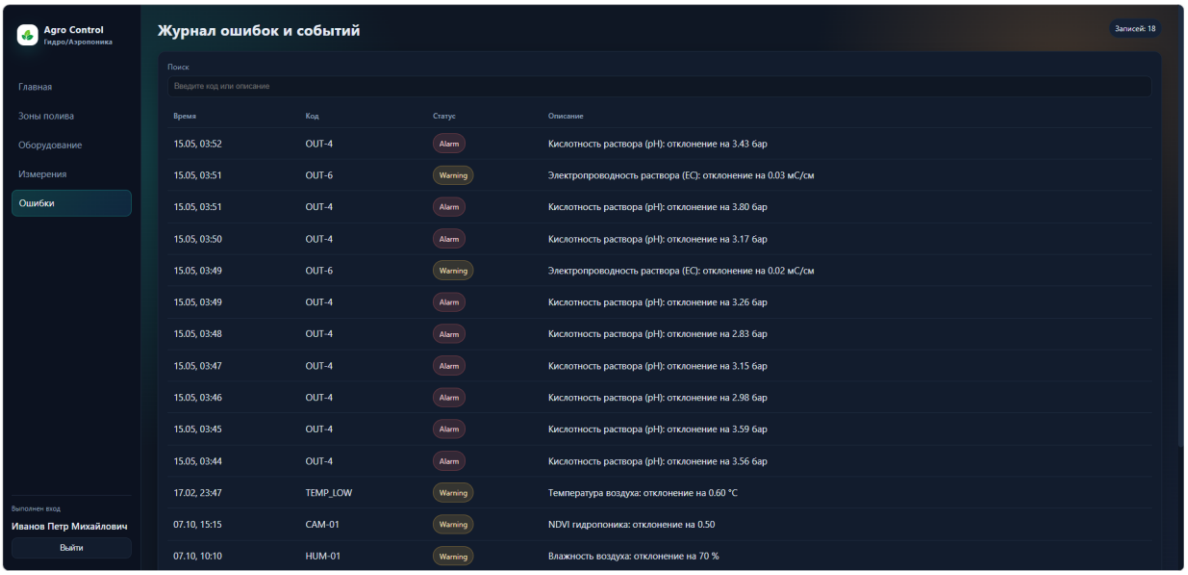


Рисунок 3.9 — Вкладка «Ошибки»

При снижении значения NDVI оператор получает предупреждение с описанием выявленной проблемы.

В уведомлении отображаются текущие параметры микроклимата, а также график изменения индекса NDVI для той зоны, в которой зафиксировано отклонение (Рисунок 3.10).

Это позволяет оператору оценить, связано ли ухудшение состояния растений с нарушением температурного режима, влажности, концентрации CO_2 или с другой причиной.

Если параметры микроклимата находятся в допустимых пределах, снижение NDVI можно рассматривать как признак локального кратковременного водно-питательного стресса. В таком случае вероятной причиной является временный недостаток влаги в корневой зоне, а не общее нарушение условий выращивания. Это может произойти при кратковременном снижении уровня раствора, задержке срабатывания насоса или клапана, неравномерной подаче раствора после обслуживания системы либо временном нарушении подачи в отдельную зону.

Поэтому оператору предлагается перейти на вкладку зон полива и запустить сеанс принудительного полива для проблемной зоны. Данное действие позволяет быстро восстановить подачу влаги и питательного раствора к корневой системе растений.

Если отклонение было разовым, после выполнения принудительного полива состояние растений постепенно нормализуется, а значение NDVI возвращается к допустимому уровню.

Если после принудительного полива NDVI не восстанавливается или снижение повторяется, это указывает уже не на разовое отклонение, а на системную проблему. В таком случае оператору необходимо проверить работу оборудования или скорректировать расписание полива.

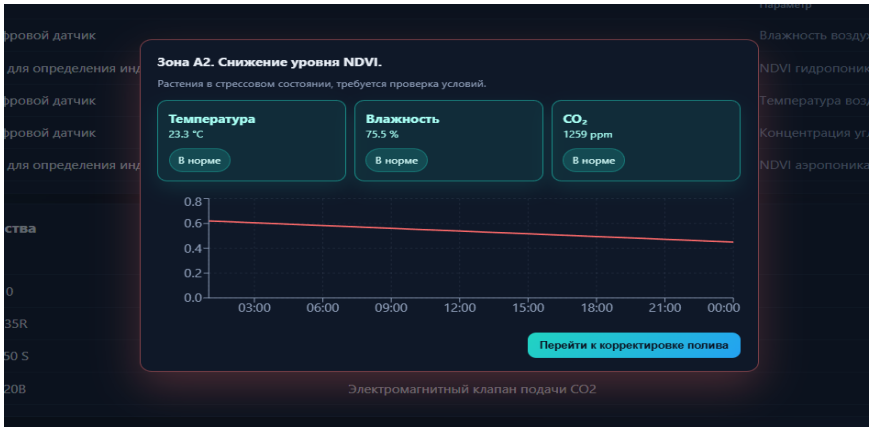


Рисунок 3.10 — Модальное окно с предупреждением о снижении индекса NDVI

При повышении влажности воздуха выше допустимого значения система фиксирует отклонение от установленного диапазона микроклимата. Влажность является одним из основных параметров, влияющих на состояние растений и условия их выращивания, поэтому ее превышение требует реакции со стороны оператора.

После фиксации отклонения системой, оператор получает уведомление с описанием проблемы. В уведомлении указывается, что значение влажности превышает допустимый уровень, и предлагается перейти в раздел изменения уставок для корректировки температурного режима (Рисунок 3.11).

Повышение температуры в данной ситуации может использоваться как корректирующее действие, так как более теплый воздух способен удерживать больше водяного пара.

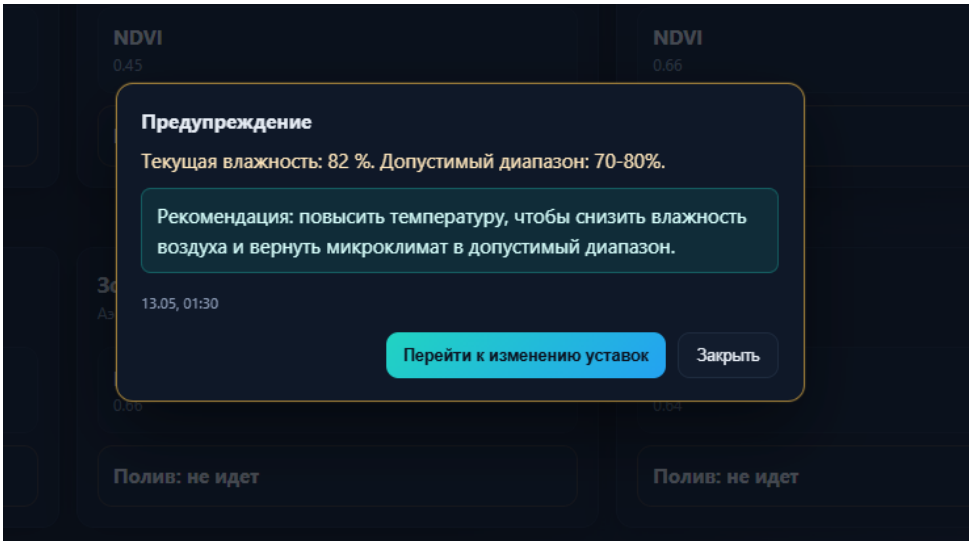


Рисунок 3.11 — Модальное окно с предупреждением о повышении влажности

ЗАКЛЮЧЕНИЕ

В результате выполнения выпускной квалификационной работы была разработана информационно-управляющая система, обеспечивающая автоматизированное управление процессом выращивания листового салата в совмещенной гидропонно-аэропонной установке. Разрабатываемая система предназначена для мониторинга параметров микроклимата, контроля состояния растений, управления поливом и своевременного информирования оператора об отклонениях технологических параметров.

В работе был рассмотрен технологический процесс выращивания листового салата в условиях контролируемой среды. Для сопоставления двух технологий выращивания установка организована как единое производственное помещение с общим микроклиматом, внутри которого выделяются гидропонная и аэропонная зоны. В гидропонной зоне используется метод периодического затопления, а в аэропонной зоне — ультразвуковая аэропоника. Такое решение позволяет обеспечить одинаковые условия воздушной среды для обеих технологий и сравнивать их эффективность по состоянию растений.

Для обеспечения благоприятных условий выращивания растений были определены основные параметры контроля и регулирования. Параметры микроклимата используются для поддержания условий роста листового салата, а индекс NDVI применяется для оценки физиологического состояния растений и эффективности полива. Использование NDVI позволяет выявлять ухудшение состояния растений до появления выраженных внешних признаков и использовать полученные данные для корректировки режима полива.

Для описания логики функционирования системы были разработаны модели в нотациях IDEF0, DFD и BPMN. Также была разработана архитектура программного обеспечения информационно-управляющей системы, описывающая взаимодействие оператора, программных подсистем и технической инфраструктуры.

В рамках программной реализации была разработана база данных, предназначенная для хранения информации о технологическом процессе.

Для реализации функций мониторинга и управления создано веб-приложение, обеспечивающее взаимодействие оператора с системой. Разработанное решение позволяет повысить эффективность процесса выращивания растений в замкнутой установке за счет централизованного контроля параметров, своевременного выявления отклонений и оперативной корректировки режимов работы системы.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Гущин А. В. Применение аэропонных технологий для адаптации микроклонов растений разных таксономических групп: канд. биол. наук / А. В. Гущин; ФГБОУ ВО «Российский государственный аграрный университет – МСХА им. К. А. Тимирязева». — Москва, 2023. — 26 с. — URL: https://rusneb.ru/catalog/000199_000009_012156810/ (Дата обращения: 19.12.2025).

2. Автоматизация выращивание растений в вертикальной гидропонной сити-ферме / Wiren Board. — URL: https://wirenboard.com/ru/pages/greenery-city/ (Дата обращения: 19.12.2025).

3. Greenery Food | Микрозелень | Съедобные цветы | Микс салатов | Санкт-Петербург. — URL: https://greenery.city/#rec548559478 (Дата обращения: 19.12.2025).

4. ГОСТ 34.602-2020 Информационная технология. Автоматизированные системы. Требования к проектной документации. — М.: Стандартинформ, 2020. — 48 с. (Дата обращения: 19.12.2025)

5. Амаева, А. Г. Сенсорная система учета азотно-углеродного баланса в гидропонной технологии выращивания огурцов и томатов (Разработка прототипа автоматизированной системы оптимизации корневого углеродного питания огурцов и томатов) : учебное пособие / А. Г. Амаева, Н. Л. Адаев. — Грозный: ЧГУ им. А.А. Кадырова, 2022. — 90 с. — Текст: электронный // Лань: электронно-библиотечная система. — URL: https://e.lanbook.com/book/339824 (Дата обращения: 19.12.2025).

6. Гидропоника с автоматизированным регулированием системы полива / В. В. Волков, V. V. Volkov, Е. А. Басуматорова, E. A. Basumatorova // Известия Оренбургского государственного аграрного университета. — 2024. — № 5 (109). — С. 193-198. — ISSN 2073-0853. — Текст: электронный // Лань: электронно-библиотечная система. — URL: https://e.lanbook.com/journal/issue/366221 (Дата обращения: 19.12.2025).

7. Применение аэропоники для адаптации микроклонов растений рода rubus l / Е. А. Калашникова, Р. Н. Киракосян, И. С. Чуксин [и др.] // Проблемы развития АПК региона. — 2019. — № 40. — С. 64-71. — ISSN 2079-0996. — Текст: электронный // Лань: электронно-библиотечная система. — URL: https://e.lanbook.com/journal/issue/312279 (Дата обращения: 19.12.2025).

8. Аэропонические фермы / С. В. Мальчихин, S. V. Malchikhin, С. М. Каюгина, S. M. Kayugina // Мир Инноваций. — 2021. — № 2. — С. 18-21. — ISSN 2687-0460. — Текст: электронный // Лань: электронно-библиотечная система. — URL: https://e.lanbook.com/journal/issue/369020 (Дата обращения: 19.12.2025).

9. Обзор способов и перспективы развития информационно-измерительных и управляющих систем выращивания растений в условиях городской среды / А. Д. Сашина, A. D. Sashina, А. О. Березина [и др.] // Измерение. Мониторинг. Управление. Контроль. — 2023. — № 4 (46). — С. 32-42. — ISSN 2307-5538. — Текст: электронный // Лань: электронно-библиотечная система. — URL: https://e.lanbook.com/journal/issue/350903 (Дата обращения: 19.12.2025).

10. Ампилогов, В. А. Теоретические основы автоматизированного управления. Лабораторный практикум / В. А. Ампилогов. — 2-е изд., стер. — Санкт-Петербург: Лань, 2023. — 208 с. — ISBN 978-5-507-47250-5. — Текст: электронный // Лань: электронно-библиотечная система. — URL: https://e.lanbook.com/book/349964 (Дата обращения: 15.04.2026).

11. Гвоздева, Т. В. Проектирование информационных систем: технология автоматизированного проектирования. Лабораторный практикум: учебное пособие / Т. В. Гвоздева, Б. А. Баллод. — 2-е изд., стер. — Санкт-Петербург: Лань, 2020. — 156 с. — ISBN 978-5-8114-5147-0. — Текст: электронный // Лань: электронно-библиотечная система. — URL: https://e.lanbook.com/book/133477 (Дата обращения: 15.04.2026).

12. Аншина, М. Л. Архитектура приложений и данных: учебное пособие / М. Л. Аншина. — Москва: РТУ МИРЭА, 2024. — 152 с. — ISBN 978-5-7339-2218-8. — Текст: электронный // Лань: электронно-библиотечная система. — URL: https://e.lanbook.com/book/421100 (Дата обращения: 15.04.2026).

13. ГОСТ 19.701–90. Схемы алгоритмов, программ, данных и систем. Условные обозначения и правила выполнения. — Москва: Издательство стандартов, 1990. (Дата обращения: 15.04.2026).

14. Influence of day and night temperature and carbon dioxide concentration on growth, yield, and quality of green butterhead and red oakleaf lettuce — URL: https://journals.plos.org/plosone/article?id=10.1371%2Fjournal.pone.0313884 (Дата обращения: 20.04.2026).

15. Hydroponic Lettuce Production in Controlled Environments — URL: https://ccd.uky.edu/sites/default/files/2026-02/ccd-cp-63_hydrolettuce-updated_accessible.pdf (Дата обращения: 20.04.2026).

16. Optimal control of environmental conditions affecting lettuce plant growth in a controlled environment with artificial lighting: A review — URL: https://www.sciencedirect.com/science/article/pii/S0254629919315315 (Дата обращения: 20.04.2026).

17. Smart plasma-enhanced aeroponic cultivation of lettuce (lactuca sativa l.): Comparative evaluation with soil and hydroponic systems on growth responses, nitrogen absorption, and nutritional quality — URL: https://www.sciencedirect.com/science/article/pii/S277237552501007X?utm_source=chatgpt.com (Дата обращения: 20.04.2026).

18. Маркова А. Е., Мишанов А. П., Ракутько С. А., Ракутько Е. Н. Поддержание оптимальных значений кислотности и электропроводности питательного раствора в рабочем цикле светокультуры салата // АгроЭкоИнженерия. 2016. №89. URL: https://cyberleninka.ru/article/n/podderzhanie-optimalnyh-znacheniy-kislotnosti-i-elektroprovodnosti-pitatelnogo-rastvora-v-rabochem-tsikle-svetokultury-salata (Дата обращения: 14.05.2026).

19. Shen, P.; Wang, L.; Qureshi, W.A.; Gao, J. Influence of Rhizosphere Temperature and Humidity Regulation on Rooting, Mortality, and Transplant Survival of Aeroponically Rapid Growth Mulberry Cutting. *Agronomy* 2025, *15*, 583. https://doi.org/10.3390/agronomy15030583 (Дата обращения: 14.05.2026).

20. UV-NDVI for real-time crop health monitoring in vertical farms — URL: https://www.sciencedirect.com/science/article/pii/S2772375524000674?utm_source=chatgpt.com (Дата обращения: 20.04.2026).

21. Диков, А. В. Web-программирование на JavaScript: учебное пособие для вузов / А. В. Диков. — Санкт-Петербург: Лань, 2026. — 436 с. — ISBN 978-5-507-54625-1. — Текст: электронный // Лань: электронно-библиотечная система. — URL: https://e.lanbook.com/book/518225 (Дата обращения: 08.05.2026).

22. Куликов, А. А. Разработка клиент-серверных приложений: учебное пособие / А. А. Куликов. — Москва: РТУ МИРЭА, 2023 — Часть 1 — 2023. — 98 с. — ISBN 978-5-7339-1858-7. — Текст: электронный // Лань: электронно-библиотечная система. — URL: https://e.lanbook.com/book/382556 (Дата обращения: 08.05.2026).

Приложение А

Листинг А.1 — Содержание файла backend/index.js

```
import express from 'express';
import cors from 'cors';
import mysql from 'mysql2/promise';
import dotenv from 'dotenv';
import crypto from 'crypto';
import http from 'http';
import { WebSocketServer } from 'ws';

dotenv.config();

const {
  DB_HOST = 'localhost',
  DB_PORT = 3306,
  DB_USER = 'root',
  DB_PASS = '',
  DB_NAME = 'plants',
  DB_TZ = '+03:00',
  PORT = 3000,
} = process.env;

// Создание пула подключений к базе данных MySQL
function createPool() {
  return mysql.createPool({
    host: DB_HOST,
    port: DB_PORT,
    user: DB_USER,
    password: DB_PASS,
    database: DB_NAME,
    timezone: DB_TZ,
    waitForConnections: true,
    connectionLimit: 10,
  });
}
const poolPromise = createPool();

const app = express();
app.use(cors());
app.use(express.json());

app.get('/api/health', (_req, res) => res.json({ status: 'ok' }));
// Получение списка колонок указанной таблицы.
async function getColumns(pool, table) {
  try {
    const [cols] = await pool.query(`SHOW COLUMNS FROM \`${table}\``);
    return cols.map((c) => c.Field);
  } catch (e) {
    return [];
  }
}

// Удаление лишних пробелов в строке
const normalizeSpace = (value) => String(value || '').trim().replace(/\s+/g, '
');

// Создание карты колонок для поиска без учета регистра
const buildColumnMap = (cols) => {
  const map = new Map();
  cols.forEach((c) => map.set(c.toLowerCase(), c));
```

Продолжение Листинга А.1

```
  return map;
};

// Выбор первой подходящей колонки из списка возможных названий
const pickColumn = (map, candidates) => candidates.map((c) =>
map.get(c)).find(Boolean);

// Выбор всех найденных колонок из списка возможных названий
const pickColumns = (map, candidates) => candidates.map((c) =>
map.get(c)).filter(Boolean);

// Проверка пароля пользователя
const verifyPassword = (password, stored) => {
  const raw = String(stored || '');
  if (!raw) return false;
  if (raw.startsWith('scrypt$')) {
    const parts = raw.split('$');
    if (parts.length !== 3) return false;
    const salt = parts[1];
    const expected = parts[2];
    if (!salt || !expected) return false;
    if (!/^[0-9a-f]+$/i.test(expected) || expected.length % 2 !== 0) return
false;
    const derived = crypto.scryptSync(String(password || ''), salt,
64).toString('hex');
    if (expected.length !== derived.length) return false;
    return crypto.timingSafeEqual(Buffer.from(expected, 'hex'),
Buffer.from(derived, 'hex'));
  }
  return raw === String(password || '');
};

// Запись действия оператора в журнал actions_log
async function logAction(pool, { employeeId, parameterId, actionText }) {
  const cols = await getColumns(pool, 'actions_log');
  if (!cols.length) return;
  const empCol = cols.find((c) => ['id_employee', 'employee_id', 'id_user',
'user_id'].includes(c));
  const paramCol = cols.find((c) => ['id_parameter',
'parameter_id'].includes(c));
  const tsCol = cols.find((c) => ['timestamp', 'created_at', 'ts',
'time'].includes(c));
  const actionCol = cols.find((c) => ['action', 'event',
'description'].includes(c));
  const fields = [];
  const values = [];
  if (empCol) {
    fields.push(empCol);
    values.push(employeeId ?? null);
  }
  if (paramCol) {
    fields.push(paramCol);
    values.push(parameterId ?? null);
  }
  if (actionCol) {
    fields.push(actionCol);
    values.push(actionText ?? 'Изменение параметра');
  }
  if (tsCol) {
    fields.push(tsCol);
    values.push(new Date());
  }
```

Продолжение Листинга А.1

```
  if (!fields.length) return;
  const placeholders = fields.map(() => '?').join(',');
  const sql = `INSERT INTO actions_log (${fields.join(',')}) VALUES
(${placeholders})`;
  try {
    await pool.query(sql, values);
  } catch (e) {
    console.error('actions_log insert error', e);
  }
}

// Запись важного действия пользователя в audit_log
async function logAudit(pool, { employeeId, action }) {
  const cols = await getColumns(pool, 'audit_log');
  if (!cols.length) return;
  const empCol = cols.find((c) => ['id_employee', 'employee_id', 'id_user',
'user_id'].includes(c));
  const actionCol = cols.find((c) => ['action', 'event',
'description'].includes(c));
  const tsCol = cols.find((c) => ['timestamp', 'created_at', 'ts',
'time'].includes(c));
  const fields = [];
  const values = [];
  if (empCol) {
    fields.push(empCol);
    values.push(employeeId ?? null);
  }
  if (actionCol) {
    fields.push(actionCol);
    values.push(action ?? 'Изменение параметра');
  }
  if (tsCol) {
    fields.push(tsCol);
    values.push(new Date());
  }
  if (!fields.length) return;
  const placeholders = fields.map(() => '?').join(',');
  const sql = `INSERT INTO audit_log (${fields.join(',')}) VALUES
(${placeholders})`;
  try {
    await pool.query(sql, values);
  } catch (e) {
    console.error('audit_log insert error', e);
  }
}

// Запуск восстановления влажности после изменения уставки температуры
async function startHumidityRecoveryAfterTemperatureReferenceUpdate(pool,
parameterId) {
  if (Number(parameterId) !== 1) return;
  try {
    await pool.query(
      `UPDATE scenario_state
       SET text_value = 'recovering',
           updated_at = CURRENT_TIMESTAMP
       WHERE scenario_key = 'humidity_temperature_correction'
         AND COALESCE(numeric_value, 0) > 70`,
    );
  } catch (e) {
    if (e?.code !== 'ER_NO_SUCH_TABLE') {
      console.error('humidity recovery state update error', e);
    }
```

Продолжение Листинга А.1

```
  }
}

// Преобразование значения в число или null
const toNumber = (value) => {
  const num = Number(value);
  return Number.isNaN(num) ? null : num;
};

// Формирование уникального ключа измерения по датчику и времени
const measurementKey = (sensorId, ts) => {
  const time = new Date(ts).getTime();
  if (!sensorId || Number.isNaN(time)) return null;
  return `${sensorId}:${time}`;
};

// Определение уровня отклонения параметра от допустимого диапазона
const getDeviationSeverity = (value, refMin, refMax) => {
  const val = toNumber(value);
  const min = toNumber(refMin);
  const max = toNumber(refMax);
  if (val === null || min === null || max === null) return null;
  if (val >= min && val <= max) return null;
  const delta = val < min ? min - val : val - max;
  const range = max - min;
  const threshold = Math.max(range * 0.05, 2);
  return delta <= threshold ? 'warning' : 'critical';
};

// Распознавание известных кодов предупреждений по CO2 и влажности
const knownAlertByCode = (code) => {
  const normalized = String(code || '').toLowerCase();
  if (normalized.includes('co2')) {
    if (normalized.includes('alarm')) return { parameterId: 3, severity:
'critical' };
    if (normalized.includes('warning')) return { parameterId: 3, severity:
'warning' };
  }
  if (normalized.includes('hum')) {
    if (normalized.includes('alarm')) return { parameterId: 2, severity:
'critical' };
    if (normalized.includes('warning')) return { parameterId: 2, severity:
'warning' };
  }
  return null;
};

// Определение идентификатора параметра по коду ошибки
const parameterIdFromErrorCode = (code) => {
  const known = knownAlertByCode(code);
  if (known?.parameterId) return known.parameterId;
  const match = String(code || '').match(/^OUT-(\d+)$/i);
  return match ? Number(match[1]) : null;
};
...

// Формирование снимка главной панели: текущие параметры и активные ошибки
async function buildDashboardSnapshot(pool) {
  const aggregates = { temperature: null, humidity: null, co2: null };
  let errors = [];
  let updatedAt = Date.now();
  const paramIds = { temperature: 1, humidity: 2, co2: 3 };
```

Продолжение Листинга А.1

```
  // Загрузка последних значений температуры, влажности и CO2
  const fetchLatest = async () => {
    try {
      const [rows] = await pool.query(
        `SELECT id_parameter, value, timestamp, sensor_id
         FROM (
           SELECT m.id_parameter,
                  m.value,
                  m.timestamp,
                  m.id_sensors AS sensor_id,
                  ROW_NUMBER() OVER (
                    PARTITION BY m.id_parameter
                    ORDER BY m.timestamp DESC, m.id_measurements DESC
                  ) AS rn
           FROM sensors_measurements m
           WHERE m.id_parameter IN (1, 2, 3)
             AND m.timestamp >= DATE_SUB(NOW(), INTERVAL 5 MINUTE)
         ) latest
         WHERE rn = 1
         ORDER BY timestamp DESC`,
      );
      return rows;
    } catch (e) {
      if (e?.code === 'ER_NO_SUCH_TABLE') {
        try {
          const [rows] = await pool.query(
            `SELECT id_parameter, value, timestamp, sensor_id
             FROM (
               SELECT m.id_parameter,
                      m.value,
                      m.timestamp,
                      m.id_sensors AS sensor_id,
                      ROW_NUMBER() OVER (
                        PARTITION BY m.id_parameter
                        ORDER BY m.timestamp DESC
                      ) AS rn
               FROM sensor_measurements m
               WHERE m.id_parameter IN (1, 2, 3)
                 AND m.timestamp >= DATE_SUB(NOW(), INTERVAL 5 MINUTE)
             ) latest
             WHERE rn = 1
             ORDER BY timestamp DESC`,
          );
          return rows;
        } catch {
          return [];
        }
      }
      return [];
    }
  };

  const latest = await fetchLatest();
  if (latest?.[0]?.timestamp) {
    const ts = new Date(latest[0].timestamp).getTime();
    if (!Number.isNaN(ts)) updatedAt = ts;
  }
  const seen = new Set();
  const scenarioKeys = tempScenario.sampleKeys && tempScenario.sampleKeys.size ?
tempScenario.sampleKeys : null;
  for (const row of latest) {
    const pid = Number(row.id_parameter);
```

Продолжение Листинга А.1

```
    if (seen.has(pid)) continue;
    if (pid === paramIds.temperature) {
      if (tempScenario.active && scenarioKeys) {
        const key = measurementKey(row.sensor_id, row.timestamp);
        if (!key || !scenarioKeys.has(key)) {
          continue;
        }
      }
      aggregates.temperature = row.value;
    }
    else if (pid === paramIds.humidity) aggregates.humidity = row.value;
    else if (pid === paramIds.co2) aggregates.co2 = row.value;
    seen.add(pid);
    if (seen.size === 3) break;
  }
  if (tempScenario.active && Number.isFinite(tempScenario.value)) {
    aggregates.temperature = tempScenario.value;
  }

  try {
    const [errRows] = await pool.query(
      `SELECT id_sensor_error AS id,
              timestamp AS ts,
              error_code AS code,
              error_name AS text,
              id_sensor_number AS sensorId,
              'warning' AS severity
       FROM sensor_error
       ORDER BY timestamp DESC
       LIMIT 100`,
    );
    errors = errRows;
  } catch {
    errors = [];
  }

  return { aggregates, errors, updatedAt };
}
...

// Принудительный полив нескольких зон
app.post('/api/zones/water/force', async (req, res, next) => {
  try {
    const pool = await poolPromise;
    const { zoneIds, startedByEmployee } = req.body || {};
    if (!Array.isArray(zoneIds) || !zoneIds.length) {
      return res.status(400).json({ error: 'zoneIds must be a non-empty array'
});
    }

    const ids = Array.from(
      new Set(
        zoneIds
          .map((v) => Number(v))
          .filter((v) => Number.isFinite(v) && Number.isInteger(v)),
      ),
    );
    if (!ids.length) {
      return res.status(400).json({ error: 'zoneIds must contain numeric ids'
});
    }
```

Продолжение Листинга А.1

```
    const startedBy =
      startedByEmployee === undefined || startedByEmployee === null ||
startedByEmployee === ''
        ? null
        : startedByEmployee;

    // Сборка запроса выбранных зон для принудительного полива
    const buildZonesSql = (techCol) =>
      `SELECT id_zone AS id, zone_name AS name, ${techCol} AS technology
       FROM zones
       WHERE id_zone IN (?)`;

    let zoneRows;
    try {
      [zoneRows] = await pool.query(buildZonesSql('id_technolog'), [ids]);
    } catch (e) {
      if (e?.code === 'ER_BAD_FIELD_ERROR') {
        try {
          [zoneRows] = await pool.query(buildZonesSql('id_tehnolog'), [ids]);
        } catch (inner) {
          if (inner?.code === 'ER_BAD_FIELD_ERROR') {
            [zoneRows] = await pool.query(buildZonesSql('id_technology'),
[ids]);
          } else {
            throw inner;
          }
        }
      } else {
        throw e;
      }
    }

    const zoneMap = new Map(zoneRows.map((z) => [Number(z.id), z]));
    const results = [];

    for (const id of ids) {
      const zone = zoneMap.get(Number(id));
      if (!zone) {
        results.push({ id, ok: false, error: 'Zone not found' });
        continue;
      }

      let duration = null;
      if (zone.technology === 1) duration = 600;
      else if (zone.technology === 2) duration = 10;
      else {
        results.push({ id, ok: false, error: 'Unknown technology' });
        continue;
      }

      try {
        const [resultSets] = await pool.query(
          'CALL start_irrigation_for_zone(?, ?, ?, ?)',
          [id, 'MANUAL', duration, startedBy],
        );
        let runId = null;
        if (Array.isArray(resultSets)) {
          for (const set of resultSets) {
            if (Array.isArray(set) && set.length && set[0]?.id_run != null) {
              runId = set[0].id_run;
              break;
            }
```

Продолжение Листинга А.1

```
          }
        }
        results.push({ id, ok: true, runId });
      } catch (err) {
        results.push({ id, ok: false, error: err?.message || 'Failed to start
irrigation' });
      }
    }

    const success = results.filter((r) => r.ok).length;
    const failed = results.length - success;
    res.json({ ok: failed === 0, success, failed, results });
  } catch (err) {
    next(err);
  }
});
...

// Обновление уставок и запись действия в журнал
app.post('/api/reference', async (req, res, next) => {
  try {
    const pool = await poolPromise;
    const { parameterId, min_value, max_value, employeeId } = req.body || {};
    const paramId = Number(parameterId);
    const min = Number(min_value);
    const max = Number(max_value);
    if (Number.isNaN(paramId) || Number.isNaN(min) || Number.isNaN(max)) {
      return res.status(400).json({ error: 'Минимум/максимум должны быть
числами' });
    }
    if (min > max) {
      return res.status(400).json({ error: 'Минимум не может быть больше
максимума' });
    }
    const cols = await getColumns(pool, 'reference_values');
    const idCol = cols.includes('id_parameter') ? 'id_parameter' :
cols.includes('id_parameters') ? 'id_parameters' : null;
    if (!idCol) return res.status(500).json({ error: 'reference_values: нет поля
id_parameter/id_parameters' });
    let paramName = '';
    try {
      const [pinfo] = await pool.query('SELECT name_parameter FROM parameters
WHERE id_parameter = ? LIMIT 1', [paramId]);
      paramName = pinfo?.[0]?.name_parameter || '';
    } catch (e) {
      paramName = '';
    }
    const [upd] = await pool.query(
      `UPDATE reference_values
       SET min_value = ?, max_value = ?
       WHERE ${idCol} = ?`,
      [min, max, paramId],
    );
    if (upd.affectedRows === 0) {
      await pool.query(
        `INSERT INTO reference_values (${idCol}, min_value, max_value)
         VALUES (?, ?, ?)`,
        [paramId, min, max],
      );
    }
    if (employeeId != null) {
      const actionText = paramName
```

Продолжение Листинга А.1

```
        ? `Изменение диапазона "${paramName}"`
        : `Изменение диапазона параметра ${paramId}`;
      await logAction(pool, { employeeId, parameterId: paramId, actionText });
      await logAudit(pool, { employeeId, action: actionText });
    }
    await startHumidityRecoveryAfterTemperatureReferenceUpdate(pool, paramId);
    res.json({ ok: true, parameterId: paramId, min_value: min, max_value: max
});
  } catch (err) {
    next(err);
  }
});

// Журнал ошибок
app.get('/api/errors', async (_req, res) => {
  const pool = await poolPromise;
  const cols = await getColumns(pool, 'sensor_error');
  const severityCol = cols.find((c) => ['severity', 'level', 'status',
'priority'].includes(c));
  const severitySelect = severityCol ? `se.${severityCol} AS severity` :
`'warning' AS severity`;
  const [rows] = await pool.query(
    `SELECT se.id_sensor_error AS id,
            se.timestamp AS ts,
            se.error_code AS code,
            se.error_name AS text,
            ${severitySelect},
            se.id_sensor_number AS sensorId
     FROM sensor_error se
     ORDER BY se.timestamp DESC
     LIMIT 100`,
  );
  let enriched = rows;
  try {
    const pairs = rows
      .map((r) => ({ sensorId: r.sensorId, ts: r.ts }))
      .filter((p) => p.sensorId && p.ts);
    const measurements = await fetchMeasurementsByPairs(pool, pairs);
    const measurementMap = new Map();
    measurements.forEach((m) => {
      const key = measurementKey(m.sensor_id, m.ts);
      if (!key) return;
      const list = measurementMap.get(key) || [];
      list.push(m);
      measurementMap.set(key, list);
    });
    const codeParamIds = rows
      .map((r) => parameterIdFromErrorCode(r.code))
      .filter(Boolean);
    const paramIds = [...new Set([...measurements.map((m) =>
m.id_parameter).filter(Boolean), ...codeParamIds])];
    const refMap = new Map();
    if (paramIds.length) {
      const refCols = await getColumns(pool, 'reference_values');
      const idCol = refCols.includes('id_parameter')
        ? 'id_parameter'
        : refCols.includes('id_parameters')
          ? 'id_parameters'
          : null;
      if (idCol) {
        const placeholders = paramIds.map(() => '?').join(',');
```

Продолжение Листинга А.1

```
const [refs] = await pool.query(
          `SELECT rv.${idCol} AS id_parameter,
                  rv.min_value,
                  rv.max_value,
                  u.unit_name AS unit
           FROM reference_values rv
           LEFT JOIN units u ON u.id_unit = rv.id_unit_values
           WHERE rv.${idCol} IN (${placeholders})`,
          paramIds,
        );
        refs.forEach((r) => refMap.set(Number(r.id_parameter), r));
      }
    }
    enriched = rows.map((r) => {
      let severity = r.severity || 'warning';
      let value = null;
      let parameterId = null;
      let refMin = null;
      let refMax = null;
      let unit = null;
      const knownAlert = knownAlertByCode(r.code);
      const expectedParameterId = knownAlert?.parameterId ??
parameterIdFromErrorCode(r.code);
      if (knownAlert) {
        severity = knownAlert.severity;
        parameterId = knownAlert.parameterId;
        const ref = refMap.get(Number(parameterId));
        refMin = ref?.min_value ?? null;
        refMax = ref?.max_value ?? null;
        unit = ref?.unit ?? null;
      }
      const key = measurementKey(r.sensorId, r.ts);
      const measurementList = key ? measurementMap.get(key) || [] : [];
      const measurement =
        measurementList.find((m) => expectedParameterId &&
Number(m.id_parameter) === Number(expectedParameterId)) ||
        measurementList.find((m) => {
          const ref = refMap.get(Number(m.id_parameter));
          return !!getDeviationSeverity(m.value, ref?.min_value,
ref?.max_value);
        }) ||
        measurementList[0] ||
        null;
      if (measurement) {
        value = measurement.value;
        parameterId = measurement.id_parameter;
        const ref = refMap.get(Number(measurement.id_parameter));
        refMin = ref?.min_value ?? null;
        refMax = ref?.max_value ?? null;
        unit = ref?.unit ?? null;
        if (!knownAlert) {
          const computed = getDeviationSeverity(measurement.value,
ref?.min_value, ref?.max_value);
          if (computed) severity = computed;
        }
      }
      return { ...r, severity, value, parameterId, refMin, refMax, unit };
    });
  } catch (e) {
    console.error('errors severity calc error', e);
  }
  res.json(enriched.map((r) => ({ ...r, zoneId: null })));
```

Продолжение Листинга А.1

```
});
...

// Главная панель с временем последнего обновления
app.get('/api/dashboard2', async (_req, res) => {
  const pool = await poolPromise;
  const snapshot = await buildDashboardSnapshot(pool);
  res.json(snapshot);
});

// Последние измерения с информацией о параметрах
app.get('/api/measurements', async (_req, res, next) => {
  try {
    const pool = await poolPromise;
    const cols = await getColumns(pool, 'reference_values');
    const idRefCol = cols.includes('id_parameter') ? 'id_parameter' :
cols.includes('id_parameters') ? 'id_parameters' : null;
    const displayLimit = 100;
    const baseLimit = tempScenario.active ? 300 : displayLimit;
    // Сборка запроса последних измерений для общей таблицы параметров.
    const buildQuery = (table) => {
      const indexHint = table === 'sensors_measurements' ? 'FORCE INDEX
(idx_measurements_timestamp)' : '';
      return `SELECT m.id,
              m.ts,
              m.value,
              m.id_parameter,
              m.sensor_id,
              p.name_parameter AS parameter,
              u.unit_name AS unit${idRefCol ? `,
              (SELECT rv.min_value
                 FROM reference_values rv
                 WHERE rv.${idRefCol} = m.id_parameter
                 LIMIT 1) AS ref_min,
              (SELECT rv.max_value
                 FROM reference_values rv
                 WHERE rv.${idRefCol} = m.id_parameter
                 LIMIT 1) AS ref_max` : ''}
       FROM (
         SELECT id_measurements AS id,
                timestamp AS ts,
                value,
                id_parameter,
                id_sensors AS sensor_id
         FROM ${table} ${indexHint}
         ORDER BY timestamp DESC
         LIMIT ${baseLimit}
       ) m
       LEFT JOIN parameters p ON p.id_parameter = m.id_parameter
       LEFT JOIN sensors s ON s.id_sensor = m.sensor_id
       LEFT JOIN units u ON u.id_unit = s.id_units
       ORDER BY m.ts DESC`;
    };
    let rows = [];
    try {
      [rows] = await pool.query(buildQuery('sensors_measurements'));
    } catch (e) {
      if (e?.code === 'ER_NO_SUCH_TABLE') {
        const [altRows] = await pool.query(buildQuery('sensor_measurements'));
        rows = altRows;
      } else {
        throw e;
```

Продолжение Листинга А.1

```
      }
    }
    ...

    if (rows?.length) {
      for (const row of rows) {
        const min = row.ref_min;
        const max = row.ref_max;
        const val = Number(row.value);
        const outLow = min != null && !Number.isNaN(val) && val < Number(min);
        const outHigh = max != null && !Number.isNaN(val) && val > Number(max);
        if (!outLow && !outHigh) continue;
        if (!row.sensor_id) continue;
        const code = `OUT-${row.id_parameter}`;
        const name = `Вне диапазона: ${row.parameter ?? row.id_parameter}`;
        const duplicateCodes = [code];
        if (Number(row.id_parameter) === 2) duplicateCodes.push('Warning_Hum',
'Alarm_Hum');
        if (Number(row.id_parameter) === 3) duplicateCodes.push('Warning_CO2',
'Alarm_CO2');
        try {
          const codePlaceholders = duplicateCodes.map(() => '?').join(',');
          const [exists] = await pool.query(
            `SELECT id_sensor_error
             FROM sensor_error
             WHERE id_sensor_number = ? AND timestamp = ? AND error_code IN
(${codePlaceholders})
             LIMIT 1`,
            [row.sensor_id, row.ts, ...duplicateCodes],
          );
          if (!exists.length) {
            await pool.query(
              `INSERT INTO sensor_error (error_code, error_name, timestamp,
id_sensor_number)
               VALUES (?, ?, ?, ?)`,
              [code, name, row.ts, row.sensor_id],
            );
          }
        } catch (e) {
          console.error('sensor_error insert error', e);
        }
      }
    }
    res.json(rows);
  } catch (err) {
    next(err);
  }
});
...

// Авторизация сотрудника по имени, фамилии и паролю
app.post('/api/auth/login', async (req, res, next) => {
  try {
    const pool = await poolPromise;
    const { firstName, lastName, password } = req.body || {};
    const first = normalizeSpace(firstName);
    const last = normalizeSpace(lastName);
    const pass = String(password || '');
    if (!first || !last || !pass) {
      return res.status(400).json({ error: 'Введите имя, фамилию и пароль' });
    }
```

Продолжение Листинга А.1

```
    const cols = await getColumns(pool, 'employee');
    if (!cols.length) return res.status(500).json({ error: 'employee: таблица не
найдена' });
    const colMap = buildColumnMap(cols);

    const idCol = pickColumn(colMap, ['id_employee', 'employee_id', 'id',
'id_user', 'user_id']);
    const passCol = pickColumn(colMap, [
      'password',
      'pass',
      'pwd',
      'passwd',
      'employee_password',
      'user_password',
      'password_hash',
      'pass_hash',
      'hash',
    ]);
    if (!passCol) return res.status(500).json({ error: 'employee: нет колонки
пароля' });

    const firstCol = pickColumn(colMap, [
      'first_name',
      'firstname',
      'name_first',
      'employee_first_name',
      'fname',
      'given_name',
    ]);
    const lastCol = pickColumn(colMap, [
      'last_name',
      'lastname',
      'surname',
      'family_name',
      'second_name',
      'employee_last_name',
      'lname',
    ]);
    const fullCols = pickColumns(colMap, [
      'employee_name',
      'name_employee',
      'full_name',
      'fio',
      'name',
      'fullname',
      'employee_fullname',
    ]);

    const conditions = [];
    const params = [];

    if (firstCol && lastCol) {
      conditions.push(`(LOWER(${firstCol}) = LOWER(?) AND LOWER(${lastCol}) =
LOWER(?))`);
      params.push(first, last);
    }

    if (fullCols.length) {
      const likeParts = fullCols.map(
        (c) => `(LOWER(${c}) LIKE LOWER(?) AND LOWER(${c}) LIKE LOWER(?))`,
      );
      conditions.push(`(${likeParts.join(' OR ')})`);
```

Продолжение Листинга А.1

```
      fullCols.forEach(() => params.push(`%${first}%`, `%${last}%`));
    }

    if (!conditions.length) {
      return res.status(500).json({ error: 'employee: нет колонок для
имени/фамилии' });
    }

    const sql = `SELECT * FROM employee WHERE (${conditions.join(' OR ')}) LIMIT
20`;
    const [rows] = await pool.query(sql, params);
    if (!rows.length) return res.status(401).json({ error: 'Неверные данные' });

    let matched = null;
    for (const row of rows) {
      const stored = row?.[passCol];
      if (!stored) continue;
      if (verifyPassword(pass, stored)) {
        matched = row;
        break;
      }
    }
    if (!matched) return res.status(401).json({ error: 'Неверные данные' });

    const raw = matched || {};
    const id = raw[idCol] ?? raw.id_employee ?? raw.employee_id ?? raw.id ??
null;
    let name = null;
    for (const col of fullCols) {
      if (raw[col]) {
        name = raw[col];
        break;
      }
    }
    if (!name) {
      const firstVal = firstCol ? raw[firstCol] : null;
      const lastVal = lastCol ? raw[lastCol] : null;
      if (firstVal || lastVal) {
        name = `${firstVal ?? ''} ${lastVal ?? ''}`.trim();
      }
    }
    if (!name) {
      name =
        raw.employee_name ??
        raw.name_employee ??
        raw.full_name ??
        raw.fio ??
        raw.username ??
        raw.name ??
        null;
    }
    res.json({ id, name: name || `${first} ${last}`.trim() });
  } catch (err) {
    next(err);
  }
});

...

app.use((err, _req, res, _next) => {
  console.error(err);
  res.status(500).json({ error: 'Internal error' });
```

Продолжение Листинга А.1

```
});

const server = http.createServer(app);
const wss = new WebSocketServer({ server, path: '/ws' });

// Отправка сообщения всем подключенным WebSocket-клиентам
const broadcast = (payload) => {
  if (!wss.clients.size) return;
  const data = JSON.stringify(payload);
  for (const client of wss.clients) {
    if (client.readyState === 1) {
      client.send(data);
    }
  }
};

let dashboardTimer = null;
let tempTimer = null;
let tempScenarioBusy = false;
let tempNormalTimer = null;

// Запуск периодической отправки данных главной панели
const startDashboardBroadcast = () => {
  if (dashboardTimer) return;
  dashboardTimer = setInterval(async () => {
    if (!wss.clients.size) return;
    try {
      const pool = await poolPromise;
      const snapshot = await buildDashboardSnapshot(pool);
      broadcast({ type: 'dashboard', data: snapshot });
    } catch (e) {
      console.error('ws dashboard error', e);
    }
  }, 3000);
};

wss.on('connection', (ws) => {
  ws.send(JSON.stringify({ type: 'welcome', ts: Date.now() }));
  ws.on('message', async (message) => {
    let payload = null;
    try {
      payload = JSON.parse(message.toString());
    } catch {
      return;
    }
    if (payload?.type === 'ping') {
      ws.send(JSON.stringify({ type: 'pong', ts: Date.now() }));
      return;
    }
    if (payload?.type === 'dashboard:once') {
      try {
        const pool = await poolPromise;
        const snapshot = await buildDashboardSnapshot(pool);
        ws.send(JSON.stringify({ type: 'dashboard', data: snapshot }));
      } catch (e) {
        console.error('ws dashboard once error', e);
      }
    }
  });
});
startDashboardBroadcast();
```

Продолжение Листинга А.1

```
// Чтение данных из базы и запуск процедур полива на сервере
server.listen(PORT, async () => {
  const pool = await poolPromise;
  await backfillNdviHistory(pool, 24).catch((e) => console.error('ndvi backfill
error', e));
  await initTempScenario(pool).catch((e) => console.error('temp scenario init
error', e));
  tempScenario.active = true;
  resetTempScenario();
  startTempScenario();
  console.log(`Backend listening on http://localhost:${PORT}`);
});
```

Приложение Б

Листинг Б.1 — Содержимое файла frontend/src/pages/Login.jsx

```
import { useEffect, useState } from 'react';
import { Navigate, useLocation, useNavigate } from 'react-router-dom';
import { login as apiLogin } from '../api';
import { useAuth } from '../auth';
import leafLogo from '../assets/листочек.png';

// Отображение страницы авторизации сотрудника
export function Login() {
  const { employee, login } = useAuth();
  const location = useLocation();
  const navigate = useNavigate();

  const [firstName, setFirstName] = useState('');
  const [lastName, setLastName] = useState('');
  const [password, setPassword] = useState('');
  const [showPassword, setShowPassword] = useState(false);
  const [error, setError] = useState('');
  const [loading, setLoading] = useState(false);
  const [formKey, setFormKey] = useState(0);

  const from = location.state?.from || '/';

// Очистка полей формы после выхода пользователя
  const resetForm = () => {
    setFirstName('');
    setLastName('');
    setPassword('');
    setShowPassword(false);
    setError('');
    setFormKey((k) => k + 1);
  };

  useEffect(() => {
    if (location.state?.clear) {
      resetForm();
    }
  }, [location.state?.clear]);

  if (employee) {
    return <Navigate to="/" replace />;
  }

// Проверка формы и выполнение входа сотрудника
  const handleSubmit = async (event) => {
    event.preventDefault();
    const first = firstName.trim();
    const last = lastName.trim();
    if (!first || !last || !password) {
      setError('Заполните имя, фамилию и пароль');
      return;
    }
    setLoading(true);
    setError('');
    try {
      const data = await apiLogin({ firstName: first, lastName: last,
password });
      login(data);
      navigate(from, { replace: true });
```

Продолжение Листинга Б.1

```
    } catch {
      setError('Неверное имя, фамилия или пароль');
    } finally {
      setLoading(false);
    }
  };

  return (
    <div className="auth-shell">
      <div className="auth-card">
        <div className="auth-brand">
          <div className="brand-mark">
            <img src={leafLogo} alt="Листочек" />
          </div>
          <div>
            <div className="auth-title">Agro Control</div>
            <div className="muted">Вход для сотрудников</div>
          </div>
        </div>
        <form key={formKey} className="auth-form" onSubmit={handleSubmit}>
          <label className="input-group">
            <div className="label">Имя</div>
            <input
              className="input"
              value={firstName}
              onChange={(e) => setFirstName(e.target.value)}
              autoComplete="given-name"
              placeholder="Введите имя"
            />
          </label>
          <label className="input-group">
            <div className="label">Фамилия</div>
            <input
              className="input"
              value={lastName}
              onChange={(e) => setLastName(e.target.value)}
              autoComplete="family-name"
              placeholder="Введите фамилию"
            />
          </label>
          <label className="input-group">
            <div className="label">Пароль</div>
            <div className="password-field">
              <input
                className="input input-with-icon"
                type={showPassword ? 'text' : 'password'}
                value={password}
                onChange={(e) => setPassword(e.target.value)}
                autoComplete="current-password"
                placeholder="Введите пароль"
              />
              <button
                className={`password-toggle ${showPassword ? 'active' : ''}`}
                type="button"
                aria-label={showPassword ? 'Скрыть пароль' : 'Показать пароль'}
                title={showPassword ? 'Скрыть пароль' : 'Показать пароль'}
                onClick={() => setShowPassword((prev) => !prev)}
              >
                {showPassword ? (
                  <svg viewBox="0 0 24 24" aria-hidden="true">
                    <path
                      d="M3 3l18 18M10.58 10.58A2 2 0 0 0 12 14a2 2 0 0 0
```

Продолжение Листинга Б.1

```
1.42-.58M9.88 4.24A10.64 10.64 0 0 1 12 4c6 0 10 8 10 8a18.9 18.9 0 0 1-4.32
5.36M6.23 6.23A18.7 18.7 0 0 0 2 12s4 8 10 8a10.7 10.7 0 0 0 4.06-.8"
                      fill="none"
                      stroke="currentColor"
                      strokeWidth="2"
                      strokeLinecap="round"
                      strokeLinejoin="round"
                    />
                  </svg>
                ) : (
                  <svg viewBox="0 0 24 24" aria-hidden="true">
                    <path
                      d="M1.5 12s4-7.5 10.5-7.5S22.5 12 22.5 12 18.5 19.5 12
19.5 1.5 12 1.5 12Z"
                      fill="none"
                      stroke="currentColor"
                      strokeWidth="2"
                    />
                    <circle cx="12" cy="12" r="3.5" fill="none"
stroke="currentColor" strokeWidth="2" />
                  </svg>
                )}
              </button>
            </div>
          </label>
          {error && <div className="pill danger">{error}</div>}
          <button className="btn primary auth-submit" type="submit"
disabled={loading}>
            {loading ? 'Проверяем...' : 'Войти'}
          </button>
        </form>
      </div>
    </div>
  );
}
```

Приложение В

Листинг В.1 — Содержимое файла frontend/src/pages/Dashboard.jsx

```
import { useCallback, useEffect, useMemo, useState } from 'react';
import { useLocation } from 'react-router-dom';
import {
  getDashboard,
  getZones,
  getZoneDetails,
  getParameterReport,
  getParameters,
  getParameterReference,
  updateReference,
  setTemperatureHeating,
  getTemperatureScenarioStatus,
} from '../api';
import { Topbar } from '../layout/Topbar';
import { formatTs } from '../utils/format';
import { useAutoRefresh } from '../hooks/useAutoRefresh';
import { useAuth } from '../auth';
import { NDVI_AERO_PARAM_ID, NDVI_COMPARE_KEY, NDVI_COMPARE_LABEL,
NDVI_HYDRO_PARAM_ID } from './dashboard/constants';
import {
  buildDenseLabelMap,
  buildReportChart,
  formatReportLabel as formatReportLabelValue,
  getReportFlags,
} from './dashboard/reportChartUtils';
import {
  getDeviationStatus,
  getWsUrl,
  normalizeDashboardPayload,
} from './dashboard/dashboardUtils';
import { buildTimeline } from './dashboard/timelineUtils';
import { isAeroZone, isHydroZone } from './dashboard/zoneUtils';
import { MetricCards } from './dashboard/components/MetricCards';
import { MicroclimatePanel } from './dashboard/components/MicroclimatePanel';
import { ReferenceEditor } from './dashboard/components/ReferenceEditor';
import { ReportPanel } from './dashboard/components/ReportPanel';
import { TemperatureNotice } from './dashboard/components/TemperatureNotice';

//Формирование экрана мониторинга из данных API, графиков и форм управления
export function Dashboard() {
  const location = useLocation();
  const { employee } = useAuth();
  const [dash, setDash] = useState(null);
  const [wsOnline, setWsOnline] = useState(false);
  const [zones, setZones] = useState([]);
  const [detail, setDetail] = useState(null);
  const [primaryZoneId, setPrimaryZoneId] = useState(null);
  const [parameters, setParameters] = useState([]);
  const [reportParam, setReportParam] = useState('');
  const [report, setReport] = useState(null);
  const [reportLoading, setReportLoading] = useState(false);
  const [reportError, setReportError] = useState('');
  const [rangeRefs, setRangeRefs] = useState({});
  const [tempNotice, setTempNotice] = useState(null);
  const [tempCriticalAck, setTempCriticalAck] = useState(false);
  const [tempNormalAck, setTempNormalAck] = useState(false);
  const [tempScenarioActive, setTempScenarioActive] = useState(true);

  const [editParam, setEditParam] = useState('');
```

Продолжение Листинга В.1

```
  const [editMin, setEditMin] = useState('');
  const [editMax, setEditMax] = useState('');
  const [loadedRef, setLoadedRef] = useState({ min: '', max: '' });
  const [editError, setEditError] = useState('');
  const [editSuccess, setEditSuccess] = useState('');
  const [saving, setSaving] = useState(false);

  // Применение обновления дашборда из HTTP или WebSocket
  const applyDashboard = useCallback((payload) => {
    const normalized = normalizeDashboardPayload(payload);
    if (normalized) setDash(normalized);
  }, []);

  //Загрузка референсных диапазонов для основных метрик
  const loadMainRefs = useCallback(async () => {
    try {
      const [tempRef, humRef, co2Ref] = await Promise.all([
        getParameterReference(1),
        getParameterReference(2),
        getParameterReference(3),
      ]);
      setRangeRefs({
        temperature: tempRef,
        humidity: humRef,
        co2: co2Ref,
      });
    } catch (e) {
      console.error('reference load error', e);
    }
  }, []);

  //Загрузка основных данных для первого отображения страницы
  const load = useCallback(async () => {
    const [dashboardData, zoneList, parameterList] = await Promise.all([
      getDashboard(),
      getZones(),
      getParameters(),
    ]);
    applyDashboard(dashboardData);
    setZones(Array.isArray(zoneList) ? zoneList : []);
    setParameters(Array.isArray(parameterList) ? parameterList : []);

    const firstParam = parameterList?.[0]?.id?.toString();
    if (firstParam) {
      setReportParam((prev) => prev || firstParam);
      setEditParam((prev) => prev || firstParam);
    }

    const primaryZone = zoneList?.[0]?.id ?? null;
    setPrimaryZoneId(primaryZone);
    if (primaryZone) {
      setDetail(await getZoneDetails(primaryZone, { hours: 6 }));
    }
  }, [applyDashboard]);

  //Обновление детализации выбранной зоны
  const loadZoneDetail = useCallback(async (zoneId) => {
    if (!zoneId) return;
    try {
      setDetail(await getZoneDetails(zoneId, { hours: 6 }));
    } catch (e) {
      console.error('zone detail load error', e);
```

Продолжение Листинга В.1

```
    }
  }, []);

  useEffect(() => {
    load();
    loadMainRefs();
  }, [load, loadMainRefs]);

  useEffect(() => {
    let mounted = true;
    getTemperatureScenarioStatus()
      .then((data) => {
        if (mounted && typeof data?.active === 'boolean') {
          setTempScenarioActive(data.active);
        }
      })
      .catch((e) => console.error(e));
    return () => {
      mounted = false;
    };
  }, []);

  useEffect(() => {
    const wsUrl = getWsUrl();
    if (!wsUrl) return undefined;

    let ws;
    let reconnectTimer;
    let active = true;

    //Открытие WebSocket и переподключение при разрыве
    const connect = () => {
      ws = new WebSocket(wsUrl);
      ws.onopen = () => {
        setWsOnline(true);
        ws.send(JSON.stringify({ type: 'dashboard:once' }));
      };
      ws.onmessage = (event) => {
        try {
          const msg = JSON.parse(event.data);
          if (msg?.type === 'dashboard' && msg.data) {
            applyDashboard(msg.data);
          }
        } catch (e) {
          console.error('ws message error', e);
        }
      };
      ws.onclose = () => {
        setWsOnline(false);
        if (active) {
          reconnectTimer = setTimeout(connect, 3000);
        }
      };
      ws.onerror = () => {
        ws.close();
      };
    };

    connect();
    return () => {
      active = false;
      if (reconnectTimer) clearTimeout(reconnectTimer);
```

Продолжение Листинга В.1

```
      if (ws) ws.close();
    };
  }, [applyDashboard]);

  useAutoRefresh(() => {
    if (!wsOnline) {
      return load();
    }
    return null;
  }, 3000);

  useAutoRefresh(() => {
    if (primaryZoneId) {
      return loadZoneDetail(primaryZoneId);
    }
    return null;
  }, 3000);

  const chartData = useMemo(() => buildTimeline(detail), [detail]);
  const reportFlags = useMemo(() => getReportFlags({ report, reportParam }),
[report, reportParam]);

  const hydroZones = useMemo(() => zones.filter(isHydroZone), [zones]);
  const aeroZones = useMemo(() => zones.filter(isAeroZone), [zones]);
  const hydroZoneIds = useMemo(() => new Set(hydroZones.map((z) =>
String(z.id))), [hydroZones]);
  const aeroZoneIds = useMemo(() => new Set(aeroZones.map((z) => String(z.id))),
[aeroZones]);
  const hydroZoneLabels = useMemo(
    () => new Map(hydroZones.map((z) => [`zone_${z.id}`, z.name || `Зона
${z.id}`])),
    [hydroZones],
  );
  const aeroZoneLabels = useMemo(
    () => new Map(aeroZones.map((z) => [`zone_${z.id}`, z.name || `Зона
${z.id}`])),
    [aeroZones],
  );

  const reportChart = useMemo(
    () =>
      buildReportChart({
        report,
        flags: reportFlags,
        hydroZoneIds,
        aeroZoneIds,
        hydroZoneLabels,
        aeroZoneLabels,
      }),
    [report, reportFlags, hydroZoneIds, aeroZoneIds, hydroZoneLabels,
aeroZoneLabels],
  );

  const labelMap = useMemo(
    () => buildDenseLabelMap(reportChart, reportFlags.reportIsDense),
    [reportChart, reportFlags.reportIsDense],
  );
  const reportAeroBaseTs = useMemo(() => {
    if (!reportFlags.reportIsAeroWindow) return null;
    return reportChart.rows[0]?.ts ?? null;
  }, [reportChart, reportFlags.reportIsAeroWindow]);
  const formatReportLabel = useCallback(
```

Продолжение Листинга В.1

```
    (value) =>
      formatReportLabelValue({
        value,
        reportIsDense: reportFlags.reportIsDense,
        labelMap,
        reportIsAeroWindow: reportFlags.reportIsAeroWindow,
        reportAeroBaseTs,
      }),
    [labelMap, reportAeroBaseTs, reportFlags.reportIsAeroWindow,
reportFlags.reportIsDense],
  );

  //Загрузка отчета по выбранному параметру
  const loadReport = useCallback(async (paramKey) => {
    if (!paramKey) return;
    try {
      setReportLoading(true);
      setReportError('');
      if (paramKey === NDVI_COMPARE_KEY) {
        const [hydro, aero] = await Promise.all([
          getParameterReport(NDVI_HYDRO_PARAM_ID),
          getParameterReport(NDVI_AERO_PARAM_ID),
        ]);
        setReport({
          parameter: NDVI_COMPARE_KEY,
          name: NDVI_COMPARE_LABEL,
          ndviCompare: {
            hydro: hydro?.series ?? [],
            aero: aero?.series ?? [],
          },
        });
        return;
      }
      setReport(await getParameterReport(paramKey));
    } catch (e) {
      console.error(e);
      setReportError('Не удалось загрузить данные отчета');
    } finally {
      setReportLoading(false);
    }
  }, []);

  useEffect(() => {
    if (reportParam) {
      loadReport(reportParam);
    }
  }, [loadReport, reportParam]);

  //Загрузка текущих пределов редактируемого параметра
  const loadReference = useCallback(async (paramId) => {
    if (!paramId) return;
    try {
      const ref = await getParameterReference(paramId);
      const minVal = ref?.ref_min ?? '';
      const maxVal = ref?.ref_max ?? '';
      setLoadedRef({ min: minVal, max: maxVal });
      setEditMin('');
      setEditMax('');
      setEditError('');
      setEditSuccess('');
    } catch (e) {
      console.error(e);
```

Продолжение Листинга В.1

```
      setLoadedRef({ min: '', max: '' });
      setEditError('Не удалось загрузить текущие пределы');
    }
  }, []);

  useEffect(() => {
    if (editParam) {
      loadReference(editParam);
    }
  }, [editParam, loadReference]);

  useEffect(() => {
    if (location.hash !== '#reference-editor') return;
    requestAnimationFrame(() => {
      document.getElementById('reference-editor')?.scrollIntoView({ behavior:
'smooth', block: 'start' });
    });
  }, [location.hash]);

  const values = {
    temperature: dash?.aggregates?.temperature ?? detail?.current?.temperature,
    humidity: dash?.aggregates?.humidity ?? detail?.current?.humidity,
    co2: dash?.aggregates?.co2 ?? detail?.current?.co2,
  };
  const statuses = {
    temperature: getDeviationStatus(values.temperature, rangeRefs.temperature),
    humidity: getDeviationStatus(values.humidity, rangeRefs.humidity),
    co2: getDeviationStatus(values.co2, rangeRefs.co2),
  };
  const tempNormalMin = useMemo(() => {
    const raw =
      rangeRefs?.temperature?.ref_min ??
      rangeRefs?.temperature?.min_value ??
      rangeRefs?.temperature?.min ??
      null;
    const val = Number(raw);
    return Number.isFinite(val) && val > 10 ? val : 20;
  }, [rangeRefs]);

  useEffect(() => {
    if (!tempScenarioActive) return;
    const value = Number(values.temperature);
    if (!Number.isFinite(value)) return;
    if (!tempCriticalAck && value <= 10) {
      setTempNotice('critical');
      return;
    }
    if (tempCriticalAck && !tempNormalAck && value >= tempNormalMin) {
      setTempNotice('normal');
    }
  }, [values.temperature, tempCriticalAck, tempNormalAck, tempNormalMin,
tempScenarioActive]);

  // Подтверждение критического падения температуры и включение отопления
  const handleTempCriticalAck = useCallback(async () => {
    try {
      await setTemperatureHeating({ on: true });
    } catch (e) {
      console.error(e);
    }
    setTempCriticalAck(true);
    setTempNotice(null);
```

Продолжение Листинга В.1

```
  }, []);
  // Закрытие уведомления о возврате температуры в норму
  const handleTempNormalAck = useCallback(() => {
    setTempNormalAck(true);
    setTempNotice(null);
  }, []);

  // Проверка и сохранение нового референсного диапазона
  const handleReferenceSubmit = useCallback(async () => {
    const minRaw = editMin === '' ? loadedRef.min : editMin;
    const maxRaw = editMax === '' ? loadedRef.max : editMax;
    const min = Number(minRaw);
    const max = Number(maxRaw);
    if (minRaw === '' || maxRaw === '') {
      setEditError('Заполните оба значения');
      setEditSuccess('');
      return;
    }
    if (Number.isNaN(min) || Number.isNaN(max)) {
      setEditError('Минимум и максимум должны быть числами');
      setEditSuccess('');
      return;
    }
    if (min > max) {
      setEditError('Минимум не может быть больше максимума');
      setEditSuccess('');
      return;
    }

    setEditError('');
    setSaving(true);
    try {
      await updateReference({ parameterId: editParam, min, max, employeeId:
employee?.id });
      setEditSuccess('Диапазон обновлен');
      setLoadedRef({ min, max });
      setEditMin('');
      setEditMax('');
      loadMainRefs();
    } catch (e) {
      console.error(e);
      setEditError('Не удалось сохранить изменения');
      setEditSuccess('');
    } finally {
      setSaving(false);
    }
  }, [editMax, editMin, editParam, employee?.id, loadedRef, loadMainRefs]);

  // Сброс формы редактирования к последним загруженным значениям
  const handleReferenceCancel = useCallback(() => {
    setEditMin(loadedRef.min ?? '');
    setEditMax(loadedRef.max ?? '');
    setEditError('');
    setEditSuccess('');
  }, [loadedRef]);

  return (
    <div>
      <TemperatureNotice
        notice={tempNotice}
        temperatureValue={values.temperature}
        onCriticalAck={handleTempCriticalAck}
```

Продолжение Листинга В.1

```
        onNormalAck={handleTempNormalAck}
      />
      <Topbar
        title="Главная"
        actions={
          <>
            {employee && (
              <div className="pill">
                Сотрудник: {employee.name}
              </div>
            )}
            <div className="pill success">Обновлено: {dash ?
formatTs(dash.updatedAt || Date.now()) : '-'}</div>
          </>
        }
      />

      <MetricCards values={values} statuses={statuses} />
      <MicroclimatePanel chartData={chartData} />
      <ReportPanel
        flags={reportFlags}
        formatReportLabel={formatReportLabel}
        labelMap={labelMap}
        onReportParamChange={setReportParam}
        parameters={parameters}
        report={report}
        reportChart={reportChart}
        reportError={reportError}
        reportLoading={reportLoading}
        reportParam={reportParam}
      />
      <ReferenceEditor
        editError={editError}
        editMax={editMax}
        editMin={editMin}
        editParam={editParam}
        editSuccess={editSuccess}
        employee={employee}
        loadedRef={loadedRef}
        onCancel={handleReferenceCancel}
        onEditMaxChange={setEditMax}
        onEditMinChange={setEditMin}
        onEditParamChange={setEditParam}
        onSubmit={handleReferenceSubmit}
        parameters={parameters}
        saving={saving}
      />
    </div>
  );
}
```

Приложение Г

Листинг Г.1 — Содержимое файла frontend/src/pages/Zones.jsx

```
import { useCallback, useEffect, useMemo, useRef, useState } from 'react';
import { useSearchParams } from 'react-router-dom';
import { forceWateringBatch, getZones } from '../api';
import { useAuth } from '../auth';
import { Topbar } from '../layout/Topbar';
import { useAutoRefresh } from '../hooks/useAutoRefresh';
import { formatNumber } from '../utils/format';
import {
  compareZonesByGrid,
  getWateringDurationSec,
  getZoneColumn,
  getZoneDisplayName,
  isAeroZone,
  isHydroZone,
} from '../utils/zones';

const STATUS_TIMEOUT_MS = 4000;

// Получение названия технологии зоны
const getTechnologyName = (zone) => zone?.technologyName || zone?.technology ||
'';

// Получение текстового статуса полива зоны
const getWateringStatus = (zone) => {
  const isRunning = zone?.isWatering ?? zone?.status === 'watering';
  return isRunning ? 'идет' : 'не идет';
};

// Отображение группы зон одной технологии
function ZoneGroup({ title, zones, maxColumns }) {
  if (!zones.length) return null;

  return (
    <div style={{ marginTop: 12 }}>
      <h3 style={{ margin: '12px 0 8px' }}>{title}</h3>
      <div className="grid zone-grid" style={{ '--zone-cols': maxColumns > 0 ?
maxColumns : 1 }}>
        {zones.map((zone) => {
          const isRunning = zone?.isWatering ?? zone?.status === 'watering';
          const statusClass = isRunning ? 'watering' : 'idle';
          const ndviValue = zone?.ndvi ?? zone?.ndvi_value ?? zone?.ndviValue;

          return (
            <div key={zone.id} className="panel zone-card">
              <div className="card-row" style={{ marginBottom: 10 }}>
                <div>
                  <h3 style={{ margin: 0 }}>{getZoneDisplayName(zone)}</h3>
                  <div className="subtitle">{getTechnologyName(zone) || '—
'}</div>
                </div>
              </div>

              <div className="row-grid zone-metrics" style={{ marginBottom:
10 }}>
                <div className="row-tile metric">
                  <div className="row-title">NDVI</div>
                  <div className="row-meta">{formatNumber(ndviValue, 2)}</div>
                </div>
              </div>
```

Продолжение Листинга Г.1

```
              <div className={`row-tile zone-watering ${statusClass}`}
title={`Полив: ${getWateringStatus(zone)}`}>
                <div className="row-title">Полив:
{getWateringStatus(zone)}</div>
              </div>
            </div>
          );
        })}
      </div>
    </div>
  );
}

// Отображение страницы зон полива
export function Zones() {
  const { employee } = useAuth();
  const [searchParams] = useSearchParams();
  const [zones, setZones] = useState([]);
  const [search, setSearch] = useState('');
  const [tech, setTech] = useState('');
  const [forceTech, setForceTech] = useState('');
  const [forceZoneIds, setForceZoneIds] = useState([]);
  const [forceDropdownOpen, setForceDropdownOpen] = useState(false);
  const [forceStatus, setForceStatus] = useState({ type: '', text: '' });
  const [forceLoading, setForceLoading] = useState(false);
  const statusTimerRef = useRef(null);
  const forceDropdownRef = useRef(null);
  const appliedForceZoneRef = useRef(false);

  // Загрузка списка зон
  const load = useCallback(async () => {
    const data = await getZones();
    setZones(Array.isArray(data) ? data : []);
  }, []);

  useEffect(() => {
    load();
  }, [load]);

  useAutoRefresh(load, 4000);

  useEffect(() => {
    if (appliedForceZoneRef.current || !zones.length) return;

    const forceZoneId = searchParams.get('forceZoneId');
    if (!forceZoneId) return;

    const zone = zones.find((item) => String(item.id) === String(forceZoneId));
    if (!zone) return;

    appliedForceZoneRef.current = true;
    setForceZoneIds([String(zone.id)]);
    setForceTech(getTechnologyName(zone));
    setForceStatus({
      type: 'warning',
      text: `Требуется запуск принудительного полива.
${getTechnologyName(zone)}. ${getZoneDisplayName(zone)}.`,
    });

    if (window.location.hash === '#watering-control') {
      requestAnimationFrame(() => {
        document.getElementById('watering-control')?.scrollIntoView({ block:
```

Продолжение Листинга Г.1

```
 'center' });
      });
    }
  }, [searchParams, zones]);

  useEffect(() => {
    if (!forceStatus.text || forceStatus.type === 'danger') return undefined;
    if (statusTimerRef.current) clearTimeout(statusTimerRef.current);
    statusTimerRef.current = setTimeout(() => {
      setForceStatus({ type: '', text: '' });
      statusTimerRef.current = null;
    }, STATUS_TIMEOUT_MS);

    return () => {
      if (statusTimerRef.current) {
        clearTimeout(statusTimerRef.current);
        statusTimerRef.current = null;
      }
    };
  }, [forceStatus]);

  useEffect(() => {
    if (!forceDropdownOpen) return undefined;
    // Закрытие выпадающего списка при клике вне блока
    const handleClick = (event) => {
      if (forceDropdownRef.current
&& !forceDropdownRef.current.contains(event.target)) {
        setForceDropdownOpen(false);
      }
    };
    document.addEventListener('mousedown', handleClick);
    return () => document.removeEventListener('mousedown', handleClick);
  }, [forceDropdownOpen]);

  const filteredZones = useMemo(() => {
    const searchValue = search.trim().toLowerCase();
    return zones.filter((zone) => {
      if (tech && getTechnologyName(zone) !== tech) return false;
      if (!searchValue) return true;
      return String(zone.name || '').toLowerCase().includes(searchValue);
    });
  }, [zones, search, tech]);

  const technologies = useMemo(
    () => Array.from(new
Set(zones.map(getTechnologyName).filter(Boolean))).sort(),
    [zones],
  );

  const forceZones = useMemo(() => {
    if (!forceTech) return zones;
    return zones.filter((zone) => getTechnologyName(zone) === forceTech);
  }, [zones, forceTech]);

  useEffect(() => {
    const allowed = new Set(forceZones.map((zone) => String(zone.id)));
    setForceZoneIds((prev) => prev.filter((id) => allowed.has(String(id))));
  }, [forceZones]);

  const maxColumns = useMemo(() => {
    const cols = filteredZones.map(getZoneColumn).filter((value) => value !=
null);
```

Продолжение Листинга Г.1

```
    return cols.length ? Math.max(...cols) : 0;
  }, [filteredZones]);

  const grouped = useMemo(() => {
    const result = { hydro: [], aero: [], other: [] };
    filteredZones.forEach((zone) => {
      if (isHydroZone(zone)) result.hydro.push(zone);
      else if (isAeroZone(zone)) result.aero.push(zone);
      else result.other.push(zone);
    });

    result.hydro.sort(compareZonesByGrid);
    result.aero.sort(compareZonesByGrid);
    result.other.sort(compareZonesByGrid);
    return result;
  }, [filteredZones]);

  const selectedZones = useMemo(() => {
    if (!forceZoneIds.length) return [];
    const byId = new Map(zones.map((zone) => [String(zone.id), zone]));
    return forceZoneIds.map((id) => byId.get(String(id))).filter(Boolean);
  }, [zones, forceZoneIds]);

  const selectedZonesLabel = useMemo(() => {
    if (!selectedZones.length) return 'Выберите зону';
    if (selectedZones.length === 1) return getZoneDisplayName(selectedZones[0]);
    if (selectedZones.length <= 3) return
selectedZones.map(getZoneDisplayName).join(', ');
    return `Выбрано: ${selectedZones.length}`;
  }, [selectedZones]);

  const durationLabel = useMemo(() => {
    if (!selectedZones.length) return null;
    const hasHydro = selectedZones.some(isHydroZone);
    const hasAero = selectedZones.some(isAeroZone);
    const hasOther = selectedZones.some((zone) => !isHydroZone(zone)
&& !isAeroZone(zone));
    const parts = [];
    if (hasHydro) parts.push('гидропоника 10 минут');
    if (hasAero) parts.push('аэропоника 10 секунд');
    if (hasOther) parts.push('другая технология');
    return parts.join(', ');
  }, [selectedZones]);

  // Переключение выбранной зоны для принудительного полива
  const toggleForceZone = useCallback((id) => {
    const key = String(id);
    setForceZoneIds((prev) => (prev.includes(key) ? prev.filter((item) =>
item !== key) : [...prev, key]));
  }, []);

  // Запуск принудительного полива выбранных зон
  const handleForce = async () => {
    if (!forceZoneIds.length || !selectedZones.length) {
      setForceStatus({ type: 'danger', text: 'Выберите зоны' });
      return;
    }

    const invalidZones = selectedZones.filter((zone) =>
getWateringDurationSec(zone) == null);
    if (invalidZones.length) {
      const names = invalidZones.map(getZoneDisplayName).join(', ');
```

Продолжение Листинга Г.1

```
      setForceStatus({ type: 'danger', text: `Неизвестная технология:
${names}` });
      return;
    }

    setForceLoading(true);
    setForceStatus({ type: '', text: '' });
    try {
      const res = await forceWateringBatch(
        selectedZones.map((zone) => zone.id),
        { startedByEmployee: employee?.id ?? null },
      );
      const results = Array.isArray(res?.results) ? res.results : [];
      const failed = results.filter((result) => !result.ok);
      const success = results.filter((result) => result.ok);

      if (success.length && !failed.length) {
        setForceStatus({ type: 'success', text: `Полив запущен:
${selectedZones.map(getZoneDisplayName).join(', ')}` });
      } else if (success.length && failed.length) {
        const failedIds = new Set(failed.map((result) => String(result.id)));
        const failedNames = selectedZones
          .filter((zone) => failedIds.has(String(zone.id)))
          .map(getZoneDisplayName)
          .join(', ');
        setForceStatus({
          type: 'warning',
          text: `Запущено: ${success.length}. Не удалось: ${failedNames ||
failed.length}`,
        });
      } else {
        setForceStatus({ type: 'danger', text: 'Не удалось запустить полив' });
      }
    } catch (e) {
      console.error(e);
      setForceStatus({ type: 'danger', text: 'Не удалось запустить полив' });
    } finally {
      setForceLoading(false);
      try {
        await load();
      } catch (loadError) {
        console.error('zones reload error', loadError);
      }
    }
  };

  return (
    <div>
      <Topbar title="Зоны полива" actions={<div className="pill">Всего зон:
{filteredZones.length}</div>} />

      <div className="filters">
        <input
          className="input"
          placeholder="Поиск по названию"
          value={search}
          onChange={(event) => setSearch(event.target.value)}
        />
        <select className="input" value={tech} onChange={(event) =>
setTech(event.target.value)}>
          <option value="">Все технологии</option>
          {technologies.map((technology) => (
```

Продолжение Листинга Г.1

```
            <option key={technology} value={technology}>
              {technology}
            </option>
          ))}
        </select>
      </div>

      <div id="watering-control" className="filters" style={{ marginTop: 10 }}>
        <select
          className="input"
          value={forceTech}
          onChange={(event) => setForceTech(event.target.value)}
          title="Технология для принудительного полива"
        >
          <option value="">Все технологии</option>
          {technologies.map((technology) => (
            <option key={technology} value={technology}>
              {technology}
            </option>
          ))}
        </select>

        <div className="multi-select" ref={forceDropdownRef}>
          <button
            className={`input multi-select-trigger${forceDropdownOpen ? '
open' : ''}`}
            type="button"
            onClick={() => setForceDropdownOpen((prev) => !prev)}
            title="Выберите зоны для принудительного полива"
          >
            <span className={!selectedZones.length ? 'placeholder' :
undefined}>{selectedZonesLabel}</span>
            <span className="multi-select-caret">▾</span>
          </button>

          {forceDropdownOpen && (
            <div className="multi-select-menu">
              {forceZones.map((zone) => {
                const id = String(zone.id);
                return (
                  <label key={id} className="multi-select-option">
                    <input type="checkbox" checked={forceZoneIds.includes(id)}
onChange={() => toggleForceZone(id)} />
                    <span>{getZoneDisplayName(zone)}</span>
                  </label>
                );
              })}
              {!forceZones.length && <div className="multi-select-empty">Нет
зон</div>}
            </div>
          )}
        </div>

        <button className="btn primary" type="button" disabled={forceLoading
|| !forceZoneIds.length} onClick={handleForce}>
          {forceLoading ? 'Запускаем...' : 'Запустить полив'}
        </button>
      </div>

      {durationLabel && <div className="pill">Длительность:
{durationLabel}</div>}
      {forceStatus.text && <div className={`pill
```

Продолжение Листинга Г.1

```
${forceStatus.type}`}>{forceStatus.text}</div>}

      <ZoneGroup title="Гидропоника" zones={grouped.hydro}
maxColumns={maxColumns} />
      <ZoneGroup title="Аэропоника" zones={grouped.aero} maxColumns={maxColumns}
/>
      <ZoneGroup title="Другое" zones={grouped.other} maxColumns={maxColumns} />
    </div>
  );
}
```

Приложение Д

Листинг Д.1 — Содержимое файла frontend/src/pages/Errors.jsx

```
import { useEffect, useMemo, useState } from 'react';
import { getErrors } from '../api';
import { Topbar } from '../layout/Topbar';
import StatusBadge from '../components/StatusBadge';
import { formatAlertText } from '../utils/alerts';
import { formatTs, statusLabel } from '../utils/format';

// Отображение журнала ошибок и событий
export function Errors() {
  const [errors, setErrors] = useState([]);
  const [query, setQuery] = useState('');

  useEffect(() => {
    // Загрузка журнала ошибок
    const load = async () => {
      try {
        const data = await getErrors();
        setErrors(data || []);
      } catch (e) {
        console.error('errors load error', e);
        setErrors([]);
      }
    };
    load();
  }, []);

  const filteredErrors = useMemo(() => {
    const q = query.trim().toLowerCase();
    if (!q) return errors;
    return errors.filter((e) => {
      const code = (e.code ?? '').toString().toLowerCase();
      const text = formatAlertText(e).toString().toLowerCase();
      const severity = (e.severity ?? '').toString().toLowerCase();
      const label = statusLabel(e.severity).toString().toLowerCase();
       return code.includes(q) || text.includes(q) || severity.includes(q) ||
label.includes(q);
    });
  }, [errors, query]);

  return (
    <div>
      <Topbar
        title="Журнал ошибок и событий"
        actions={<div className="pill">Записей: {filteredErrors.length}</div>}
      />

      <div className="panel">
        <div className="filters">
          <label className="input-group">
            <div className="label">Поиск</div>
            <input
              className="input"
              value={query}
              onChange={(e) => setQuery(e.target.value)}
              placeholder="Введите код или описание"
            />
          </label>
        </div>
        <table className="table">
```

Продолжение Листинга Д.1

```
          <thead>
            <tr>
              <th>Время</th>
              <th>Код</th>
              <th>Статус</th>
              <th>Описание</th>
            </tr>
          </thead>
          <tbody>
            {filteredErrors.map((e) => (
              <tr key={e.id}>
                <td>{formatTs(e.ts)}</td>
                <td>{e.code}</td>
                <td>
                                                                                <StatusBadge
status={e.severity}>{statusLabel(e.severity)}</StatusBadge>
                </td>
                <td>{formatAlertText(e)}</td>
              </tr>
            ))}
            {!filteredErrors.length && (
              <tr>
                <td colSpan={4} className="muted">
                  Нет записей или загрузка не удалась
                </td>
              </tr>
            )}
          </tbody>
        </table>
      </div>
    </div>
  );
}
```